FemboyEngine2D Complete Reference Manual

Architecture, rendering, ECS, the animation/AI canvases with blend-cycle and PIE contracts, the global audit, DOD optimization and production distribution

Version 0.1.0-alpha.1 (Alpha) - 2026-10-31 - English

Contents

- 1. Scope, conformance and engine rules
- 2. Building, targets and compiler flags
- 3. Core value types, FORCEINLINE and the cast family
- 4. FrameArena: the only frame allocator
- 5. JobSystem and per-thread diagnostics
- 6. ECS, scheduling and snapshots
- 7. RHI, render items and frame statistics
- 8. Materials: cooked graphs, the pipeline cache and the shipped path
- 9. Physics: Box2D v3, joints, cloth and SdfCollisionSystem
- 10. NetRollbackSession: deterministic rollback
- 11. Audio: mixer, DSP graph, limiter
- 12. PinkUI and WidgetCanvasSystem
- 13. Accessibility kernel (BlindMode, DeafMode, FlashGuard)
- 14. Asset pipeline, VFS and localization
- 15. Platform services: Steam, Epic Games Store, GOG
- 16. Editor: dashboard, fonts, LogBoy, Scene Diagnostics
- 17. Step-by-step: a game from an empty folder
- 18. Testing, style gate and release checklist
- 19. Known gaps and recommended next work
- 20. GOG Galaxy: provider, shim ABI and PostUpdate systems
- 21. Font pipeline: fby_fontc, .fbyfont and the stable buffer
- 22. Cloth render path: from Verlet vertices to one indirect draw
- 23. Atmosphere: weather context, fBm wind and GPU rain with SDF splashes
- 24. ECS by composition: a 2D shooter without inheritance
- 25. Multi-Draw Indirect and Texture Cooking in production
- 26. Repository layout, CMake targets and the gates
- 27. Threading contract: RHI, the OS main loop and TTS
- 28. The 32-bit DrawBatch pipeline: end of the 65,536-texture ceiling
- 29. Atomic VFS font hot reload and the strict MountFonts contract
- 30. TTS: the main-thread contract and localized voice fallback
- 31. State-Driven Gameplay: FemAnimGraph, StateTree AI and the Animation Graph Canvas
- 32. Advanced AAA DOD Pipelines: Virtual Texturing, Component Change Masks, Relevance Grids, and Audio Buses
- 33. Production Distribution: Encryption, LZMA Compression, and the Steam/EGS/GOG Release Checklist

- 34. SystemGraph: the parallel scheduling contract, then the blend-cycle contract
- 35. Box2D rollback teardown, then the PIE animation hand-off
- 36. Global audit: release-blocking findings and the gate
- 37. Profiling cache misses and vectorisation toward 8000+ FPS
- 38. Materials in the engine: cook, cache and the frame
- 39. Appendix A - generated symbol index
- 40. Appendix B - macros and platform defines

1. Scope, conformance and engine rules

This manual specifies the public surface of FemboyEngine2D: every header under Source/Engine and the editor infrastructure that a host links against. It is written against the release-candidate tree, so a symbol described here exists in the headers that ship with that tree, and the appendix index is generated from those headers on every run.

The engine targets C++26 with GCC 15 or newer and Clang 19 or newer; the Windows release candidate is built with clang-cl against the MSVC standard library. Four rules are load bearing and every public API is designed around them:

- Simulation state is plain, trivially copyable data. No virtual calls, no owning pointers and no std::string belong in a component, snapshot or network payload.
- Hot paths do not allocate. Frame scratch memory comes from FrameArena, logs go through fixed char buffers, and callbacks are function pointers plus void*, never std::function.
- Determinism is a property of the data, not of the scheduler. Every subsystem exposes a checksummed image, and NetRollbackSession can only replay what it can reproduce byte for byte.
- No RTTI in engine code: dynamic_cast and typeid are rejected by the style gate. Cast<> walks stable type ids instead (see section 3).

The style gate (Source/Tools/Dev/CheckStyle.py) enforces the naming and structure rules: PascalCase file stems, Allman braces, braced control bodies, In-prefixed parameters, Scoped-prefixed locals, no trailing line comments and no Cyrillic in engine sources.

2. Building, targets and compiler flags

One CMake tree produces three kinds of artefact: the engine static library (FemboyEngine2D::Engine), the editor library and application (FemboyEngine2D::Editor), and the offline tools. A shipped game links the engine only; Dear ImGui, ImGuizmo, ImPlot and the TOML writer never enter a shipping target.

CMake option	Default	Effect
FBY_BUILD_EDITOR	ON	Builds fby_editor, the editor app and the panel plugins.
FBY_BUILD_TESTS	OFF	Builds the ctest suite under Source/Tests.
FBY_BUILD_TOOLS	ON	FempakCooker, LocTool, SceneTool, ShaderCompiler, AotCompiler.
FBY_BUILD_PLATFORM_PROVIDERS	ON	Steam and Epic Games Store provider plugins.
FBY_RHI_BACKEND	auto	auto, vulkan, metal or null. auto selects per platform.

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=RelWithDebInfo \
      -DFBY_BUILD_TESTS=ON -DFBY_BUILD_EDITOR=ON
```

```
cmake --build build --target fby_editor
ctest --test-dir build --output-on-failure
python Source/Tools/Dev/CheckStyle.py
```

The compiler wrapper (cmake/FbyCompiler.cmake) owns the C++26 flag, the warning set and the sanitizer switches. Do not add flags per target: a target that needs a different dialect is a target that should not exist.

3. Core value types, FORCEINLINE and the cast family

Definitions.hpp is the contract header of the engine. It supplies the export macros, the assertion macros (FBY_ASSERT, FBY_VERIFY, FBY_UNREACHABLE), the stable TypeId machinery, the cast family and the component registration macros. Every other header includes it, directly or transitively.

```
#if defined(_MSC_VER)
#define FORCEINLINE __forceinline
#elif defined(__GNUC__) || defined(__clang__)
#define FORCEINLINE inline __attribute__((always_inline))
#else
#define FORCEINLINE inline
#endif
```

FORCEINLINE is the only inlining spelling in public headers. It is used for accessors, hot arithmetic and template casts; it never replaces a measured optimisation. The three families are:

Symbol	Contract
FemCast<To>(value)	Unchecked static_cast with reference collapsing. Use when the concrete type is proven by construction (factory output, storage tag). No runtime cost.
Cast<To>(object)	Checked cast. Upcast is a compile-time static_cast; downcast compares the object's GetTypeId() against StableTypeId<To>() and its registered parent chain; unrelated types fail to compile. Returns nullptr on a mismatch, never throws, never uses RTTI.
CastComponent<To>(registry, entity)	ECS spelling of Cast<>: EnTT proves the component type at compile time and the registry validates the entity. Returns nullptr when the entity lacks the component.

```
// Downcast through the stable type table, not through RTTI.
Fby::TypeObject* base = factory.Create("Sprite");
if (Fby::Sprite* sprite = Fby::Cast<Fby::Sprite>(base))
{
    sprite->Animate(dt);
}
```

```
// Registration binds Sprite to TypeObject and derives its stable id.
FBY_REGISTER_TYPE(Fby::Sprite, Fby::TypeObject);
```

StableTypeId<T>() is a consteval FNV-1a of the type's registered name (or the compiler signature when no name is pinned). A type that changes its registered name changes its id, which is why component names are part of the serialized schema and why Snapshot v4 stores a LayoutHash next to the name.

4. FrameArena: the only frame allocator

FrameArena is a bump pointer over caller-owned storage. It is reset at the start of a frame and never frees individual allocations. Systems that need transient arrays take it from SystemContext::frameArena; a system that calls operator new in its frame path is a defect, not a style preference.

```
Fby::FrameArena arena(storage);          // caller owns storage
arena.Reset();                           // once per frame
auto* items = arena.AllocateArray<MeshRenderItem>(count);
if (items == nullptr)
{
    // Arena exhausted: degrade, never allocate on the heap.
    return;
}
const std::size_t used = arena.Used();
const std::size_t capacity = arena.Capacity();
```

- Allocate() returns nullptr instead of throwing when the arena is exhausted; callers must handle the failure.
- AllocateArray<T>() static_asserts std::is_trivially_copyable_v<T>: the arena stores POD payloads only.
- Over-aligned types must pass their alignment explicitly; the default is alignof(std::max_align_t).

5. JobSystem and per-thread diagnostics

JobSystem owns a pool of std::jthread workers. Tasks are function pointers plus a context pointer plus a half-open range, so submitting a task never allocates. ParallelFor splits a range into chunks, lets every participant pull the next chunk from an atomic counter, and always runs at least one chunk on the caller, which keeps single-core machines correct and worker re-entry safe.

```
jobs.ParallelFor(entityCount, 4096, [&](std::uint32_t begin, std::uint32_t end)
{
    for (std::uint32_t i = begin; i < end; ++i)
    {
        Integrate(entities[i], dt);
    }
});
jobs.WaitIdle();
```

```
// Diagnostics: one row per thread, slot 0 is the main thread.
std::array<Fby::JobSystem::ThreadLoad, Fby::JobSystem::kMaxThreadSlots> loads{};
const std::uint32_t slots = jobs.SampleThreadLoad(loads);
```

ThreadLoad carries busyNanoseconds (wall time inside engine work), taskCount and a 16-byte name. The counters are one cache line per thread, so two workers finishing at the same time do not contend. SceneDiagnosticsPanel turns two samples into a per-thread occupancy percentage.

ThreadSlotCount() is ThreadCount() + 1 and is clamped by the fixed MCounters array; a device with more than 64 workers reports the first 64 slots.

6. ECS, scheduling and snapshots

The registry is EnTT. Systems are registered on a SystemGraph in one of five phases (PreUpdate, FixedUpdate, Update, PostUpdate, Render). SystemContext is the only parameter a system receives; it carries the registry, JobSystem, frame arena, renderer, window, VFS, AssetDatabase, audio, physics, joints, cloth, fluid, camera stack, accessibility kernel and font atlases.

```
Fby::Ecs::Schedule schedule;
schedule.Add(Fby::Ecs::Phase::FixedUpdate, "physics", &PhysicsStepSystem,
&physics);
schedule.Add(Fby::Ecs::Phase::Update, "movement", &MovementSystem, nullptr);
schedule.Add(Fby::Ecs::Phase::PostUpdate, "platform", &GogAchievementSystem,
&gog);

Fby::Ecs::SystemContext context{registry, jobs};
context.physics = &physics;
context.window = &window;
schedule.Run(Fby::Ecs::Phase::FixedUpdate, context);
```

Snapshot v4 is a canonical, checksummed byte image of the registry. Components are written by hand-written codecs when they contain runtime handles (b2BodyId, b2ShapeId); the snapshot never serializes those handles, and the restore path recreates bodies from the component fields. The schema stores the Mirror name and LayoutHash of every component, so a field change is rejected instead of silently misread.

API	Contract
GetOrAdd<T>(registry, entity)	Adds the component when missing and returns a stable reference.
ecs::SnapshotWriter / SnapshotReader	Writes and validates the canonical image; checksum is FNV-1a over the payload.
Fby::Net::NetRollbackSession	Consumes snapshot v4 and replays ticks on late input (section 10).

7. RHI, render items and frame statistics

The RHI is a hand-written Vulkan/Metal layer. Frames are recorded into persistent ring buffers; sprite items are 48-byte PODs; meshes are drawn through one indirect call per material/layer; linear-blend skinning is one compute dispatch. FrameStats exposes what the pass actually submitted, which is what SceneDiagnosticsPanel plots.

FrameStats field	Meaning
items / batches / drawCalls	Sprite items, batches and submitted draws.
meshItems / meshIndirectCalls	MeshRenderItem count and indirect draws.
skinDispatches / skinRanges	Skinning compute dispatches and ranges.
uploadedBytes	Bytes uploaded this frame.

The editor's Viewport renders into an offscreen target, not into the swapchain. EnsureViewportTarget() sizes that target in physical pixels; a target sized in points is the usual cause of a blurry Viewport on a HiDPI monitor.

8. Materials: cooked graphs, the pipeline cache and the shipped path

Material authoring and material compilation live on opposite sides of the engine boundary. `Render::MaterialGraph` is the authoring DAG: fixed capacities (128 nodes, 384 pins, 64 parameters), trivially copyable, and hashed without the editor positions. `Render::MaterialGraphCompiler` turns a validated graph into GLSL 450 text. `fby_matc` runs that compiler and glslang at build time and writes a `.femmat` payload, so a shipped game never links glslang and never parses GLSL. The full rationale is in [ADR 0002](../../adr/0002-materials-in-engine.md).

Piece	Header	Contract
`Render::MaterialGraph`	`Engine/RHI/MaterialGraph.hpp`	Authoring DAG; `Hash()` is the pipeline identity and excludes UI positions
`MaterialGraphCompiler`	`Engine/RHI/MaterialGraphCompiler.hpp`	DAG to GLSL 450 and the shader contract flags
`EncodeMaterialGraph` / `DecodeMaterialGraph`	`Engine/RHI/MaterialAsset.hpp`	`.fgraph` wire format: explicit little-endian fields, no struct memcpy
`EncodeCookedMaterial` / `DecodeCookedMaterial`	`Engine/RHI/MaterialAsset.hpp`	`.femmat` payload: 32-byte header, vertex and fragment SPIR-V, zero-copy decoding
`Render::MaterialPipelineCache`	`Engine/RHI/MaterialPipelineCache.hpp`	128 pipelines keyed by graph hash, reference counted, LRU eviction, no heap use
`Rhi::RHI::SupportsRuntimeShaderPipelines`	`Engine/RHI/RHI.hpp`	Capability query: Vulkan returns true, Null and Metal return false

```
# Author the graph once, cook it in the build step
build/bin/fby_matc assets/materials/tinted.fgraph -o
assets/materials/tinted.femmat
```

```
// `InBytes` is the .femmat payload from the VFS or a package entry.
Render::CookedMaterialView ScopedMaterial;
if (!Render::DecodeCookedMaterial(InBytes, &ScopedMaterial))
{
    return false;
}

// One cache per device: acquire at load time, release at shutdown.
const Rhi::ShaderPipelineHandle ScopedPipeline = InMaterials.Acquire(InRhi,
ScopedMaterial);

Rhi::MeshRenderItem ScopedItem;
ScopedItem.materialPipeline =
    ScopedPipeline.IsValid() ? ScopedPipeline.id :
Rhi::kDefaultMaterialPipeline;
```

- The graph hash is the pipeline identity. Editing a graph produces a new hash, so the old pipeline becomes an unreferenced cache entry instead of being mutated in place.
- The contract mask travels with the payload: generated vertex stage, fragment view projection, material push constants, world and vertex position varyings. A live entry whose hash arrives with another contract is refused, because that is a different shader.
- `Acquire()` adds a reference and `Release()` drops one. A pipeline at zero references stays resident, so a material acquired once per frame never recreates a pipeline; it is destroyed by LRU eviction, `ReleaseAll()` or `Shutdown()`.
- The cache never allocates and never formats text on the frame path; `LastError()` returns the fixed error buffer.
- Backends without runtime shader pipelines keep the engine shaders. Query `SupportsRuntimeShaderPipelines()` instead of probing for an invalid handle.

Payload	Fields	Validated on load
`.fgraph`	magic `FBYG`, version, node/pin/param counts, nodes, pins, params	magic, version, counts against capacity, exact size, node kinds, pin and param indices
`.femmat`	magic `FBYM`, version, contract mask, graph hash, vertex and fragment byte lengths	magic, version, size against the lengths, both SPIR-V modules, contract consistency

The editor keeps live editing on top of the same cache: `Fby::Editor::MaterialGraphRuntime` recompiles a graph when its hash changes, reports per-tab errors and delegates pipeline ownership to `MaterialPipelineCache`. `MaterialGraphRuntime::SetDomain` switches the live compile between the mesh and the sprite contract, and the panel's `SaveGraph`/`LoadGraph` write and read the

`.fgraph` payload the cooker consumes. A shipped build skips glslang entirely and only calls `DecodeCookedMaterial` and `Acquire`.

Sprites take materials through the third artefact: `MaterialCompileOptions::domain` selects the sprite domain, where the generated fragment stage matches the `quad.vert` varyings (`vUv`, `vColor`, `vLayer`, and `vWorld` when a node needs it) and samples the item's atlas slice through `vLayer`. `Time` is available: the sprite push block carries a material clock at offset 80, so the generated stage reads `pc.materialParams.x` exactly like a mesh shader. `fby_matc --sprite` cooks that variant. `DrawOrder::MaterialBucket` places the sprite in a bucket, `ChangeMaskRenderExtract::SetMaterialPipelines` maps buckets to cached pipeline ids, and the extractor emits one `DrawBatch` per bucket with `DrawBatch::pipeline` already resolved. The 64-bit render key carries the bucket (16 layer, 8 material, 32 depth, 8 sequence), so a material batch is never split by depth. Nodes that need the mesh view-projection (`Fresnel`, `VertexPosition`) are refused in the sprite domain with a readable reason.

9. Physics: Box2D v3, joints, cloth and SdfCollisionSystem

PhysicsSystem owns one b2WorldId and exposes it to extension modules as a packed uint32, so box2d.h never leaks into a public header. Bodies are mirrored from RigidBody2D/Collider2D components into Box2D, stepped at the fixed rate, and read back before interpolation. PhysicsJointSystem adds rope and revolute-link chains; ClothSystem integrates a Verlet grid and projects it against the same Box2D colliders.

SdfCollisionSystem contract:

Member	Contract
SetField(const SdfField&)	Installs a non-owning mdspan over a row-major grid. Negative values are solid; the field's memory is owned by the renderer/terrain, not by the system.
Step(registry, physics, fixedDt)	Samples the grid at body corners and centre, converts the gradient to a normal, applies a mass-scaled push impulse, reflects inward velocity and writes the corrected state back. Returns the number of corrected bodies.
Update(SystemContext&)	ECS convenience wrapper around Step(); a headless call without physics is a no-op.
Contacts()	Fixed array of SdfContact {entity, point, normal, penetration, impulse, sequence}.
SdfCollisionDesc	pushOut, restitution, maxImpulse, subSamples, dynamicOnly.

```
Fby::SdfCollisionDesc desc;
desc.pushOut = 0.65f;
desc.restitution = 0.05f;
desc.subSamples = 2;
sdf.Initialize(desc);
```

```
sdf.SetField(terrain.Field());
sdf.Register(graph);                    // Phase::FixedUpdate

// Viewport overlay: red vectors whose length is the penetration.
debug.Build(context, Fby::Editor::PhysicsDebugSdf, &sdf);
```

SdfCollisionSystem is not part of SystemContext. A host that wants SDF physics registers the system explicitly through Register(graph) and keeps the instance alive; the Viewport debug pass takes the same pointer.

10. NetRollbackSession: deterministic rollback

A session binds snapshot v4, CommandLog, Pcg32 and GameplayState into one fixed-capacity rollback loop. Everything after Initialize() is allocation-free: one snapshot ring, one input ring per player, a fixed entity map and fixed frame metadata. The simulation callback is a function pointer plus void*.

Method	Contract
Initialize(registry, schema, config, arena)	Allocation-free setup with a caller-owned snapshot arena.
Initialize(registry, schema, config)	Convenience overload; allocates one arena (the only allocation the session makes).
SetSimulation(fn, user) / SetCommandLog(log)	Binds the pure simulation function and the structural command log.
Start(rng)	Captures the base tick and stores the session RNG. Required before Step().
Step(localInput)	Simulates exactly one tick with prediction for missing remote inputs and captures the snapshot.
SubmitRemoteInput(player, frame)	Stores a late or current input; a frame for an already simulated tick marks a pending rollback.
ReplayPending(report)	Restores the snapshot before the earliest corrected tick, restores RNG and CommandLog cursor, re-simulates, captures new snapshots and reports changed ticks.
CompareChecksums(tick, snap, state, out)	Compares a remote report with the stored frame; the two checksums isolate a byte-level desync from a gameplay-state desync.

```
Fby::Net::NetRollbackSession::Config cfg;
cfg.playerCount = 2;
cfg.localPlayer = 0;
cfg.rollbackTicks = 64;
session.Initialize(registry, schema, cfg, arena);
session.SetSimulation(&SimulateTick, &game);
```

```
session.Start(pcg);

while (running)
{
    session.SubmitRemoteInput(remote, received);
    session.Step(localInput);
    if (session.RollbackPending())
    {
        Fby::Net::RollbackReport report;
        session.ReplayPending(&report);
        if (report.changedTicks > 0)
        {
            LogBoy().Log(Fby::Editor::LogBoySeverity::Desync, "net",
                         "rollback {}..{} changed {}", report.fromTick,
report.toTick,
                         report.changedTicks);
        }
    }
}
```

- NetInputFrame is 24 bytes of integer axes (int16 fixed point). Floats are not allowed: a float bit pattern is not guaranteed to survive a different compiler.
- Result enumerates every failure mode (NotInitialized, BufferFull, RollbackTooOld, ChecksumMismatch, ...); ResultName() is the log spelling.
- A session is non-copyable and owns its arena when the convenience overload is used.

11. Audio: mixer, DSP graph, limiter

AudioSystem owns the miniaudio device and a fixed mixer. DspGraph routes nodes into buses, supports ducking and reverb, and receives commands through a lock-free ring so the game thread never blocks the audio callback. Limiter.hpp is the final true-peak stage.

- DeafMode mutes the graph at the Master root and turns sound triggers into subtitle lines; it is a policy of the audio system, not a post-effect.
- Ogg/Opus is decoded off the audio thread; the mixer only sees cooked .fpcm buffers.

12. PinkUI and WidgetCanvasSystem

PinkUI is the in-game UI: a fixed-capacity canvas, mdspan grid layout, anchor resolution, hit testing and one-batch packing. WidgetCanvasSystem turns a WidgetCanvas into FrameOutput (draw items) without allocating; font glyphs come from a .fbyfont atlas mapped through the VFS, so the shipping runtime carries no TTF rasterizer. Glyphs are atlas-only: there is no embedded 8x8 bitmap fallback, and a text frame without a mounted font array is refused instead of drawn with the wrong layer.

Member	Contract
WidgetCanvasSystem::MountFonts(RHI&)	Mandatory cold-start stage on the main thread. Builds the GPU texture array from the VFS .fbyfont set; a failure is a PANIC log plus

	FBY_ASSERT_MSG, never a fallback.
BuildFrame(context, canvas, output)	Lays out and culls one canvas, returning BuildStatus and filling FrameOutput. A text canvas without a current mounted atlas returns FontsNotMounted and writes nothing. Allocation-free; safe to call from Phase::Render.
Update(context, canvases, output)	Builds every active canvas into one output; the host submits the result as one batch. A font-contract violation stops the pass and sets fontContractViolated.
WidgetCanvas::SetDisplayScale	Density from CameraStackSystem::DisplayScale; font sizes are rounded to whole physical pixels.
OledShiftState	Integer 2 px orbit for OLED safety; ApplyOledShift() binds it to a canvas pool.

WidgetCanvasSystem is also the accessibility surface: BlindMode reads widget labels for the narrative log, DeafMode receives subtitle widgets, and colour-vision matrices plus FlashGuard are applied to the canvas colours before packing.

13. Accessibility kernel (BlindMode, DeafMode, FlashGuard)

Accessibility is a frame policy, not an overlay. BlindMode skips the game pass, indirect draws and the skinning dispatch, then publishes a bounded narrative log. DeafMode mutes the Master bus and routes sound triggers into SubtitleLines. FlashGuard compresses sudden luminance changes. Colour-vision correction uses the published Machado 2009 matrices, and SrgbRelativeLuminance()/ContrastRatio() implement WCAG 2.2 for authoring-time validation.

Item	Contract
NarrativeEvent	128-slot SPSC ring of 160-byte text payloads; the simulation thread pushes, the speech thread pops.
ColorVisionMatrix(deficiency)	Returns the 3x3 matrix for protanopia, deuteranopia or tritanopia.
ApplyColorVision(color, matrix, strength)	Applies the matrix in linear light to an sRGB Color32.
FlashGuard::Evaluate(luminance, dt)	Returns the gain for the current frame; a large delta is compressed to the threshold.
EditorSettings::ide*	The same policies applied to the editor window itself: Machado correction, WCAG high-contrast, FlashGuard, keyboard navigation and LogBoy as the narration sink.

14. Asset pipeline, VFS and localization

Assets are addressed by relative path inside the VFS. A .fempak archive is mounted with a priority; a directory mount can shadow it, which is how the editor writes a loose file before the cooker folds it back in. Read() returns a span into an mmap that stays valid until the mount is dropped or the file is hot-reloaded.

Format	Owner	Purpose
.fempak	FempakCooker	Immutable shipped archive with an index and optional encryption.
.fbyfont	FontAtlas (PinkUI)	Baked glyph atlas: [header] [glyph records][pixels]; no runtime TTF rasterizer.
.femeta	AssetDatabase	Typed metadata, import settings and stable AssetId.
.fbin	LocTool	Compiled localization table; the editor UI itself is English-only.

The editor UI is not localizable by design: menus, panel titles, errors and the inspector are English-only. Localization applies to game content and documentation, where a .fbin table and the EN/RU manual cover the shipped languages.

15. Platform services: Steam, Epic Games Store, GOG

PlatformService is the engine-side bridge; PlatformAPI.h is a flat pure-C ABI that a store SDK plugin implements. The provider is loaded dynamically, resolves only documented entry points and converts every store-specific type at the boundary, so a game never includes steam_api.h, EOSSDK or Galaxy headers.

Provider	Status in this tree	Notes
SteamProvider.cpp	Implemented	Steamworks flat API through a function table.
EgsProvider.cpp	Implemented	Epic Online Services provider through the same C ABI.
GogProvider.cpp	Implemented (shim)	Loads galaxy64.dll / libgalaxy64.so / libgalaxy64.dylib at runtime and resolves the flat fby_gog_* shim ABI; the game ships the shim built against its SDK revision. Galaxy is a C++ interface library, so a shim is mandatory. GogAchievementSystem and GogStorageSystem write the

		binary PlatformCommandLog in Phase::PostUpdate.
fby_fontc	Implemented	Offline TTF -> .fbyfont cooker (stb_truetype, RGBA8 atlas, verified header).

```
// Pure C ABI: the provider is a shared library, not a C++ dependency.
typedef struct FbyPlatformApi {
    uint32_t version;
    void* user;
    bool (*init)(void* user);
    void (*shutdown)(void* user);
    void (*unlockAchievement)(void* user, const char* id);
    bool (*writeCloudFile)(void* user, const char* name, const uint8_t* data,
uint32_t size);
} FbyPlatformApi;
```

16. Editor: dashboard, fonts, LogBoy, Scene Diagnostics

The dashboard is the only editor surface that exists before a project is open. It installs the editor font set before the first frame, so the first window is already Inter at the physical DPI. A new project starts in Projects/ under the engine root, resolved by DefaultProjectsRoot() - never in the process working directory and never in the install directory.

Component	Contract
EditorFonts	Loads Inter and JetBrains Mono from the VFS mmap. Subpixel advances (PixelSnapH=false), auto oversampling, no bitmap fallback, dynamic atlas baked at the viewport pixel density. Legacy backends get RasterizerDensity + inverse FontGlobalScale.
ProjectDashboard	Initialize(paths, config, visible, window, vfs, fonts, uiScale): installs typography, resolves the default Projects/ location and creates it.
LogBoySystem / LogBoyPanel	Fixed 4096-record ring of 192-byte text payloads with severity Trace/Info/Warning/Critical/Desync, FNV-1a system hashes and colour+hash filtering.
SceneDiagnosticsPanel	Per-thread occupancy bars and heatmap, FrameArena bytes, MeshRenderItem count, indirect draws, draw calls and exact simulation microseconds with avg/p99/peak.
PhysicsDebugVisualizer	Fixed-buffer line/point geometry for Box2D bodies, ropes, cloth and SDF normals, drawn over the Viewport through a caller-supplied world-to-screen map.

The .fbyfont atlas is a baked PinkUI asset and cannot feed ImGui's vector rasterizer. The editor therefore loads the Inter TTF from the same VFS mount; the atlas header is still validated on startup, so a corrupt game-UI atlas is reported by the editor, not by the player.

17. Step-by-step: a game from an empty folder

This walkthrough produces a playable project with a sprite, physics, keyboard input, a PinkUI HUD and a deterministic rollback session. Every step maps to a real header from the index.

- 1. Launch the editor and create a project with New Project. The dashboard creates <engineRoot>/Projects/<name> with assets/{textures,audio,scenes}, configs/ and project.femproj.
- 2. Import a sprite into assets/textures. The watcher imports it, writes a .femeta and cooks a .ftex texture array layer.
- 3. Create assets/scenes/main.toml with one SpriteRenderer, one Transform2D and one RigidBody2D + Collider2D.
- 4. Register the systems: input in PreUpdate, physics in FixedUpdate, movement in Update, PinkUI in Render.
- 5. Give the camera a CameraStackSystem and call SetDisplayScale() from SDL_GetWindowDisplayScale so UI glyphs are crisp on HiDPI.
- 6. Add an AccessibilityKernel from project settings; BlindMode and DeafMode are read from project.femproj, not hard-coded.
- 7. Create a NetRollbackSession per match, bind a pure SimulateTick callback and feed NetInputFrame values.
- 8. Log through LogBoy: severity + system hash + fixed-format message. Open the LogBoy panel and filter by Desync while testing netcode.
- 9. Enable Physics Debug Draw in the Viewport while tuning colliders and cloth.
- 10. Cook with FempakCooker, run the suite with ctest and the style gate, then ship the engine-only target.

```
// 4-7 in code: systems, accessibility and rollback for one match.
Fby::Ecs::Schedule schedule;
schedule.Add(Fby::Ecs::Phase::PreUpdate, "input", &InputSystem);
schedule.Add(Fby::Ecs::Phase::FixedUpdate, "physics", &PhysicsStepSystem,
&physics);
schedule.Add(Fby::Ecs::Phase::Update, "movement", &MovementSystem);
schedule.Add(Fby::Ecs::Phase::Render, "ui", &UiSystem, &ui);

Fby::AccessibilityKernel kernel;
kernel.Configure(settings);              // blind/deaf/colour/flash

Fby::Net::NetRollbackSession session;
if (session.Initialize(registry, schema, cfg, arena) !=
Fby::Net::NetRollbackSession::Result::Ok)
{
    LogBoy().Log(Fby::Editor::LogBoySeverity::Critical, "net", "rollback init
failed");
```

```
}
```

18. Testing, style gate and release checklist

The suite is deliberately framework-free: every test is an executable with a main() that returns non-zero on failure, so it runs on a headless CI agent, a console devkit and a developer workstation with the same command. The full suite passes at Release.

```
ctest --test-dir build --output-on-failure          # the full suite
python Source/Tools/Dev/CheckStyle.py               # style gate
python Documentation/Tools/GenerateAPIReference.py  # this manual
python Documentation/Tools/GenerateDocumentation.py
python Documentation/Tools/GenerateApplicationGuide.py
```

- A release build enables asserts and runs the whole suite once with FBY_ENABLE_ASSERTS=1, because two latent debug-only compile errors existed in JobSystem at RC.
- The POSIX branch of WriteFileAtomically() and LogLine() must be compiled on Linux/macOS in CI; the Windows-only build never saw them.
- The style gate must report zero failures before a commit is eligible for release.

19. Known gaps and recommended next work

The live backlog, with file/line evidence and a status per item, is Documentation/ROADMAP.md. The items previously listed here (the GOG shim build step, a canvas mixing text with its own image array, std::function in the window observers/dialogs, BlindMode focused-widget narration, SceneDiagnostics simulation time and fixed-step enforcement) are tracked there; this chapter keeps no second copy.

20. GOG Galaxy: provider, shim ABI and PostUpdate systems

GOG Galaxy ships a C++ interface library, not a flat C API. The engine therefore keeps the same architecture it uses for Steam and Epic: a provider shared object speaks only PlatformAPI.h, and a thin Galaxy shim (built by the game against the exact SDK revision it ships) exposes a flat C surface that the provider resolves with dlopen/LoadLibrary at runtime. No Galaxy header enters the engine, and an SDK upgrade replaces the shim without recompiling the game.

Flat shim export	Maps to
fby_gog_initialize(clientId, clientSecret)	Galaxy::Init with a User/Listener registration.
fby_gog_process_data()	Galaxy::ProcessData, called once per Pump().
fby_gog_unlock_achievement(name)	IAchievements::SetAchievement + StoreAchievements.
fby_gog_write_cloud/read_cloud/ delete_cloud	IStorage file API; BufferTooSmall reports the required size.
fby_gog_pending_cloud_writes()	Outstanding Galaxy write listeners.


```
// The provider resolves the shim once, then every call is a pointer hop.
bool EnsureLibrary() noexcept
{
    GState.library = OpenGalaxyLibrary();          //
galaxy64.dll / .so / .dylib
    if (GState.library == nullptr)
    {
        return false;                              // honest NotAvailable
    }
    return ResolveFlat(GState.library, "fby_gog_unlock_achievement",
                       GState.api.unlockAchievement);
}

// Game side: PostUpdate turns components into logged commands.
Fby::Platform::GogAchievementSystem achievements;
achievements.Initialize(platform, commandLog, 1);
Fby::Platform::GogAchievementSystem::Register(graph, achievements);

Fby::Platform::GogStorageSystem storage;
storage.Initialize(platform, commandLog, 1);
std::span<const std::byte> save = SerializeSlot(slot);
storage.StageWrite(0x51u, "slot0.sav", save);
Fby::Platform::GogStorageSystem::Register(graph, storage);
```

Both systems run in Phase::PostUpdate. They iterate a bounded view (64 entities), sort deterministically by id and entity, append one byte-exact record to PlatformCommandLog and queue one fixed-size PlatformCommand. PlatformService::Pump() is the only caller of the shim, so a Galaxy callback can never run on a simulation thread. When the shim is absent the provider returns NotAvailable and the log keeps the intent instead of pretending the achievement unlocked.

21. Font pipeline: fby_fontc, .fbyfont and the stable buffer

Pink UI never rasterizes a TTF at runtime. fby_fontc bakes the vector outlines into the engine's atlas format, the cooker packs the blob into .fempak, and the VFS maps it on demand. The tool is the only target that links stb_truetype.

```
fby_fontc --input assets/fonts/Inter-Regular.ttf \
          --output assets/fonts/Inter.fbyfont \
          --size 32 --font-id 0 --ranges latin,cyrillic --verify

fby_fempak --input assets --output assets.fempak
```

Region	Contents
Header	48 bytes: magic FBYFNT01, version, glyph count, atlas size, format, offsets, line height, ascent/descent, font id.

Glyph table	40 bytes per glyph: codepoint, normalized UVs, advance, bearings, quad size. Sorted ascending for the binary search in FontAtlas::Glyph().
Pixels	RGBA8: white RGB with the coverage mask in A, so a label tint multiplies correctly.

Editor typography is a different consumer: ImGui needs vector outlines to bake glyphs at arbitrary DPI and cannot read .fbyfont. EditorFonts copies the TTF bytes from the VFS into an EditorFonts-owned buffer before handing the pointer to ImGui, so a hot reload that unmaps the file can never leave the atlas pointing at freed pages. The atlas header is still validated on startup, which is how a broken game-UI font is caught by the editor.

22. Cloth render path: from Verlet vertices to one indirect draw

ClothSystem integrates a Verlet grid and writes MeshVertex positions that are already final. The renderer must not skin them: with boneCount == 0 a skinning dispatch would multiply every vertex by an identity-stack default and collapse the cloth to a point. The engine therefore has exactly one classification helper and one producer:

```
// Mesh.hpp: one predicate, used by VulkanRHI and any future backend.
inline bool IsUnskinnedMeshItem(const MeshRenderItem& item) noexcept
{
    return (item.Flags & MeshItemUnskinned) != 0 || item.boneCount == 0;
}

// VulkanRHI: unskinned vertices bypass the compute dispatch entirely.
const bool unskinned = !isStatic && IsUnskinnedMeshItem(item);
memcpy(targetVertexRing + totalSkinned + unskinnedCursor,
item.vertices.data(), ...);
record.unskinned = true;   // mesh.vert reads binding 3, not the skinned SSBO

// App: ClothRenderSystem is the producer the path was missing.
Fby::ClothRenderSystem clothRender;
clothRender.Initialize(&clothSystem);
Fby::ClothRenderSystem::Register(graph, clothRender);   // Phase::PostUpdate
extract.Build(registry, camera, clear, textureArray, clothRender.Items());
```

The bridge fills a fixed 64-item array, forces MeshItemUnskinned and sorts by (sortLayer, depth, ClothId), so draw order is deterministic without an allocation. The regression test checks the mesh still spans the authored 64x64 pixels - the exact symptom of a collapsed cloth - and that a zero-bone item is classified as unskinned even when its flag is missing.

23. Atmosphere: weather context, fBm wind and GPU rain with SDF splashes

WeatherContextComponent is the single authored source of atmosphere constants and wind. WeatherSystem resolves one gust-modulated vector per frame with fBm noise and writes it back

into the component and into the std140-friendly WeatherWindField block. Cloth, plant vertices and rain all read that same vector, so nothing disagrees about where the wind blows.

```
Fby::Weather::WeatherContextComponent weather;
weather.atmosphereDensity = 1.35f;   // denser alien air
weather.airViscosity = 1.1e-5f;
weather.gravityScale = 0.82f;
weather.windSpeed = 26.0f;
weather.windDirectionX = 0.8f;
weather.windDirectionY = -0.6f;
weather.turbulence = 0.55f;
weather.rainIntensity = 0.85f;
weather.Flags |= Fby::Weather::WeatherRain | Fby::Weather::WeatherWind;

// WeatherSystem -> ClothSystem -> shader block, one resolved vector.
cloth.SetAmbientWind({weather.resolvedWindX, weather.resolvedWindY});
shaders.SetWeatherBlock(WeatherWindFieldFrom(weather));
```

WeatherPreconciliatedSystem owns no particles on the CPU. It uploads one WeatherGpuConstants block per frame and records four graph passes: integrate droplets (weather_rain.comp), age splashes (weather_rain_splash.comp), draw line segments (weather_rain.vert/frag) and draw splash sprites. The rain compute shader integrates wind, samples the global terrain SDF and appends a splash with atomicAdd when a droplet crosses the implicit surface.

```
// weather_rain.comp: one invocation per droplet, 262144 drops per dispatch.
const vec2 previous = particle.state0.xy;
particle.state0.xy += velocity * dt;

const vec2 uv = (particle.state0.xy - weather.fieldTransform.xy) / fieldSize;
const float surface = texture(uSdf, uv).r;          // negative = solid
if (surface <= 0.0 && previousSurface > 0.0)
{
    const uint slot = atomicAdd(splashBuffer.splashCount, 1u);
    splashBuffer.splashes[slot].state0 = vec4(particle.state0.xy, 0.0, 1.0);
    splashBuffer.splashes[slot].color = particle.color;
    spawnParticle(particle, randomState);              // respawn at the top
}
particleBuffer.particles[particleIndex] = particle;
```

The Metal mirror embeds the same constants and the same splash arithmetic; the MSL source is compiled into the engine at build time. The CPU cost of a storm is one constant upload plus the graph records, independent of the drop count.

24. ECS by composition: a 2D shooter without inheritance

A Player and an Enemy are not classes. They are the same entity type with different component sets, and the systems that run are selected by data. This section builds the canonical shooter from four components and three systems; adding a boss means adding components and a spawn row, not a new subclass.

```
struct HealthComponent   { float health = 100.0f; float maxHealth = 100.0f;
float invuln = 0.0f; };
struct WeaponComponent   { float damage = 10.0f; float cooldown = 0.0f; float
interval = 0.12f;
                           float projectileSpeed = 900.0f; std::uint32_t
archetype = 0; };
struct InputComponent    { std::int16_t moveX = 0; std::int16_t moveY = 0;
std::uint32_t buttons = 0; };
struct AIBehaviorComponent { std::uint32_t behaviour = 0; float aggroRadius =
400.0f;
                             float fireRange = 320.0f; std::uint32_t targetNetId
= 0; };

// Player: Transform + PhysicsBodyDesc + Health + Weapon + Input
// Enemy : Transform + PhysicsBodyDesc + Health + Weapon + AIBehavior
// Boss  : same as Enemy + phases[] + WeaponArray; no new class.
```

System	Phase	Reads / writes
InputSystem	PreUpdate	Reads InputComponent, writes it from the tick input packet.
AISystem	Update	Reads AIBehavior + Health, writes InputComponent (AI produces input, not velocity).
WeaponSystem	FixedUpdate	Reads Weapon + Input, spawns projectiles through CommandBuffer.
DamageSystem	PostUpdate	Reads collision events, decrements Health, queues despawn.

```
void SpawnEnemy(Fby::Ecs::Registry& registry, const Fby::CharacterDesc& desc,
                const Fby::PhysicsBodyDesc& body, Fby::Vector2D at)
{
    const Fby::Entity enemy = registry.create();
    registry.emplace<Fby::Transform2D>(enemy, Fby::Transform2D{at, {}, {1.0f,
1.0f}, 0.0f});
    registry.emplace<Fby::RigidBody2D>(enemy, Fby::RigidBody2D{.Position = at});
    registry.emplace<Fby::Collider2D>(enemy, Fby::Collider2D{body.halfExtents});
    registry.emplace<HealthComponent>(enemy, HealthComponent{desc.maxHealth,
desc.maxHealth, 0.0f});
    registry.emplace<WeaponComponent>(enemy,
        WeaponComponent{desc.damage, 0.0f, desc.fireInterval,
desc.projectileSpeed, desc.weaponArchetype});
    registry.emplace<AIBehaviorComponent>(enemy, AIBehaviorComponent{0, 420.0f,
300.0f, 0});
}
```

Determinism follows from the component set: the same seed, the same integer inputs and the same systems produce the same snapshot checksum. A boss that needs a phase machine adds a PhaseComponent and a system in the same phase; it does not change Player, Enemy, the weapon code or the netcode.

25. Multi-Draw Indirect and Texture Cooking in production

Sprites and meshes take two different paths through the same FramePacket. Sprites are 48-byte RenderItems sorted into batches; meshes are MeshRenderItems that become one MeshDrawData record and one indirect command each, grouped by (textureArray, layer, materialPipeline). The GPU never sees a per-object descriptor bind on this path.

```
Fby::Rhi::MeshRenderItem item;
item.vertices   = skinVertices;              // or MeshSystem::WriteMesh()
item.indices    = skinIndices;
item.boneOffset = paletteOffset;
item.boneCount  = paletteSize;               // 0 => unskinned bind pose
item.textureArray = atlas.Handle().id;       // one binding for the scene
item.Layer        = sprite.Layer;
item.model        = Fby::Mat4::Identity();

packet.meshItems = renderExtract.MeshItems();
packet.boneMatrices = renderExtract.BonePalette();
rhi.Submit(packet);                                // one indirect call per group
```

Budget	What to measure
MeshDrawData	80 bytes per draw, uploaded once per frame; firstInstance selects it.
Bone palette	One contiguous Mat4 block per frame; never per draw.
Texture array	One sampler2DArray binding for the whole scene; cooking picks BC7 (desktop) or ASTC (mobile).
SceneDiagnostics	Watch MeshRenderItem, indirect calls, draw calls and the frame arena in one panel.

```
# Texture cooking: source PNG -> .ftex (BC7/ASTC mips) -> .fempak
fby_fempak --input assets --output assets.fempak --format bc7-srgb
fby_fontc --input assets/fonts/Inter-Regular.ttf --output
assets/fonts/Inter.fbyfont

# Verify the shipping pack through the same VFS the game uses
ctest --test-dir build -R data_driven --output-on-failure
```

The editor's Project Settings panel edits the cook format and the collision matrix transactionally; both persist into project.femproj and are applied to live b2Shape filters on commit. A project that changes its target from desktop to mobile keeps the same scene and re-cooks the atlas.

26. Repository layout, CMake targets and the gates

The tree is part of the contract. CheckStyle.py now fails when a translation unit appears directly under Source/Engine or Source/Editor, when an unexpected module directory exists, or when a build log, object file, screenshot or Python cache is committed outside the disposable build tree. The module set is fixed: Core, RHI, ECS, Physics, Audio, Network, Assets, Script, UI, Video, plus the first-class Weather and Platform modules.

Target	Kind	Purpose
fby_engine	static library	Runtime; no ImGui, no TOML writer, no storefront SDK.
fby_editor / fby_editor_app	library + executable	ImGui/ImPlot panels and the IDE.
fby_fontc	tool	TTF -> .fbyfont atlas; the only stb_truetype consumer.
fby_fempak	tool	Asset cooker and archive writer.
fby_platform_steam/egs/gog	shared libraries	Storefront providers behind the C ABI.

```
python Source/Tools/Dev/CheckStyle.py                # style + layout + hygiene
cmake --build build --target fby_editor fby_fontc fby_platform_gog
ctest --test-dir build --output-on-failure        # the full suite
python Documentation/Tools/GenerateCompleteReference.py
```

Shaders still live under Source/Engine/RHI/Shaders because the engine embeds their SPIR-V at configure time; the directory is the shader module, not a loose asset folder. Moving it to a top-level Shaders/ would require changing the embed step and every shader path for no packaging benefit.

27. Threading contract: RHI, the OS main loop and TTS

Three subsystems have non-negotiable thread affinity: the RHI command stream, the window/ImGui loop and OS speech. Simulation systems may run on JobSystem workers, but workers never call the RHI, never touch the window and never create an AVSpeechSynthesizer or an ISpVoice. They only append to lock-free queues and fixed-size logs.

Entry point	Affinity	Rule
VulkanRHI::DrawPacket	main render thread	Records vkCmdBindDescriptorSets inside the batch loop; the batch carries full 32-bit pipeline and texture-array ids, 0 inherits the packet array.
WidgetCanvasSystem::MountFonts	cold path, main thread	Mandatory UI host stage. Creates the GPU texture array and may WaitIdle. A missing

		set, an empty set or a failed mount logs PANIC and trips FBY_ASSERT_MSG: there is no fallback.
WidgetCanvasSystem::BuildFrame	main render thread	Reads FontAtlasSet::IsCurrent() and the 32-bit array id. Text without a mounted array returns BuildStatus::FontsNotMounted and writes nothing; no GPU creation, no WaitIdle, no VFS remap, no allocation after Add().
FontAtlasSet::MountToTextureArray	cold path	Validates the device maxTextureLayers before touching GPU memory, publishes the new array only after every layer is built, and calls WaitIdle before destroying the previous array (no GPU use-after-free).
EditorFonts hot reload	main thread, before NewFrame	DetectFontChanges() -> RHI::WaitIdle() -> ReloadFonts(). PollForFontChanges() was removed: the combined helper destroyed the atlas without the barrier.
AccessibilityKernel::Update	simulation / worker	Builds the Narrative Tree and pushes NarrativeEvents. It never calls SAPI or AVSpeech.
AccessibilityKernel::SetBlindMode / SetTtsBackend	OS main thread (asserted)	RequireMainThread() first: a worker call is logged, dropped and trips FBY_ASSERT_MSG in an assert build. Enabling BlindMode may attach the native backend and select a voice.
AccessibilityKernel::SetNarrativeLocale	any thread	Changes the narrative tree language. A worker call only records a pending TTS language; the setVoice() is applied by FlushSpeech() on the owner thread.
AccessibilityKernel::FlushSpeech	OS main thread	Called from EditorSystem::Tick() / the SDL main loop. Applies a deferred locale, then drains the log and calls the backend speak

		callback. A worker call returns before the log is touched.
SAPI 5 (Windows)	STA main thread	CoInitializeEx(COINIT_APARTMENTTHREADED) is paired with CoUninitialize. SetVoice enumerates every installed token and scores it against the BCP-47 tag; no match falls back to the system default voice.
AVSpeechSynthesizer (macOS)	main run loop	Initialize/Speak/SetVoice assert [NSThread isMainThread]. SetVoice resolves the best installed voice language and falls back to the system default.
JobSystem workers	any worker	May run ECS systems and append to lock-free rings; may not call RHI, window, VFS remap or TTS.

```
// Cold path: app init / scene load, OS main thread. Creates the GPU array once.
ui::WidgetCanvasSystem scopedUi{&fonts};
if (!scopedUi.MountFonts(*gpu))
{
    // Panic already logged; an assert build never reaches this line.
    return Shutdown(ExitCode::UiFontsUnavailable);
}

// Hot path: Render phase, main thread. No GPU creation, no WaitIdle.
ui::FrameOutput out;
const ui::BuildStatus status = ui::WidgetCanvasSystem::BuildFrame(ctx, canvas,
out);
if (status == ui::BuildStatus::FontsNotMounted) { /* refuse the whole UI pass */
}
// out.batches[0].textureArray is the 32-bit font array id and out.items carry
the
// real layer (0 = white, 1..N = fonts); DrawPacket rebinds per batch.

// Font hot reload: the barrier is deliberate and one-shot.
if (fonts.DetectFontChanges())
{
    gpu->WaitIdle();
    fonts.ReloadFonts(ImGui::GetIO());
}

// Speech: Simulation never flushes. The editor/Main loop owns it.
void EditorSystem::Tick(ecs::SystemContext& ctx)
{
```

```
    if (ctx.accessibility) { ctx.accessibility->FlushSpeech(); }  // main thread
only
    // ... ImGui frame ...
}
```

Rule of thumb: if an operation can wait on the GPU, allocate a VkImage, close a VFS mapping or talk to a COM/Objective-C UI object, it belongs to the cold path or the OS main thread. The render and simulation hot paths only read immutable state and append to queues. A debug build turns every one of those boundaries into an FBY_ASSERT_MSG; a release build logs and drops the unsafe call instead of issuing it.

28. The 32-bit DrawBatch pipeline: end of the 65,536-texture ceiling

DrawBatch is the only place where a texture-array id and a shader-pipeline id cross the render-extract boundary. Until the RC-2 audit both fields were std::uint16_t, while every RHI handle (TextureArrayHandle, ShaderPipelineHandle) is std::uint32_t. The result was a silent alias: the 65,537th texture array or pipeline was truncated to an unrelated low id and the batch sampled the wrong resource - with no error, no assert and no way to see it in a log. The fix widens both fields to std::uint32_t and reorders them so the struct is still exactly 16 bytes.

Offset	Field	Type	Contract
0	firstItem	uint32_t	First RenderItem of the run; an index into the frame's item span.
4	itemCount	uint32_t	Number of contiguous items in the run.
8	pipeline	uint32_t	ShaderPipelineHandle ::id, 0 = the quad pipeline. No uint16 narrowing.
12	textureArray	uint32_t	TextureArrayHandle::i d, 0 = inherit FramePacket::texture Array (id 0 is the backend's built-in atlas by contract).

```
// RenderItem.hpp - 16 bytes, no padding, four batches per 64-byte cache line.
struct DrawBatch
{
    std::uint32_t firstItem = 0;
    std::uint32_t itemCount = 0;
    std::uint32_t pipeline = 0;      // ShaderPipelineHandle::id
    std::uint32_t textureArray = 0;  // TextureArrayHandle::id, 0 = inherit
};
static_assert(sizeof(DrawBatch) == 16);
static_assert(alignof(DrawBatch) == 4);
```

```
static_assert(offsetof(DrawBatch, textureArray) == 12);
static_assert(std::is_trivially_copyable_v<DrawBatch>);

// VulkanRHI::DrawPacket - the id is consumed at full width.
const std::uint32_t requested =
    batch.textureArray != 0 ? batch.textureArray : packet.textureArray;
if (requested != boundArray) { BindDescriptorFor(requested); boundArray =
requested; }

// Pink UI writes the same 32-bit id: the old "<= 0xFFFF" clamp is gone.
Output.batches[0] = rhi::DrawBatch{start, count, canvas.pipeline, fontArray.id};
```

- No narrowing cast survives on the path: a grep for 0xFFFF in the batch, WidgetCanvas and Vulkan batch loop returns nothing.
- WidgetCanvas::textureArray and WidgetCanvas::pipeline are also 32-bit; SetTextureArray and SetPipeline take std::uint32_t.
- MeshRenderItem::textureArray was already std::uint32_t; static_asserts now pin its width so a future tightening cannot re-introduce the ceiling on the mesh path.
- A host that indexes its own descriptor table from a batch id must validate it: the engine forwards the id, the backend logs and falls back to the built-in atlas for an unknown id instead of dereferencing it.

Why the struct did not grow: the two former uint16_t fields plus the 4-byte pad became two uint32_t fields. The batch is still 16 bytes, the sorted batch array keeps its density, and four consecutive batches share one cache line. Widening a resource id must never cost a cache miss in the per-frame batch loop.

29. Atomic VFS font hot reload and the strict MountFonts contract

Pink UI text is atlas-only. The VFS mmap supplies cooked .fbyfont blobs, the cold path packs them into one GPU texture array (white layer 0, font atlases 1..N), and the hot path only reads that array. The RC-2 audit found two holes in this chain: a lazy mount that let the first frame silently fall back to an 8x8 bitmap layer, and a remount that destroyed the previous array while a frame could still sample it. Both are now hard contracts.

Stage	Thread / timing	Rule
FontAtlasSet::Add / LoadFromVfs	cold path	Validates the .fbyfont header and the shared pixel size immediately, grows the mount scratch outside the frame, and bumps the set Generation.
WidgetCanvasSystem::MountFonts	cold start, main thread	Mandatory. Empty/null/failed mount -> PANIC in LogBoy + FBY_ASSERT_MSG (debug). Remounts when the set is not current.
FontAtlasSet::MountToTextureArray	cold path	Checks maxTextureLayers before allocating, builds every

		layer, publishes atomically, and calls WaitIdle before releasing the previous array.
FontAtlasSet::IsCurrent	read from the hot path	MountedGeneration == Generation. Adding an atlas invalidates the mount; the next frame is refused until MountFonts() runs again.
WidgetCanvasSystem::BuildFrame	main render thread	Text with a missing/stale/non-current set, an invalid atlas index or a foreign image array returns FontsNotMounted and writes nothing. No mounting, no WaitIdle, no allocation.
EditorFonts::DetectFontChanges	main thread, between frames	Detection only. The host then calls RHI::WaitIdle() and ReloadFonts(). PollForFontChanges() was deleted because it skipped the barrier.

```
// 1. Cold start: load -> mount -> build. Order is mandatory.
ui::FontAtlasSet fonts;
if (!fonts.LoadFromVfs(vfs, "fonts/Inter.fbyfont")) { return Fail("atlas"); }
ui::WidgetCanvasSystem ui{&fonts};
if (!ui.MountFonts(*gpu)) { return Fail("font array"); }  // PANIC + assert in
debug

// 2. Hot frame: atlas-only text, full 32-bit array id.
ui::FrameOutput out{items, keys, batches, 0};
if (ui::WidgetCanvasSystem::BuildFrame(ctx, canvas, out) ==
ui::BuildStatus::FontsNotMounted)
{
    frame.fontContractViolated = true;  // release: draw no text, never a bitmap
}

// 3. Adding a locale atlas at runtime invalidates the mount.
fonts.LoadFromVfs(vfs, "fonts/NotoSansJP.fbyfont");
if (!fonts.IsCurrent()) { (void)ui.MountFonts(*gpu); }  // remount, old array
waits idle

// 4. Editor ImGui typography: the only supported hot-reload chain.
if (editorFonts.DetectFontChanges())
{
    gpu->WaitIdle();
    editorFonts.ReloadFonts(ImGui::GetIO());  // between frames, before
NewFrame()
}
```

- VFS ownership: EditorFonts copies each TTF into stable storage before ImGui sees it, so a VFS remap can never leave the atlas pointing into an unmapped page.
- GPU ownership: FontAtlasSet waits for the device before destroying a texture array, because the previous frame may still be sampling its descriptor set.
- Failure policy: a missing atlas is loud. There is no 8x8 fallback in any build; a release build refuses the text frame and sets FrameOutput::fontContractViolated.

Known limitation, now enforced instead of hidden: a canvas that mixes text with its own image texture array cannot be one batch, because one batch samples exactly one array. BuildFrame rejects it with FontsNotMounted. The follow-up is a per-array batch split (sorted runs of items grouped by array), which needs a multi-batch FrameOutput contract and is scheduled after Release.

30. TTS: the main-thread contract and localized voice fallback

Speech is the one accessibility surface that cannot be simulated on a worker. SAPI 5 creates an apartment-threaded COM object, AVSpeechSynthesizer owns a main-run-loop object, and neither tolerates being built or driven from a JobSystem thread. The kernel now enforces that in three layers: RequireMainThread() on every entry point, FBY_ASSERT_MSG in an assert build, and a release fallback that logs and drops the unsafe call. A simulation worker can still change the narrative locale; it only records the pending language, which FlushSpeech() applies on the owner thread before the next utterance.

Entry point	Affinity	Worker call
SapiTts::Initialize / AvSpeechTts::Initialize	OS main thread / main run loop	Returns false after RequireMainThread(); assert build aborts.
SapiTts::SetVoice / AvSpeechTts::SetVoice	OS main thread / main run loop	Dropped with a log line; no COM/Objective-C call is issued.
SapiTts::Speak / Stop / SetRate	OS main thread	Dropped; SAPI stays usable for the owner thread.
AccessibilityKernel::SetBlindMode	OS main thread	Returned before touching settings or attaching the backend.
AccessibilityKernel::SetTtsBackend	OS main thread	Returned; the previous backend stays installed.
AccessibilityKernel::SetNarrativeLocale	any thread	Records a pending language; setVoice() runs later in FlushSpeech().
AccessibilityKernel::FlushSpeech	OS main thread	Returns before the narrative log is touched, so no event is stolen.
NativeTtsBackend	OS main thread	Returns nullptr; COM/AVFoundation is never initialized on the worker.

Voice selection is a locale search, not a token filter. The old implementation mapped only the first letter of the tag to Language=409/419 and returned silently for everything else, so a Russian narrative on an English OS kept an English voice (or, after a failed filter, kept whatever was already installed). SetVoice() now parses the BCP-47 primary language, resolves the full tag through LocaleNameToLCID, enumerates every installed SAPI token and scores it.

Score	Meaning	Example
4	Exact LCID match	requested ru-RU, installed ru-RU
3	Same language and same sublanguage/region	requested ru-RU, installed ru (0x0419)
2	Same primary language, different region	requested en-GB, installed en-US
0	No shared language -> system default voice	requested ru-RU on an English-only OS

```
// AccessibilityKernel::SetBlindMode - main thread only.
if (!RequireMainThread("AccessibilityKernel::SetBlindMode")) { return; }
MTtsBackend = NativeTtsBackend();   // SAPI COM / AVSpeech created here
ApplyTtsLanguage();                // setVoice on the owner thread

// A worker that changes the locale never reaches the backend.
void AccessibilityKernel::SetNarrativeLocale(std::string_view locale) noexcept
{
    // ... update MTtsLanguage ...
    if (IsMainThread()) { ApplyTtsLanguage(); } else { MPendingVoiceLanguage =
true; }
}

// The owner thread applies it before draining the log.
void AccessibilityKernel::FlushSpeech() noexcept
{
    if (std::this_thread::get_id() != MMainThread) { return; }  // drop, do not
steal
    if (MPendingVoiceLanguage) { ApplyTtsLanguage(); }
    // ... speak NarrativeEvents ...
}

// SAPI fallback: score 4/3/2, else the system default. Never silent.
LogWarn("tts", "no installed voice matches '{}'; falling back to LCID 0x{:04X}",
        tag, defaultLcid);
```

- The SAPI implementation no longer includes sphelper.h: it uses ISpObjectTokenCategory directly, so the engine build does not depend on the ATL component (a machine without atlbase.h still builds).

- AVSpeech enumerates [AVSpeechSynthesisVoice speechVoices], scores the language prefix and stores the best installed language; the system default is the fallback.
- Every decision is logged once with the requested tag, the selected LCID/language and whether it was a fallback, so a support ticket can say what the player actually heard.

Debug builds compile FBY_ASSERT_MSG into every guard listed above. A call from the wrong thread stops in the debugger with the exact function name instead of producing an intermittent COM failure or a silent run loop corruption. Release builds keep the guard semantics but not the abort: they log and refuse the call.

31. State-Driven Gameplay: FemAnimGraph, StateTree AI and the Animation Graph Canvas

The high-level gameplay layer follows the same rule as the renderer: behaviour is a function over plain data, not a graph of objects. FemAnimGraph is a flat animation state machine and FemStateTree is a flat AI state tree. Neither one has a base class, a virtual Evaluate(), a per-node heap allocation or a std::function callback. An entity owns a fixed-size POD component; the system owns the shared tables (the animation clips and the AI node/transition arrays) and walks only the active path.

Why not a Behaviour Tree: a BT re-walks its hierarchy from the root every tick, so the cost of an idle NPC is proportional to the size of its tree. A State Tree stores the active path once and re-evaluates one guard; transitions are a flat, trigger-indexed array. The same reasoning applies to AnimGraph: transition conditions are bit tests and float comparisons on the component, and the expensive part - the bone pose - is blended once per tick, never per graph node.

AnimStateComponent field	Type	Role
CurrentState	uint64_t	Opaque logical state id. 0 = rest pose; gameplay owns the numbering.
PreviousState	uint64_t	Source pose of the active cross-fade.
StateTime	float	Seconds in the current state; reset on every fired transition.
TransitionCount	uint32_t	Diagnostics. Placed at offset 20 to fill the pad before the 8-byte-aligned transition array.
Transitions[8]	AnimTransition	Authored transitions, evaluated in array order; first match wins. Inert when Conditions == AnimConditionNone.
Clips[8] / ClipCount	AnimClip	Per-entity clip table: bone keys plus the SpriteAnimation frame range for the same state.
Floats[8]	float	Float parameters (speed, health, aim) read by the

		transition conditions.
RestPose[128]	AnimRestBone	Captured once from the skeleton; a clip only authors the bones it moves.
BlendDuration / BlendTime / BlendWeight	float	Cross-fade timing and the eased weight the SkeletalAnimation bridge applies.

Condition flag	Evaluator
AnimConditionStateTimeAtLeast	StateTime >= StateTimeThreshold
AnimConditionStateTimeAtMost	StateTime <= StateTimeThreshold
AnimConditionTagsAny	(GameplayState.Bits & TagsAny) != 0
AnimConditionTagsAll	(GameplayState.Bits & TagsAll) == TagsAll
AnimConditionTagsNone	(GameplayState.Bits & TagsAny) == 0
AnimConditionFloatAtLeast / AtMost	Floats[FloatParam] compared with FloatThreshold; a non-finite value fails closed.

```
// AnimGraph.hpp - the component prefix is stable and visible in an inspector.
struct AnimStateComponent
{
    std::uint64_t CurrentState = 0;   // 0  logical state, 0 = rest
    std::uint64_t PreviousState = 0;  // 8  cross-fade source
    float StateTime = 0.0f;           // 16 seconds in state
    std::uint32_t TransitionCount = 0;// 20 diagnostics / pad
    std::array<AnimTransition, 8> Transitions{};  // 24
    std::array<AnimClip, 8> Clips{};
    std::array<float, 8> Floats{};
    std::array<AnimRestBone, kMaxBones> RestPose{};
    float BlendDuration = 0.0f, BlendTime = 0.0f, BlendWeight = 1.0f;
    // ...
};

// Once per fixed tick: conditions -> state -> two bridges.
for (auto [entity, state] : registry.view<AnimStateComponent>().each())
{
    state.StateTime += dt;
    const std::uint64_t tags = gameplayBits(registry, entity);
    for (auto& t : state.Transitions)
    {
        if (t.FromState != kAnimAnyState && t.FromState != state.CurrentState)
continue;
        if (!AnimTransitionMatches(t, state, tags)) continue;
        state.PreviousState = state.CurrentState;
        state.CurrentState = t.ToState;
        state.StateTime = 0.0f;
```

```
        state.BlendDuration = t.BlendDuration;
        break;   // first match wins, at most 8 comparisons
    }
    // SkeletalAnimation: blend local TRS, then one hierarchy pass.
    if (auto* skeleton = registry.try_get<SkeletonComponent>(entity))
        AnimGraphApplySkeletal(state, *skeleton);
    // SpriteAnimation: only on a state change, so the frame advance is not
fought.
    if (changed) AnimGraphApplySpriteClip(state, spriteAnimation, grid,
spritePtr);
}
```

The SkeletalAnimation bridge blends in local TRS space - shortest-arc rotation, linear position and scale - and then calls SkeletalAnimation::Evaluate() exactly once. That single hierarchy pass rebuilds globalMatrices and the Mat4 skinningMatrices palette that the GPU consumes. Lerping two composed Mat4 palettes would be cheaper by one pass but produces shear on a rotating parent/child chain and is not a rigid transform; the local blend keeps every bone rigid. The SpriteAnimation bridge is the opposite trade-off: it is instantaneous by design (startFrame, frameCount, duration, currentFrame = 0, UV applied in the same tick), because a flipbook has no interpolatable pose.

StateTreeComponent field	Type	Role
ActiveStates[4]	uint16_t	Dense root-first path: [0] root, last = active leaf, unused = kStateTreeNone. A State Tree never stores the whole node list per entity.
CurrentLeaf	uint32_t	Node index of the active leaf (also ActiveStates[ActiveDepth - 1]).
ActiveDepth	uint32_t	1..4; validated by StateTreeAddNode so BuildPath cannot overrun.
Triggers	uint64_t	Pending change triggers. 0 -> the transition table is not touched at all this tick. Bit 63 is reserved for the internal leaf-guard-failed event.
Tags	uint64_t	Snapshot of GameplayState.Bits for this tick; every evaluator sees the same mask.
Target / TargetPosition	Entity / Vector2D	Blackboard: a live entity target, with a cached position fallback for patrol points.
Floats[4]	float	Float blackboard read by FloatBelow / FloatAbove

		(health, threat, ...).
StateTime / Ticks	float / uint32_t	Time in the current leaf and fixed ticks since entry; reset by BuildPath.
TreeId	uint32_t	Selects a StateTreeDefinition owned by the system; many entities share one tree.

Flat table	Contents	Cost per tick
StateTreeDefinition::Nodes[64]	Root/Selector/Leaf kind, parent, intrusive child list, depth, evaluator, task.	Not walked in steady state.
StateTreeDefinition::Transitions[32]	FromLeaf, ToNode, TriggerMask, ConsumeMask, RequireTagsAll, ForbidTagsAny.	Scanned only when Triggers != 0.
StateTreeEvaluator	Flat enum + uint64 tag mask + float threshold + float index, switched in one function.	One call for the active leaf.
StateTreeTask	None / MoveToTarget / Stop / AttackTarget / SetTag / ClearTag.	One switch; movement and attack go through CommandBuffer.

```
// Authoring: a fixed table, no allocations, depth enforced at AddNode().
StateTreeDefinition tree; tree.TreeId = 7;
const auto root  = StateTreeAddNode(tree, StateTreeNodeKind::Root);
const auto group = StateTreeAddNode(tree, StateTreeNodeKind::Selector, root);
const auto attack = StateTreeAddNode(tree, StateTreeNodeKind::Leaf, group);
const auto move   = StateTreeAddNode(tree, StateTreeNodeKind::Leaf, group);
StateTreeSetLeaf(tree, attack,
                 StateTreeEvaluator{StateTreeEvaluatorKind::TargetWithinRadius,
0, 64.0f, 0},
                 StateTreeTask{StateTreeTaskKind::AttackTarget, 0, 0.0f, 48.0f,
0});
StateTreeSetLeaf(tree, move,
                 StateTreeEvaluator{StateTreeEvaluatorKind::AlwaysTrue, 0, 0.0f,
0},
                 StateTreeTask{StateTreeTaskKind::MoveToTarget, 0, 120.0f, 0.0f,
0});
StateTreeTransition spotted{};
spotted.FromLeaf = move;
spotted.ToNode = attack;
spotted.TriggerMask = 1ull << 0;   // "player spotted"
spotted.ConsumeMask = 1ull << 0;
StateTreeAddTransition(tree, spotted);
```

```
// Runtime: the path is data, the transition table is consulted only on a
trigger.
void StateTreeSystem::Run(ecs::SystemContext& ctx) const noexcept
{
    for (auto [entity, state] : ctx.registry.view<StateTreeComponent>().each())
    {
        const auto* tree = Tree(state.TreeId);
        if (!state.Active || tree == nullptr) continue;
        state.Tags = gameplayBits(ctx.registry, entity);
        ++state.Ticks; state.StateTime += ctx.dt;
        if (!state.Initialized) { BuildPath(*tree, SelectLeaf(...), state); }
        else if (!GuardPasses(state, *tree)) state.Triggers |=
kStateTreeTriggerLeafFailed;
        if (state.Triggers != 0) TryTransitions(*tree, state);  // flat, <= 32
entries
        RunTask(ctx, entity, state, *tree);  // emits AICommandComponent
    }
}
```

- No inheritance, no virtual dispatch: StateTreeNodeKind, StateTreeEvaluatorKind and StateTreeTaskKind are enums switched in one translation unit. A plugin adds a new evaluator by extending the enum and the switch, not by subclassing a node.
- No per-frame tree walk: with no pending trigger the system reads one guard and runs one task. A Behaviour Tree tick would touch every branch of the hierarchy.
- No runtime allocation: nodes, transitions, the active path and the command payloads are fixed arrays; CommandBuffer copies the tiny AICommandComponent into the frame arena and Play() applies it after the fixed steps.
- Determinism: Phase::FixedUpdate + fixed dt + insertion-order command replay. Network rollback can reproduce the same ActiveStates path from the same triggers.

The runtime tables above are authored by AnimationGraphCanvasPanel, the ImNodes editor in Source/Editor/Panels. The panel is intentionally not a second runtime model: its document is a trivially copyable AnimationGraphDocument registered as one CommandStack external buffer. A clip node compiles to AnimClip, a float-parameter node compiles to AnimStateComponent::Floats, a direct clip-to-clip link compiles to AnimTransition, and a blend node compiles to the AnimTransition of the Idle -> Attack style cross-fade. The runtime never sees ImNodes, positions, names or the editor's blend topology.

Editor node	Pins	Compiles to
Clip	In, Out	AnimClip (State, Keys[8], SpriteAnimation start/count/duration/loop/play).
Float Parameter	Float out	AnimStateComponent::Floats[ParameterIndex] = ParameterValue; the index is also used by AnimConditionFloatAtLeast/At Most.
Blend	Src, Mod, Out	One AnimTransition from the

		source clip to the target clip; BlendDuration and the condition bitmask live on the blend node, the optional Mod binding supplies FloatParam.

AnimationGraphDocument field	Type / capacity	Role
GraphId	uint32_t	Identity used by the scene TOML block and by the PIE binding.
NodeCount / Nodes[24]	uint16_t / AnimationGraphNode	Fixed union-like POD node: kind, enabled, name[32], position, AnimClip, AnimTransition blend payload, parameter index/value.
LinkCount / Links[32]	uint16_t / AnimationGraphLink	Kind, endpoints, pins and the direct AnimTransition payload. The four link kinds are Transition, BlendSource, BlendParameter and BlendTarget.

```
// CompileGraph(): fixed storage, one flat AnimStateComponent, no allocation.
AnimStateComponent out;
std::array<bool, kMaxAnimParams> paramUsed{};
std::array<std::uint64_t, kMaxAnimClips> stateIds{};
std::uint16_t clipCount = 0, transitionCount = 0;
for (node : document.Nodes)
    if (node.Kind == Clip)
    {
        if (node.Clip.State == 0 || duplicate(stateIds, node.Clip.State))
reject();
        stateIds[clipCount] = node.Clip.State;
        out.Clips[clipCount++] = node.Clip;
    }
    else if (node.Kind == FloatParameter)
        out.Floats[node.ParameterIndex] = node.ParameterValue;
// Isolated-node check: every node needs at least one incident link.
for (node : document.Nodes) if (degree[node] == 0) reject("isolated");
// Direct links become transitions; each Blend needs one source, one target
// and at most one parameter binding.
for (link : document.Links)
    if (link.Kind == Transition && conditionsValid(link.Transition))
        out.Transitions[transitionCount++] = link.Transition;
for (blend : document.Nodes)
    if (blend.Kind == Blend)
```

```
        out.Transitions[transitionCount++] = makeTransition(blend, source,
target);
for (i = transitionCount; i < 8; ++i)
    out.Transitions[i].Enabled = false;   // inert slots, not garbage
out.ClipCount = clipCount;
out.TransitionCount = 0;                  // runtime diagnostic, not an authored
count
```

Every gesture is one CommandStack transaction. The document is a single external buffer, so a node drag records only the changed position bytes and a condition toggle records only the changed condition word. The ImNodes callbacks never call Record() directly: DrawCanvas reports a state change, ProcessCanvasEvents opens BeginEdit(), the edit method mutates the document, RecordEdit() captures the new bytes, and CommitEdit() commits one undo group. Ctrl+Z/Ctrl+Y are handled by EditorSystem, which first commits any open canvas transaction and then asks CommandStack to apply the inverse delta. Inspector edits use the Mirror-backed AnimationGraphTransitionFieldsEqual()/_ClipFieldsEqual() helpers; both compare only the FieldDesc byte ranges through reflect::FieldBytesEqual(), so padding bytes cannot create an empty undo command.

Condition bit	Inspector control	Runtime evaluator
AnimConditionStateTimeAtLeast	StateTimeAtLeast checkbox + StateTimeThreshold	StateTime >= threshold
AnimConditionTagsAll	TagsAll checkbox + TagsAll mask	(tags & TagsAll) == TagsAll; empty mask fails closed
AnimConditionFloatAtMost	FloatAtMost checkbox + FloatParam + FloatThreshold	Floats[FloatParam] <= FloatThreshold

```
# Appended by "Compile Animations"; EditorSystem re-appends it after Ctrl+S.
[anim_graph]
version = 1
graph_id = 12
node_count = 4
link_count = 4

[[anim_graph.node]]
kind = 0                        # Clip
name = "Idle"
position = [-260.0, 40.0]
clip_state = "0x0000000000000001"
clip_key_count = 1
clip_sprite_count = 4
[[anim_graph.node.key]]
bone = 1
rotation = 0.5
position = [0.0, 0.0]
scale = 1.0

[[anim_graph.node]]
```

```
kind = 2                        # Blend
blend_duration = 0.2
blend_conditions = 64           # FloatAtMost/AtLeast family
blend_float_param = 0

[[anim_graph.link]]
kind = 0                        # Transition
from = 1
to = 0
t_conditions = 1                # StateTimeAtLeast
t_state_time = 1.0
t_duration = 0.15
```

PIE debugging is a read-only projection of the live component back onto the authored document. RefreshPIEState() finds the selected entity (EditorSelected), reads AnimStateComponent::CurrentState/PreviousState and maps those 64-bit ids to Clip nodes; LastTransition maps through the compiler's transition-to-link table to the exact authored link. The current node is drawn with a pulsing neon outline and a ProgressBar for BlendWeight, the PreviousState is drawn in the cross-fade source colour, and BlendTime/BlendWeight are shown both on the canvas and in the Mirror inspector. No debug draw allocates; it only reads the component and writes ImGui draw commands.

Both systems are render-agnostic: AnimStateComponent writes only skeleton/SpriteAnimation data and StateTreeComponent writes only AI intent. The Metal backend now consumes the same 32-bit DrawBatch contract as Vulkan - per-batch setFragmentTexture: and a vertex-buffer offset instead of baseInstance - so the batch a State Tree or AnimGraph frame produces looks identical on both APIs.

32. Advanced AAA DOD Pipelines: Virtual Texturing, Component Change Masks, Relevance Grids, and Audio Buses

Chapter 32 documents the four pipelines added to the Release build. They share one constraint: a frame may not allocate, may not walk a scene graph and may not upload a resource that did not change. VirtualTextureSystem turns the world into VFS-mapped chunk files and a fixed 2048 texture array; ChangeMaskRenderExtract turns a 64-bit per-entity word into coalesced SSBO ranges; RelevanceGrid turns a spatial mdspan into a per-player snapshot filter; AudioZoneSystem turns a Box2D sensor volume into a lock-free bus filter ramp. Each subsystem has a fixed-capacity data layout and a headless acceptance test.

Pipeline	Mutable state	Bounded by	Per-frame allocation
VirtualTextureSystem	Page residency table + VFS mappings	layerCount x 32 x 32 pages	0
ChangeMaskRenderExtract	Stable entity slots + sorted output cache	Capacity template parameter	0
RelevanceGrid	mdspan cells + one 64-bit bitset per plan	cellsX x cellsY <= 4096	0
AudioZoneSystem	One context record	kMaxZones = 64	0

	per listener	authored zones	

31.1 Virtual texturing. The world address space is split into 64x64 tiles. Four tiles per axis form one 256x256 RGBA8 chunk file, so a single VFS mapping supplies sixteen tile uploads. The GPU owns a fixed 2048x2048 texture array with N layers; a layer is a 32x32 page pool and every page is one tile. The resident bytes are therefore exactly

```
// Constant VRAM, independent of the camera path or the world size.
constexpr std::uint64_t kPageBytes = 2048ull * 2048ull * 4ull;
const std::uint64_t Budget = layerCount * kPageBytes;

// Residency is a two-pass clock over a fixed page table.
void VirtualTextureSystem::Update(const VirtualTextureCamera& InCamera) noexcept
{
    ++MFrame;
    const float HalfX = InCamera.ViewSize.X * 0.5f / InCamera.Zoom;
    const float HalfY = InCamera.ViewSize.Y * 0.5f / InCamera.Zoom;
    const float X = std::fabs(std::cos(InCamera.Rotation));
    const float Y = std::fabs(std::sin(InCamera.Rotation));
    const float ExtentX = HalfX * X + HalfY * Y;   // rotated AABB
    const float ExtentY = HalfX * Y + HalfY * X;
    for (std::int64_t Ty = MinY; Ty <= MaxY; ++Ty)
        for (std::int64_t Tx = MinX; Tx <= MaxX; ++Tx)
        {
            const std::uint32_t Layer = LayerForTile(Tx, Ty);
            const std::uint32_t Hash = FindHash(Tx, Ty);
            if (Hash != kHashCapacity)
            {
                MSlots[Layer * 1024u + MHash[Hash].slot].lastUsedFrame = MFrame;
                continue;
            }
            std::uint32_t Slot = 0;
            if (LoadTile(Tx, Ty, Layer, &Slot))   // mmap chunk, upload one
64x64 tile
            {
                MSlots[Layer * 1024u + Slot].lastUsedFrame = MFrame;
                ++MStats.uploads;               // one vkCmdCopyBufferToImage
            }
        }
}
```

Chunk header field	Offset	Value	Validation
magic	0	FBYVTC01	Rejected before the mapping is indexed
version	8	1	A future format fails closed
tileSize	12	64	Must equal the descriptor

chunkTiles	16	4	Must equal the descriptor
format	20	0 = RGBA8	No runtime decode
pixels	32	256 x 256 x 4	Exact size check before subspan

- Eviction is a per-layer clock. A page whose lastUsedFrame is the current frame is visible and is never a victim; if all 1024 pages of a pool are visible the request is counted as a spill instead of corrupting a tile that is on screen.
- The resident lookup is open addressing with tombstones. A hit costs one hash probe and a store into lastUsedFrame; a chunk mapping is never re-read for a resident tile.
- VirtualTextureSystem itself is about 2 MiB (page table plus the 65536-entry hash). It is a startup object, allocated once on the heap, not a stack local in a frame function.

31.2 Component change masks. A render-visible entity carries ComponentChangeMask next to Transform2D/Sprite2D. A mutating system calls change::MarkTransform(), MarkSprite() or MarkDrawOrder(); the mark is one atomic fetch_or on the 64-bit word. The extract peeks the word, consumes it with fetch_and(0), and recomputes a RenderItem only when one of the three render bits was present. An entity with a zero mask reaches one pointer lookup and one relaxed load - no Transform read, no Sprite copy, no RenderKey arithmetic.

Bit	Name	Consumer	Effect
0	Transform2D	ChangeMaskRenderExtract	Rebuild position/scale/rotation; key unchanged
1	Sprite2D	ChangeMaskRenderExtract	Rebuild UV/size/colour/layer
2	DrawOrder	ChangeMaskRenderExtract	RenderKey may move: structural rebuild
3-9	Velocity/ GameplayState/ Animation/Audio/ Physics/Lifetime/ Custom	Gameplay systems	Consumed by the owner; the GPU sees nothing
16-63	GameBitBase + n	Game code	Reserved for project aliases

```
struct ComponentChangeMask          // 16 bytes, align 8
{
    ChangeMask   Bits      = 0;     // offset 0, atomic_ref target
    std::uint32_t LastFrame = 0;    // offset 8, diagnostic only
    std::uint32_t Reserved  = 0;    // offset 12, zero
};
static_assert(sizeof(ComponentChangeMask) == 16);
```

```
// A system marks after the write, not before: the extract must never see a bit
// for a value it has not finished writing.
transform.Position = NewPosition;
Fby::Change::MarkTransform(registry, entity, frameIndex);

// Extract: static items fall through after one relaxed load.
for (auto [e, mask, transform, sprite] : view.each()) {
    const ChangeMask Bits = change::Peek(mask);
    if (Bits == 0 && SlotKnown(e)) { ++stats.skippedStatic; continue; }
    const ChangeMask Hit = change::Consume(mask);
    if ((Hit & kRenderBits) == 0) { continue; }
    BuildItem(registry, e, &items[slot], &keys[slot]);
}
```

```
// Output-relative dirty ranges coalesced in one sorted pass.
struct DirtyRange { std::uint32_t first; std::uint32_t count; };
static_assert(sizeof(DirtyRange) == 8);

// Transform-only edit: key unchanged, order unchanged, patch one output item.
MOutputItems[MOutputIndex[slot]] = MItems[slot];
MChangedScratch[MChangedCount++] = MOutputIndex[slot];
// ... sort + coalesce into 1..N ranges; the backend uploads only those bytes.

// DrawOrder edit: the item can move in the sort. The safe upload is the whole
// sorted range, because firstItem in every DrawBatch may have shifted.
if (OldKey != NewKey) { RebuildOutput(); return DirtyRange{0, MOutputCount}; }
```

FillPacket() publishes the same ranges as FramePacket::instanceRanges and sets instanceRangesAreDeltas. VulkanRHI copies only those ranges into its persistent per-frame instance slice once that slice has received one full image; a first use, a resize or a structural rebuild falls back to the full copy. MetalRHI keeps the full-copy fallback, so the packet contract is portable while the optimisation is explicit.

- No allocation: every slot, key, batch, dirty range and lookup bucket lives inside the Capacity-templated object. The default 4096 covers a large 2D scene; a streaming world instantiates ChangeMaskRenderExtract<1u << 18> without changing a line of API.
- The extract exposes both arrays: Items()/Keys()/Batches() fill the existing FramePacket, while DirtyRanges() is the upload plan for a backend that owns a persistent instance SSBO. A zero-mask frame with no structural change produces an empty range list.
- A layer change is deliberately treated as structural even when the float depth did not move: batching groups by the top 16 RenderKey bits, so a stable order is a correctness condition, not an optimisation.

31.3 Network relevance grid. NetRollbackSession owns the stable NetEntityId map; RelevanceGrid owns an mdspan of cells. The grid is rebuilt once per tick: entities are sorted by NetEntityId, their Transform2D position selects a cell, and each cell appends a fixed number of ids. The plan for one player walks only the square of cells inside farRadius. Current and neighbouring cells are immediate; the near ring is gated by nearInterval; the far ring by farInterval. The phase is NetEntityId::value % interval, so distant objects do not all update on the same tick.

Band	Chebyshev distance	Cadence	Typical payload
Immediate	0..neighborRadius (default 1)	every tick	player, nearby actors, doors
Near	neighborRadius+1..nearRadius	every nearInterval ticks	mid-distance props
Far	nearRadius+1..farRadius	every farInterval ticks	ambient movers, VFX anchors
Outer	> farRadius	never	not in the plan

```
// The cell grid is an mdspan over one flat vector<RelevanceCell>.
std::mdspan<RelevanceCell, std::dextents<std::size_t, 2>>
    Grid(MCells.data(), MConfig.cellsY, MConfig.cellsX);

// Rebuild once per tick, deterministically sorted by NetEntityId.
InEntities.ForEach([&](NetEntityId Net, Entity Local) noexcept {
    MSortScratch[Count++] = SortableEntity{Net.Value, Local};
});
std::sort(MSortScratch.begin(), MSortScratch.begin() + Count,
          [](const auto& A, const auto& B) { return A.id < B.id; });
for (const SortableEntity& E : span(MSortScratch.data(), Count)) {
    const Transform2D& T = InRegistry.get<Transform2D>(E.local);
    Grid[CellY(T.Position), CellX(T.Position)].Append(NetEntityId{E.id});
}

// Plan: walk only cells inside farRadius; the bitset is O(1) membership.
const RelevanceBand Band = BandFor(std::abs(Cx - PlayerCx), std::abs(Cy -
PlayerCy));
const std::uint32_t Interval = Band == RelevanceBand::Immediate ? 1u
    : (Band == RelevanceBand::Near ? Config.nearInterval : Config.farInterval);
if (!IsDue(Interval, Id, Tick)) { continue; }
OutPlan.items[OutPlan.count++] = RelevanceEntity{Id, Cx, Cy, Band};
OutPlan.Set(Id);
```

```
// Filtering is a record copy, not a registry walk. The output is a valid
// Snapshot v4 delta against the same base: checksum and counts are rebuilt.
for (std::uint32_t R = 0; R < View.header.entityCount; ++R) {
    const FilteredRecordHeader& Rec = ReadRecord(View.payload, Offset);
    const bool Removed = (Rec.Flags & SnapshotEntityRemoved) != 0;
    const NetEntityId Net = MEntities.IdOf(Rec.entity);
    const bool Keep = Removed ? InKeepRemovals : InPlan.Contains(Net);
    if (Keep) { CopyRecord(Rec, Out); ++Out.header.entityCount; }
}
Out.header.checksum = Fnv1a(OutPayload);
// Client: RestoreDelta(base, filtered) - each record still restores the
// component bytes that the server captured for that tick.
```

RelevancePlan field	Size	Purpose

player, tick	16 B	Identifies the wire view
items[4096]	64 KiB	Sorted RelevanceEntity records
bits[64]	512 B	One bit per NetEntityId for O(1) delta filtering
immediate/near/far counts	12 B	Diagnostics and bandwidth budgeting

31.4 Hierarchical audio buses. AudioZoneComponent is the authored Box2D sensor volume: an AABB copied from the shape, a stable zoneId, a priority, a bus mask and the two target parameters ReverbGain and LowPassHz. AudioContextComponent is the listener's resolved state. AudioZoneSystem picks the highest-priority containing zone, interpolates the context in game time with a one-pole ramp and emits at most three fixed commands (Master, SFX, Music). The audio thread then ramps the actual coefficient per sample inside ApplyFilter and the reverb return gain inside ApplyReverb; neither function allocates, locks or calls into ECS.

Bus	Flag	Zone parameter	Audio-thread stage
Master	bit 0	ReverbGain, LowPassHz	ApplyFilter -> ApplyReverb -> ApplyBusGain
SFX	bit 1	ReverbGain, LowPassHz	Same chain, independently
Music	bit 2	ReverbGain, LowPassHz	Same chain, independently
Cave (example)	Master+SFX	0.45, 900 Hz	A 0.1 s context fade, then a sample ramp
Underwater (example)	All	0.20, 450 Hz	Priority 9 wins over Cave

```
struct AudioZoneComponent                 // authored, trivially copyable
{
    Vector2D min, max;                   // Box2D sensor AABB, world units
    std::uint32_t zoneId, kind, priority, busMask, flags;
    float reverbGain;                    // 0..4, return gain
    float lowPassHz;                     // 20..20000 one-pole cutoff
    float fadeSeconds;                   // context cross-fade time
};

struct AudioContextComponent              // resolved per listener
{
    std::uint32_t activeZoneId, previousZoneId, activeKind, flags;
    float blend, reverbGain, lowPassHz;
    float targetReverbGain, targetLowPassHz, fadeSeconds;
    Vector2D listenerPosition;
    float lastSentReverbGain, lastSentLowPassHz;
```

```
    std::uint32_t lastBusMask;
};

// Zone system: retarget and ramp the game-side context.
const float Alpha = 1.0f - std::exp(-dt / Fade);
Context.reverbGain += (Context.targetReverbGain - Context.reverbGain) * Alpha;
Context.lowPassHz  += (Context.targetLowPassHz  - Context.lowPassHz)  * Alpha;
Context.blend = Saturate(Context.blend + dt / Fade);
Audio.SetBusZoneParameters(AudioBus::Sfx, Context.reverbGain, Context.lowPassHz,
Fade);
```

```
// Audio thread: coefficient and reverb gain are ramped per sample.
const float Start = ScopedCoefficient(CurrentCutoff);
const float End   = ScopedCoefficient(TargetCutoff);
for (std::uint32_t F = 0; F < Frames; ++F) {
    const float C = Start + (End - Start) * T(F);
    State[0] += C * (Buffer[F * 2 + 0] - State[0]);
    State[1] += C * (Buffer[F * 2 + 1] - State[1]);
    Buffer[F * 2 + 0] = State[0];
    Buffer[F * 2 + 1] = State[1];
}
// Reverb return: value = wet * wetParams * zoneGainRamp(F).
// Command ring: SetBusFilter, fixed size, no lock on the producer side.
```

The four pipelines are covered by the headless advanced_pipelines test. It mounts a real directory, writes real .fvtc chunks and verifies that a second view produces zero uploads; it checks that a static entity produces an empty dirty range; it builds a real Snapshot v4 delta and verifies that FilterDeltaForPlayer keeps one of two records; and it drives a priority-9 underwater zone to 450 Hz and back to open. No GPU or audio device is opened, so the test runs on a headless CI agent.

33. Production Distribution: Encryption, LZMA Compression, and the Steam/EGS/GOG Release Checklist

Chapter 33 closes the distribution gap between the isolated console tools and the editor. Before this chapter the tree had fby_fempak (cook, secure, list, extract), FemArchiver (LZMA/FbyLZ plus ChaCha20-Poly1305/AES-256-GCM), FemCrypto, AotBuilder and the per-panel 'Cook AOT' action, but no single operation that ran them as one release. The ReleaseCooker class now owns that operation: one Build & Pack Game button starts the native logic plugin, the ordered asset cook, the deterministic .fempak writer and the encrypted .femarc wrapper, then publishes progress to SceneDiagnosticsPanel.

Stage	Command / class	Writes	Failure contract
Key	ReleaseCooker::PrepareKey	.fbyrelease.seed, release.key.tmp, ReleaseKey.generated.hpp	No plaintext key in project.femproj or editor.ini
AOT	script::AotBuilder +	logic_graph.cpp / .dll	Skipped when no LogicGraph is bound,

	forceIncludeHeader		never faked
Cook	fby_fempak cook --progress	assets.fempak + cook.progress	Non-zero child exit fails the build
Secure	fby_fempak secure --key-file --progress	game.femarc	AEAD tag failure or missing key fails closed
Progress	CookProgressRecord -> ReleaseCookProgress	cook.progress / secure.progress	Torn record rejected by double-read sequence check

31.1 ReleaseCooker. The class lives in Source/Editor/Infrastructure/ReleaseCooker.hpp and links only the editor's existing services. Start() validates paths, creates the release tree, copies the LogicGraph POD and starts one std::jthread. The worker calls AotBuilder, then the cooker and the secure step, while the editor thread calls Progress() once per frame. The UI copy is a fixed POD under a short mutex; the worker never calls ImGui and the editor never blocks on the compiler or the compressor.

```
// One click from the toolbar: Build menu or the Viewport button.
ReleaseCookConfig Config = ReleaseCooker::FromProject(
    projectConfig.Paths(), projectConfig.Project(), EngineRoot, FempakPath);
Config.compileAot = visualScript.Initialized() && visualScript.Graph().nodeCount
!= 0;
char Error[256]{};
if (!releaseCooker.Start(Config, Graph, Error, sizeof(Error)))
{
    SetStatus(Error);
}

// SceneDiagnosticsPanel every frame. No allocation while drawing.
releaseCooker.Poll();
sceneDiagnostics.SetReleaseProgress(releaseCooker.Progress());

// The worker owns the sequence, in this order:
//   PrepareKey -> AotBuilder -> fby_fempak cook -> fby_fempak secure
```

32.2 Key baking. The earlier secure command required --key or --key-env and the editor had no caller. ReleaseCooker now reads a project-local 32-byte seed from .fbyrelease.seed (created with SecureRng when absent), writes a build-scoped binary key file for fby_fempak, and force-includes a generated header into the AOT translation unit. The header exports a C ABI function that reconstructs the key from two XOR arrays; the plugin carries the key, while the project file carries no password. The seed is the only persistent secret, and deleting it rotates the release key on the next build.

```
// Generated into build/release/ReleaseKey.generated.hpp and force-included
// into the AOT plugin (clang-cl /FI, clang/g++ -include).
FBY_RELEASE_KEY_EXPORT bool fby_release_archive_key(std::uint8_t* OutKey,
                                                    std::uint32_t InCapacity)
noexcept
```

```
{
    if (OutKey == nullptr || InCapacity < 32u) return false;
    constexpr std::uint8_t kMasked[32] = { /* key ^ mask */ };
    constexpr std::uint8_t kMask[32]   = { /* build mask */ };
    for (std::uint32_t i = 0; i < 32u; ++i)
        OutKey[i] = static_cast<std::uint8_t>(kMasked[i] ^ kMask[i]);
    return true;
}

// A launcher can resolve the same plugin and call the export before
// opening game.femarc. No password is read from ini or the environment.
```

File	Lifetime	Contains	On success
.fbyrelease.seed	Persistent project secret	32 random bytes	Kept, mode-restricted by the host
release.key.tmp	One build only	Plaintext AEAD key for fby_fempak	Overwritten with zeroes and deleted
ReleaseKey.generated.hpp	Compile only	Masked key + C ABI export	Deleted
archives/ game.femarc	Shipped artifact	LZMA/FbyLZ + ChaCha20/AES body	Kept

32.2a Project-local output tree. A project created by the dashboard owns every artifact it produces: build intermediates and the release key live in <root>/build/release, the canonical archive is <root>/archives/game.femarc, and the per-store stages are <root>/stores/{steam,epic,gog}. ReleaseCooker copies the archive and the AOT plugin into each store directory after a successful build and writes a store.txt manifest (project, archive, plugin, cipher, key_baked). Moving or deleting the project directory therefore moves the build, the archive and all store copies together; nothing is written into the engine tree or the process working directory.

32.3 Progress protocol. fby_fempak writes a 424-byte CookProgressRecord to --progress. The record is a POD: stage, completed/total, current asset, raw and stored bytes, elapsed microseconds, bytes/second, cipher and flags. The writer seeks to zero, writes the whole record and flushes after every update; ReleaseCooker reads it twice and accepts only a matching sequence, so a partially written record can never produce a false 90% bar. ArchiveBuildOptions carries a plain function-pointer progress callback, not std::function, so FemArchiver can report per-entry compressed bytes inside its noexcept build.

```
struct CookProgressRecord                       // 424 bytes, ABI-stable
{
    char magic[8];                              // FBYCOOK1
    std::uint32_t version, sequence;
    std::uint32_t stage, completed, total, bytesPerSecond;
    std::uint32_t cipher, flags, reserved;
    std::uint64_t rawBytes, storedBytes, elapsedMicroseconds;
    char asset[192];
    char message[160];
};
```

```
// SceneDiagnosticsPanel: the only UI work per frame.
ImGui::ProgressBar(Progress.Fraction(), ImVec2(-1.0f, 0.0f), Overlay);
ImGui::Text("asset: %s", Progress.asset);
ImGui::Text("LZMA %.2f MB/s", Progress.bytesPerSecond / (1024.0 * 1024.0));
ImGui::Text("encrypted: %s", CookCipherName(Progress.cipher));
```

- The current asset is exact: fby_fempak updates the record before and after every cooked file, so the bar shows the texture, audio or localization file being processed, not a generic 'cooking' label.
- Encryption state is exact: cipher 1/2 and the encrypted flag are copied from the secure build; a missing key or a rejected AEAD tag ends the stage as Failed, never Succeeded.
- LZMA throughput is measured from the archiver's progress bytes and elapsed time. Because the 7-Zip SDK call compresses one .fempak entry as a single operation, the rate is sampled at the entry boundary rather than every 64 KiB; the UI labels the value as measured, not predicted.

32.4 Release checklist. The same artifacts feed Steam, Epic Games Store and GOG. The cook tree is per-project and reproducible: a fixed seed and the deterministic .fempak writer make the archive hash a build stamp; the .femarc keeps LZMA and AEAD settings in its header; the AOT plugin records graph and component layout hashes. A store submission should attach the hash of game.femarc, the plugin, the seed identifier (not the seed itself) and the BuildProfile snapshot.

Store	Required artifact	ReleaseCooker check	Manual follow-up
Steam	game.femarc + plugin + depot VDF	LZMA/AEAD header, archive hash	SteamPipe upload, build id
Epic Games Store	game.femarc + plugin + EOS manifest	same bytes, EOS artifact hash	EOS BuildPatch, sandbox
GOG Galaxy	game.femarc + plugin + Galaxy shim	same bytes, shim ABI	Galaxy build, cloud saves
All stores	Seed identity, BuildProfile	Graph hash + component LayoutHash	Keep seed offline, rotate on leak

The build orchestrator needs the fby_fempak executable in build/bin. If the editor was built with FBY_BUILD_TOOLS=OFF, Start() fails before spawning anything and the status bar names the missing packer. The AOT stage is optional by construction: a project with no bound LogicGraph still cooks and secures its assets, and the result reports keyBaked=false.

34. SystemGraph: the parallel scheduling contract, then the blend-cycle contract

G8 changes SystemGraph::Run from a registration-order walk into a dependency-aware dispatcher. Build() already folds every before edge into an after mask; it now also computes a longest-path level per node. Two nodes on the same level cannot depend on each other, so a level is a safe parallel wave. Run(phase) launches every enabled node of one wave through JobSystem, leaves one node for the caller, and joins the wave through a counting_semaphore that lives on the caller's stack.

There is no std::function, no allocation and no shared per-system semaphore; a 24-worker pool therefore executes 24 independent systems in the same wave instead of 24 sequential calls.

```
// Build(): longest-path level, same-phase edges only.
for (Position in TopologicalOrder) {
    Node = Order[Position];
    Level = 0;
    for (Dep in DependencyMask(Node))
        if (Phase(Dep) == Phase(Node)) Level = max(Level, MLevels[Dep] + 1);
    MLevels[Node] = Level;
}

// Run(Phase): one wave, one per-call semaphore.
std::counting_semaphore<kMaxGraphSystems> Done{0};
for (I = 1; I < WaveSize; ++I) {
    Wave[I].completed = &Done;
    Jobs.Submit(&InvokeGraphNode, &Wave[I], 0, 0);   // no allocation
}
Wave[0].run(Context, Wave[0].user);                    // caller keeps one node
for (I = 1; I < WaveSize; ++I) Done.acquire();          // join before next
level
```

Guarantee	Why it is enforced
A node never starts before every after/before dependency completed	Level = max(dep level) + 1; Run joins a wave before opening the next one
Nodes in one wave never depend on each other	A dependency edge always raises the dependent's level
No worker thread is leaked or waited on twice	One release per submitted thunk, one acquire per submission, semaphore is stack-local
Render stays on the render thread	Run(Render) takes the sequential path: the RHI command stream is single-threaded
A mutation invalidates the schedule	Add/RemoveUser/Clear set MBuilt = false; Run does nothing until Build() succeeds

The dependency mask is the whole data-race contract. Two systems that write the same component pool must have an edge; 'they happened to be registered in this order' is not a dependency. The editor's FixedUpdate chain (events_clear -> movement -> physics -> contacts -> spriteAnimation -> skeletalAnimation_animate) therefore declares five after bits, while independent PostUpdate nodes (skeletalAnimation_evaluate, physics_interpolate, cloth collect) form one wave. JobSystem is part of the fix: MPending and MRunning are now placed on separate 64-byte lines, so submit/completion traffic from different cores does not bounce one cache line, and ParallelFor uses a per-call semaphore instead of a shared member.

CompileGraph() is not a serializer: it is the last place where an authored graph can be rejected before AnimStateComponent becomes runtime state. The contract has two layers. The first is local: a Clip node owns one non-zero state id, a Float node owns one unique parameter index, a Blend node owns exactly one source link, exactly one target link and at most one parameter link, and the

source and target states must differ. The second is global: the compiled transition graph must have no cycle that can fire forever without an exit.

Rule	Compiler result	Authoring action
Clip state is 0 or kAnimAnyState	Rejected: needs a non-zero state id	Assign a unique 64-bit gameplay id
Blend source state == target state	Rejected: cross-fades a state into itself	Pick two different clips
Blend has 0 or 2 source/target links	Rejected: exactly one source, one target	Delete the extra link
Cycle of hot transitions with no exit	Rejected: dead blend cycle without an exit	Add a gated exit or remove the loop
Cycle with any enabled exit	Accepted	Keep the exit before the hot edge in array order

A transition is 'hot' when it is Enabled, has no tag/float condition, and its state-time predicate is already true at StateTime == 0: StateTimeAtLeast with threshold <= 0, and/or StateTimeAtMost with threshold >= 0. AnimGraphSystem resets StateTime to zero on every fired transition and adds dt before evaluating the next tick, so a hot edge fires on every entry. A hot cycle is a dead loop only when no enabled transition leaves the set of states in that cycle; a gated exit keeps the graph legal even if the loop itself is hot.

```
// Iterative three-colour DFS. Bounded by kMaxAnimClips == 8; no recursion.
std::array<std::uint8_t, kMaxAnimClips> Color{};   // 0 white, 1 gray, 2 black
std::array<std::uint16_t, kMaxAnimClips> Stack{};
for (std::uint16_t Start = 0; Start < N; ++Start) {
    if (Color[Start] != 0) continue;
    std::int32_t Top = 0;
    Stack[0] = Start; Color[Start] = 1;
    while (Top >= 0) {
        const std::uint16_t V = Stack[Top];
        std::uint32_t& Edges = HotEdges[V];
        if (Edges == 0) { Color[V] = 2; --Top; continue; }
        std::uint16_t W = 0;
        while (((Edges >> W) & 1u) == 0) ++W;
        Edges &= ~(1u << W);
        if (Color[W] == 0) { Color[W] = 1; Stack[++Top] = W; continue; }
        if (Color[W] == 1) {
            // W is on the gray stack: reconstruct the cycle, test for an exit.
            if (!HasEnabledExit(Stack, Top, W)) {
                SetStatus("Anim compile: dead blend cycle without an exit: %s",
Path);
                LogLine(LogLevel::Warn, "AnimGraph", MStatus);
                return false;
            }
        }
    }
}
```

The validator runs on the compiled edge set, not on the ImNodes document, so it sees exactly what the runtime will evaluate: disabled links are already filtered by AnimTransitionMatches, and state ids have been resolved from the links. The warning goes through LogLine (not std::format), so the error path stays noexcept and LogBoy still receives the record. A fully cyclic document is at most 8 vertices: the explicit stack cannot overflow, and the worst case is 64 edge visits.

35. Box2D rollback teardown, then the PIE animation hand-off

R1 is an ownership bug, not a Box2D bug. Registry::clear() destroys ECS objects; it cannot know that RigidBody2D::body is a native b2BodyId, so the old bodies stay in the world while the restored snapshot rows all carry body == 0. The next EnsureBodies()/Step() then creates a second body for every entity. The fix is ordered: (1) ask PhysicsSystem to destroy every tracked live body and sweep the registry for joint-link bodies it did not create; (2) clear and restore the snapshot; (3) recreate bodies from the restored bytes. A body is destroyed only after b2Body_IsValid, so a joint system that already tore its anchors down is safe.

```
NetRollbackSession::Result NetRollbackSession::RestoreSnapshot(
    std::span<const std::byte> InSnapshot) noexcept
{
    if (!MInitialized || MRegistry == nullptr || MSchema == nullptr)
        return Result::NotInitialized;

    // R1: native bodies die BEFORE Registry::clear().
    if (MPhysics != nullptr)
        (void)MPhysics->DestroyAllBodies(MRegistry);

    if (!ecs::RestoreSnapshot(*MRegistry, *MSchema, InSnapshot, &MRng))
        return Result::RestoreFailed;

    // Snapshot bytes never carry a b2 handle: rebuild from the restored rows.
    if (MPhysics != nullptr)
        MPhysics->EnsureBodies(*MRegistry);
    return Result::Ok;
}

// PhysicsSystem::DestroyAllBodies: tracked pass + registry sweep.
for (Record : MBodies)
    if (b2Body_IsValid(Load(Record.body)))
        { b2DestroyBody(Load(Record.body)); ++Destroyed; }
MBodies.clear();
for (Body : Registry.view<RigidBody2D>())
    if (Body.body != 0) {
        if (b2Body_IsValid(Load(Body.body))) { b2DestroyBody(Load(Body.body)); +
+Destroyed; }
        Body.body = 0;                       // no stale handle survives
    }
for (Collider : Registry.view<Collider2D>()) Collider.shape = 0;
```

Restore phase	Native state	Invariant after the phase
DestroyAllBodies(registry)	Every valid tracked body + every registry body destroyed	World contains no body owned by the old frame
Registry::clear() + snapshot decode	No native object is touched	Restored RigidBody2D rows have body == 0
EnsureBodies(registry)	One b2Body/b2Shape per restored row	Body count equals the pre-rollback count, never twice it
Replay to the current tick	Step() finds body != 0 and skips creation	No duplicate physics on any replayed tick

Play-In-Editor uses the same rule at a larger granularity. The clone owns a separate PhysicsSystem, so Play never inserts clone bodies into the editor world. Stop calls DestroyAllBodies(clone) and then shuts that world down; the authoring scene was never mutated, so Stop is a constant-time rollback with no orphan body. The same path is what makes the toolbar Play/Pause/Stop honest: the buttons drive a real simulation, not a preview flag.

The canvas needs six scalars per frame: CurrentState, PreviousState, BlendWeight, BlendTime, StateTime and LastTransition. Reading them from a live AnimStateComponent on the ImGui thread is not a benign race: SystemGraph workers may be inside AnimGraphSystem::Run, and the EnTT lookup itself is unsafe while component pools are created or destroyed. Atomic reads of the six fields would not fix the registry/lifetime part. The panel therefore owns a 64-byte POD and a two-slot seqlock: the simulation thread publishes, the editor thread consumes, and the live component is never dereferenced from Draw().

```
struct PIEAnimFrame {                          // exactly 64 bytes
    std::uint64_t CurrentState = 0;
    std::uint64_t PreviousState = 0;
    float BlendWeight = 0.0f, BlendTime = 0.0f, StateTime = 0.0f;
    std::uint16_t LastTransition = kNoAnimTransition;
    std::array<std::byte, 34> Reserved{};
};

// Producer (simulation thread, after the FixedUpdate join):
void PublishPIEState(const AnimStateComponent& S) noexcept {
    const std::uint32_t Write = MPieWriteSlot.load(std::memory_order_relaxed);
    PIEFrameSlot& Slot = MPieSlots[Write ^ 1u];
    const std::uint64_t Seq = Slot.Sequence.load(std::memory_order_relaxed);
    Slot.Sequence.store(Seq + 1u, std::memory_order_relaxed);
    std::atomic_thread_fence(std::memory_order_release);
    StoreRelaxed(Slot.Frame.CurrentState, S.CurrentState);
    StoreRelaxed(Slot.Frame.PreviousState, S.PreviousState);
    StoreRelaxed(Slot.Frame.BlendWeight, S.BlendWeight);
    StoreRelaxed(Slot.Frame.BlendTime, S.BlendTime);
    StoreRelaxed(Slot.Frame.StateTime, S.StateTime);
    StoreRelaxed(Slot.Frame.LastTransition, S.LastTransition);
    std::atomic_thread_fence(std::memory_order_release);
    Slot.Sequence.store(Seq + 2u, std::memory_order_release);
    MPieWriteSlot.store(Write ^ 1u, std::memory_order_release);
```

```
    MPiePublished.store(1u, std::memory_order_release);
}

// Consumer (ImGui/Render thread): retry only if the sequence changed.
const std::uint64_t Before = Slot.Sequence.load(std::memory_order_acquire);
if ((Before & 1u) == 0) {
    ReadRelaxed(Frame.CurrentState, Slot.Frame.CurrentState);
    /* ... five more ... */
    std::atomic_thread_fence(std::memory_order_acquire);
    if (Before == Slot.Sequence.load(std::memory_order_relaxed)) return true;
}
```

Operation	Ordering	Why
sequence increment (odd)	relaxed + release fence	Marks the slot as being written; the fence prevents payload stores from moving before it
payload stores	relaxed atomic_ref	No data race; ordering is supplied by the sequence brackets
sequence increment (even)	release store	Publishes the payload before the reader can accept it
writer slot publish	release store	Reader takes the slot index with acquire and sees a complete frame
reader retry	acquire + relaxed re-check	Rejects a torn read after the writer lapped the reader

- BindRuntimeWorld() only invalidates the buffer when the world pointers or the PIE flag actually change; EditorSystem calls it every frame, so an unconditional invalidate would erase the sample before Draw().
- PublishPIEStateFromWorlds() is the convenience producer: it resolves EditorSelected, looks up AnimStateComponent in the runtime world and publishes. It must run on the simulation thread, never from Draw().
- RefreshPIEState() and DrawPIEInspector() read only the seqlock slot. The inspector no longer walks the Mirror table over a live pointer; it prints the published scalar fields.
- The hand-off adds exactly one frame of latency and one 64-byte copy per published frame. It allocates nothing and never takes a mutex.

If PIE is later stepped on a worker pool, the host must publish after joining the pool and before the editor draws. Do not call PublishPIEState() from the render thread while workers still run: the producer side touches the registry and a live component, which are not protected by the seqlock. The seqlock protects the hand-off, not the capture.

36. Global audit: release-blocking findings and the gate

Historical snapshot. This table is frozen at the 2026-10-27 audit: several findings are already fixed, and the live status with file/line evidence is Documentation/ROADMAP.md. Do not schedule work from this table.

The audit covered VulkanRHI/MetalRHI, NetRollbackSession, Snapshot v4, the Box2D remap, the weather compute path, VFS and the asset cooker, JobSystem, SystemGraph, hot reload and the animation canvas. It confirmed more than fifty defects. This chapter records the frozen snapshot for reference, and Documentation/ROADMAP.md is the live gate a reviewer should apply.

ID	Severity	Finding	Minimal fix
R1	Critical	Rollback restore never destroys old Box2D bodies; each rollback duplicates the world	PhysicsSystem::DestroyAllBodies() inside NetRollbackSession::RestoreSnapshot(), before Registry::clear()
L1	High	FilterDeltaForPlayer checks the source payload length, not the destination capacity	Compute capacity from OutFiltered and test before every memcpy
C1/C2	Critical	Readback runs before frame submit; failed frame leaves fence/semaphore signalled	Record copy into the frame CB; reset fence only before a successful submit
S1	High	ParticleEmitterComponent loses six non-reflected members on restore	Add the six FBY_FIELD entries
V2	Critical	Store codec memcpy uses attacker-controlled stored.size()	Require StoredSize == RawSize for Store
V3	High	Video decoder keeps a VFS mmap span across hot reload	Own the bytes or return a shared mapping + generation
G1	Critical	New projects have no CMakeLists/GameMain and cannot be built from the dashboard	ProjectScaffold.hpp + CreateProject writes CMakeLists.txt, project.femproj and GameMain.cpp
G2	Critical	Play/Pause/Stop are cosmetic; the editor never snapshots or steps an isolated	EditorSystem toolbar -> PlayInEditor::Start/Step/Stop with a private

		world	physics world
G8	High	SystemGraph::Run is a sequential walk regardless of after/before	Dependency levels + one wave per level + per-call semaphore + cache-line split
H1	High	Nested ParallelFor shares one semaphore and can return before helpers finish	Per-call semaphore + TLS nesting flag
H4	High	SystemGraph::Run ignores MBuilt; Add/RemoveUser leave a stale or empty order	Gate Run on MBuilt; rebuild on mutation
H3 (RHI)	High	Instance deltas are applied to the N-3 slice	Full copy when framesInFlight > 1, or timestamp slices
W1	Critical	Splash counter is never reset; splashes stop after capacity	Clear per frame in a pre-pass, or ring modulo capacity

Gate	Contents	Exit criterion
A — blocks freeze	R1, L1, C1, C2, S1, V2, V3, G1, G2, G8, H1, H3, H4, W1-W3	Every item has a fix or a signed waiver plus a regression test
B — before RC	R2, R3, S2-S8, C4-C6, V4-V7, H5/H6, W2/W6, M8-M10	Fix or documented limitation in the release notes
C — after freeze	Low findings, SoA split, remaining SIMD work	Scheduled for 1.1 with a profiling baseline

The audit also verified the parts that are correct and must not be 'fixed' by accident: snapshot canonical ordering and header layout, Pcg32 reference compatibility, VFS pak range checks, LZ decompressor bounds, fence-before-acquire, per-image present semaphores, descriptor accounting, plugin unload order, and the TTS main-thread contract. Several findings are latent because the weather records are not consumed by a backend yet; they are still release blockers once that consumer lands.

37. Profiling cache misses and vectorisation toward 8000+ FPS

8000 FPS is 125 microseconds per frame. That number is meaningful only together with the scene, resolution and hardware: a 2D batch of a few thousand sprites at 1080p is a different problem from a 4K frame with post-processing. Treat 8000 FPS as a target for the engine's own frame cost (simulation + extract + submission) on a Zen 5 machine with a warm cache, and always report the

99th percentile frame time, not just the average FPS. A stable 8000+ means p99 <= 125 us, not one lucky frame.

- Step 1 — baseline. Record CPU and GPU frame time separately. On Windows use WPA/ETW or AMD uProf; on Linux use perf stat with cache-references, cache-misses, L1-dcache-load-misses, LLC-load-misses and stalled-cycles-frontend. Store the numbers in the repository so a regression is visible.
- Step 2 — find the hot array. The animation tick walks AnimStateComponent per entity (4 KB stride); the extract walks RenderItem; the job pool touches one shared cache line. A 4 KB stride with 8 transitions and 8 clips per component thrashes L1 and L2 before any math runs.
- Step 3 — split hot/cold. Keep a 64-byte AnimHot per entity (states, StateTime, Blend*) and move Transitions/Clips/RestPose into a shared graph object or a cold side table. The system then streams a dense array; the shared tables stay L1-resident.
- Step 4 — vectorise the presentation blend, not the deterministic simulation. Use __restrict, SoA float arrays (X[], Y[], Rotation[], Scale[]) and check the vectoriser report. Do not reorder floating-point reductions used by rollback or physics.
- Step 5 — reduce RHI binds. Sort draws by (pipeline, material, texture array), use push constants for the draw id, keep one bindless set, and record one indirect draw per batch. Descriptor-set churn is usually more expensive than the draw itself at 8k FPS.
- Step 6 — parallelise phases, not just entities. SystemGraph::Build() computes dependency levels once; Run() launches each level as one wave through JobSystem and joins it with a per-call semaphore before Render. MPending and MRunning sit on separate 64-byte lines, so submit/completion traffic does not bounce a shared line between 24 workers. Keep the main thread for submission and reserve one physical core for render/UI; 23 SMT workers on 12 physical cores often lose to 11-12 dedicated workers on cache-bound work.

```
// SoA blend: one AVX-512 iteration covers 16 bones on Zen 5.
void BlendBones(float* __restrict OutX, float* __restrict OutY,
                const float* __restrict AX, const float* __restrict AY,
                const float* __restrict BX, const float* __restrict BY,
                std::size_t Count, float Weight) noexcept
{
    for (std::size_t I = 0; I < Count; ++I)
    {
        OutX[I] = AX[I] + (BX[I] - AX[I]) * Weight;
        OutY[I] = AY[I] + (BY[I] - AY[I]) * Weight;
    }
}

// Verify the vectoriser actually fired:
// clang-cl /O2 /Rpass=loop-vectorize /Rpass-missed=loop-vectorize
// clang++  -O2 -Rpass=loop-vectorize -Rpass-missed=loop-vectorize
```

Counter	Healthy range	If it is worse
L1d load misses	< 2% of loads	Hot/cold split, prefetch, reduce stride
LLC load misses	< 1-2%	Shrink working set, huge pages, reorder loops

stalled-cycles-frontend	< 10%	Branch layout, inline hot paths, avoid i-cache thrash
IPC	> 2.5	Check dependency chains, false sharing, lock contention
p99 frame time	<= 125 us for 8000 FPS	Cap fixed steps, remove WaitIdle from the frame, budget submissions

Use RenderDoc or PIX to separate CPU from GPU: an 8k FPS CPU frame with a 2 ms GPU frame is still 500 FPS on screen. Do not chase the CPU target by removing correctness work (validation, barriers, checksums) — move it to a background job or a less frequent pass. And keep a rollback-determinism test green after every SIMD change: vectorised float order is a classic source of 'works on my machine' desync.

38. Materials in the engine: cook, cache and the frame

The material system was the last subsystem whose runtime still lived on the editor side. The graph model and the GLSL emitter shipped in `fby_engine`, but the driver that turned a graph into a pipeline - glslang, SPIR-V, `CreateShaderPipeline` and a private 128-entry table - shipped only in `fby_editor`. A game links `FemboyEngine2D::Engine` alone, so it could validate a material and had no way to draw with it. [ADR 0002](../../adr/0002-materials-in-engine.md) moves the missing half into the engine and leaves the editor with a thin live-recompile adapter.

Stage	Owner	What it validates
Author: `MaterialGraph`	Editor panel, or a tool that writes `.fgraph`	One output node, connected required pins, type compatibility, acyclicity, capacity limits
Cook: `fby_matc`	Build step, links glslang	The whole graph, then each generated GLSL stage through glslang, then the payload encoder
Load: `DecodeCookedMaterial`	Shipped runtime, no allocation	Magic, version, size against the stage lengths, both SPIR-V modules, contract consistency
Acquire: `MaterialPipelineCache`	Shipped runtime, one cache per device	Hash and contract against the resident entries, reference count, LRU eviction, backend capability
Frame: `MeshRenderItem::materialPipeline`	Render extract and the backend	Vulkan groups mesh records by the id and binds one material pipeline per group; other backends keep the engine shader

Cooking a material is one command, and the output travels like any other cooked asset:

```
# fby_matc <graph.fgraph> [-o <cooked.femmat>]
build/bin/fby_matc assets/materials/tinted.fgraph -o
assets/materials/tinted.femmat

# The tool refuses a graph that does not validate, prints the hash and the SPIR-
V word
# counts, and never writes a payload the runtime would reject.
```

```
// One cache per device, owned by the game. Acquire at load time.
Render::MaterialPipelineCache ScopedMaterials;

// `InBytes` comes from the VFS or a package entry.
Render::CookedMaterialView ScopedTinted;
if (!Render::DecodeCookedMaterial(InBytes, &ScopedTinted))
{
    LogError("materials", "tinted.femmat is not a cooked material");
    return false;
}

const Rhi::ShaderPipelineHandle ScopedTintedPipeline =
    ScopedMaterials.Acquire(InRhi, ScopedTinted);

// A missing pipeline is not a crash: the item falls back to the engine shader.
ScopedItem.materialPipeline = ScopedTintedPipeline.IsValid()
    ? ScopedTintedPipeline.id
    : Rhi::kDefaultMaterialPipeline;

// At shutdown, while the device is still alive.
ScopedMaterials.Shutdown(InRhi);
```

- Retention over churn: a pipeline at zero references stays resident, because a game that acquires a material every frame and releases it at the end of the frame would otherwise rebuild the pipeline sixty times per second. The 128-entry cap and LRU eviction bound the cost instead.
- The contract mask is part of the identity check. A graph hash that arrives with a different contract while the entry is live is refused; the same hash with a different contract and no references is replaced in place, which is what the editor does when push-constant support changes.
- The backend says whether it can build a runtime pipeline. Vulkan returns true; Null and Metal return false and keep the engine shaders, so a game targeting them fails at load with a readable error instead of drawing with an unbound pipeline.
- Nothing on this path allocates or formats text per frame: the cache is a fixed array, the decode is a view, and the error is a fixed buffer.

The editor keeps its live experience through the same cache. `MaterialGraphRuntime` stores one graph pointer per tab, recompiles when `Hash()` changes, keeps the last good pipeline while an edit is invalid, and reports the failure per tab. It no longer owns a pipeline table: `Pipeline()` returns the handle the engine cache handed out, so an editor session and a game share one budget.

Test	Covers
`material_graph`	DAG validation, deterministic GLSL, glslang acceptance
`material_asset`	Graph round trip, payload corruption, contract flags
`material_cook`	End-to-end: the tool cooks, an Engine-only binary decodes and checks the hash
`material_pipeline_cache`	Sharing, reference counts, eviction, exhaustion, refusal paths
`material_mesh_path`	Two meshes share one cached pipeline, a third keeps the engine shader

The sprite path is now part of the same system. A sprite material is a second shader contract, not a second compiler: `MaterialCompileOptions::domain` switches the fragment interface to the `quad.vert` varyings, `vLayer` supplies the item's atlas slice, and the sprite push block carries a material clock at offset 80, so `Time` works in both domains; only `Fresnel` and `VertexPosition` are refused because they need a view-projection. The 64-bit render key carries the material bucket (16 bits layer, 8 bits material, 32 bits depth, 8 bits sequence), so `DrawOrder::MaterialBucket` both sorts a sprite into its material run and, through `ChangeMaskRenderExtract::SetMaterialPipelines`, resolves the run to one `DrawBatch` with the cached pipeline id. The Vulkan sprite loop binds that pipeline per batch and restores the engine quad pipeline for the next run; a backend without runtime shader pipelines keeps the quad pipeline and draws the sprite unshaded by the graph, never with an unbound handle. Compile-time layer composition, if it is ever added, becomes node kinds in the same graph, never a variable-length closure buffer.

39. Appendix A - generated symbol index

Generated from the headers on every run. Header and line are the authoritative location.

Editor/Infrastructure (94)

Symbol	Kind	Header	Line
AIStateTreeCanvasPanel	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	39
AnimationGraphCanvasPanel	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	40
AnimationTimelinePanel	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	41
AssetDatabaseWatchSystem	Infrastructure	Source/Editor/Infrastructure/AssetDatabaseWatchSystem.hpp	20
AssetDatabaseWatchSystem	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	42
AssetImportService	Infrastructure	Source/Editor/Infrastructure/AssetImportService.hpp	30
AssetImportService	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	43
Callbacks	Infrastructure	Source/Editor/Infrastructure/AssetDatabaseWatchSystem.hpp	38
Change	Infrastructure	Source/Editor/Infrastructure/AssetDatabaseWatchSystem.hpp	102
Codec	Infrastructure	Source/Editor/Infrastructure/CommandStack.hpp	217
Command	Infrastructure	Source/Editor/Infrastructure/CommandStack.hpp	231
CommandKind	Infrastructure	Source/Editor/Infrastructure/CommandStack.hpp	30

CommandStack	Infrastructure	Source/Editor/ Infrastructure/ CommandStack.hpp	27
CommandStack	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	44
Config	Infrastructure	Source/Editor/ Infrastructure/ AssetDatabaseWatch System.hpp	27
Config	Infrastructure	Source/Editor/ Infrastructure/ AssetImportService.hp p	39
Config	Infrastructure	Source/Editor/ Infrastructure/ CommandStack.hpp	85
ContentBrowserPan el	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	45
CookRequest	Infrastructure	Source/Editor/ Infrastructure/ AssetImportService.hp p	115
CookResult	Infrastructure	Source/Editor/ Infrastructure/ AssetImportService.hp p	126
EditorCamera	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	85
EditorDesc	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	56
EditorFontRole	Infrastructure	Source/Editor/ Infrastructure/ EditorFonts.hpp	22
EditorFonts	Infrastructure	Source/Editor/ Infrastructure/ EditorFonts.hpp	79
EditorFonts	Infrastructure	Source/Editor/ Infrastructure/ EditorTypography.hpp	7
EditorFontSpec	Infrastructure	Source/Editor/ Infrastructure/	38

		EditorFonts.hpp	
EditorFontStatus	Infrastructure	Source/Editor/ Infrastructure/ EditorFonts.hpp	66
EditorSettings	Infrastructure	Source/Editor/ Infrastructure/ EditorSettings.hpp	18
EditorSettingsPanel	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	46
EditorSystem	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	94
Entry	Infrastructure	Source/Editor/ Infrastructure/ TextureThumbnails.hpp	52
ExternalClientDesc	Infrastructure	Source/Editor/ Infrastructure/ PlayInEditor.hpp	37
FrameComposer	Infrastructure	Source/Editor/ Infrastructure/ FrameComposer.hpp	28
ImFont	Infrastructure	Source/Editor/ Infrastructure/ EditorFonts.hpp	10
ImGuiIO	Infrastructure	Source/Editor/ Infrastructure/ EditorFonts.hpp	11
Impl	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	200
InteractionTransaction	Infrastructure	Source/Editor/ Infrastructure/ EditTransactions.hpp	16
LoadedSource	Infrastructure	Source/Editor/ Infrastructure/ EditorFonts.hpp	133
MaterialGraphPanel	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	47
MaterialGraphRuntime	Infrastructure	Source/Editor/ Infrastructure/ MaterialGraphRuntime.hpp	41

MaterialRuntimeStats	Infrastructure	Source/Editor/Infrastructure/MaterialGraphRuntime.hpp	25
NetSession	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	27
NetworkDiagnosticsPanel	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	48
PendingEdit	Infrastructure	Source/Editor/Infrastructure/CommandStack.hpp	252
PendingSpawn	Infrastructure	Source/Editor/Infrastructure/CommandStack.hpp	261
PhysicsDebugFlags	Infrastructure	Source/Editor/Infrastructure/PhysicsDebugDraw.hpp	19
PhysicsDebugFrame	Infrastructure	Source/Editor/Infrastructure/PhysicsDebugDraw.hpp	47
PhysicsDebugLine	Infrastructure	Source/Editor/Infrastructure/PhysicsDebugDraw.hpp	32
PhysicsDebugPoint	Infrastructure	Source/Editor/Infrastructure/PhysicsDebugDraw.hpp	40
PhysicsDebugVisualizer	Infrastructure	Source/Editor/Infrastructure/PhysicsDebugDraw.hpp	58
PiePlayerInput	Infrastructure	Source/Editor/Infrastructure/PlayInEditor.hpp	27
PlayInEditor	Infrastructure	Source/Editor/Infrastructure/PlayInEditor.hpp	44
ProjectConfig	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	52
ProjectConfig	Infrastructure	Source/Editor/	142

		Infrastructure/ ProjectConfig.hpp	
ProjectConfigStore	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	49
ProjectConfigStore	Infrastructure	Source/Editor/ Infrastructure/ ProjectConfig.hpp	202
ProjectDashboard	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	50
ProjectPaths	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	53
ProjectPaths	Infrastructure	Source/Editor/ Infrastructure/ ProjectConfig.hpp	164
ProjectPaths	Infrastructure	Source/Editor/ Infrastructure/ ReleaseCooker.hpp	21
ProjectSettings	Infrastructure	Source/Editor/ Infrastructure/ ProjectConfig.hpp	90
ProjectSettings	Infrastructure	Source/Editor/ Infrastructure/ ReleaseCooker.hpp	22
ProjectSettingsPanel	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	51
Range	Infrastructure	Source/Editor/ Infrastructure/ CommandStack.hpp	246
ReleaseCookConfig	Infrastructure	Source/Editor/ Infrastructure/ ReleaseCooker.hpp	70
ReleaseCooker	Infrastructure	Source/Editor/ Infrastructure/ ReleaseCooker.hpp	154
ReleaseCookProgress	Infrastructure	Source/Editor/ Infrastructure/ ReleaseCooker.hpp	117
ReleaseCookResult	Infrastructure	Source/Editor/ Infrastructure/ ReleaseCooker.hpp	138
ReleaseCookStage	Infrastructure	Source/Editor/	28

		Infrastructure/ ReleaseCooker.hpp	
Result	Infrastructure	Source/Editor/ Infrastructure/ CommandStack.hpp	40
Result	Infrastructure	Source/Editor/ Infrastructure/ PlayInEditor.hpp	47
SdfCollisionSystem	Infrastructure	Source/Editor/ Infrastructure/ PhysicsDebugDraw.hpp	13
SdfField	Infrastructure	Source/Editor/ Infrastructure/ FrameComposer.hpp	17
Services	Infrastructure	Source/Editor/ Infrastructure/ PlayInEditor.hpp	220
Slot	Infrastructure	Source/Editor/ Infrastructure/ MaterialGraphRuntime.hpp	118
Source	Infrastructure	Source/Editor/ Infrastructure/ EditTransactions.hpp	19
SpawnComponent	Infrastructure	Source/Editor/ Infrastructure/ CommandStack.hpp	275
SpawnHeader	Infrastructure	Source/Editor/ Infrastructure/ CommandStack.hpp	268
SpawnTransaction	Infrastructure	Source/Editor/ Infrastructure/ EditTransactions.hpp	190
StableSource	Infrastructure	Source/Editor/ Infrastructure/ EditorFonts.hpp	140
Stats	Infrastructure	Source/Editor/ Infrastructure/ AssetDatabaseWatchSystem.hpp	88
Stats	Infrastructure	Source/Editor/ Infrastructure/ AssetImportService.hpp	55

Stats	Infrastructure	Source/Editor/ Infrastructure/ CommandStack.hpp	92
TextureArrayAsset	Infrastructure	Source/Editor/ Infrastructure/ AssetImportService.hp p	23
TextureBinding	Infrastructure	Source/Editor/ Infrastructure/ AssetImportService.hp p	142
TextureThumbnailCa che	Infrastructure	Source/Editor/ Infrastructure/ TextureThumbnails.hp p	17
VirtualFileSystem	Infrastructure	Source/Editor/ Infrastructure/ AssetImportService.hp p	20
VirtualFileSystem	Infrastructure	Source/Editor/ Infrastructure/ EditorFonts.hpp	15
VirtualFileSystem	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	18
VirtualTextureSettin gs	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	32
VirtualTextureSettin gs	Infrastructure	Source/Editor/ Infrastructure/ ProjectConfig.hpp	30
VirtualTextureSyste m	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	22
VisualScriptPanel	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	54
WindowManager	Infrastructure	Source/Editor/ Infrastructure/ EditorFonts.hpp	16

Editor/Panels (76)

Symbol	Kind	Header	Line
Action	Panels	Source/Editor/	25

		Panels/ ProjectDashboard.hpp	
AIStateTreeCanvasPanel	Panels	Source/Editor/ Panels/ AIStateTreeCanvasPanel.hpp	55
AIStateTreeCompiled	Panels	Source/Editor/ Panels/ AIStateTreeCanvasPanel.hpp	35
AIStateTreeDocument	Panels	Source/Editor/ Panels/ AIStateTreeCanvasPanel.hpp	24
AnimationGraphCanvasPanel	Panels	Source/Editor/ Panels/ AnimationGraphCanvasPanel.hpp	116
AnimationGraphDocument	Panels	Source/Editor/ Panels/ AnimationGraphCanvasPanel.hpp	79
AnimationGraphLink	Panels	Source/Editor/ Panels/ AnimationGraphCanvasPanel.hpp	65
AnimationGraphLinkKind	Panels	Source/Editor/ Panels/ AnimationGraphCanvasPanel.hpp	33
AnimationGraphNode	Panels	Source/Editor/ Panels/ AnimationGraphCanvasPanel.hpp	45
AnimationGraphNodeKind	Panels	Source/Editor/ Panels/ AnimationGraphCanvasPanel.hpp	26
AnimationTimelinePanel	Panels	Source/Editor/ Panels/ AnimationTimelinePanel.hpp	34
AssetImportService	Panels	Source/Editor/ Panels/ ContentBrowserPanel.hpp	24

AudioSystem	Panels	Source/Editor/ Panels/ AnimationTimelinePanel.hpp	15
AudioSystem	Panels	Source/Editor/ Panels/ VoiceDevicePanel.hpp	11
CanvasModelOps	Panels	Source/Editor/ Panels/ NodeCanvasPanel.hpp	21
CommandStack	Panels	Source/Editor/ Panels/ ContentBrowserPanel.hpp	25
CommandStack	Panels	Source/Editor/ Panels/ SettingsPanels.hpp	28
ComponentRegistry	Panels	Source/Editor/ Panels/ ScriptDebuggerPanel.hpp	18
Config	Panels	Source/Editor/ Panels/ ContentBrowserPanel.hpp	35
ContentBrowserPanel	Panels	Source/Editor/ Panels/ ContentBrowserPanel.hpp	27
DragPayload	Panels	Source/Editor/ Panels/ ContentBrowserPanel.hpp	60
EditorFonts	Panels	Source/Editor/ Panels/ ProjectDashboard.hpp	20
EditorFonts	Panels	Source/Editor/ Panels/ VisualScriptPanel.hpp	20
EditorSettingsPanel	Panels	Source/Editor/ Panels/ SettingsPanels.hpp	72
EditSource	Panels	Source/Editor/ Panels/ AIStateTreeCanvasPa	58

		nel.hpp	
EditSource	Panels	Source/Editor/ Panels/ AnimationGraphCanvasPanel.hpp	119
Entry	Panels	Source/Editor/ Panels/ ContentBrowserPanel.hpp	43
FileDialog	Panels	Source/Editor/ Panels/ ContentBrowserPanel.hpp	140
ImNodesContext	Panels	Source/Editor/ Panels/ AIStateTreeCanvasPanel.hpp	13
ImNodesContext	Panels	Source/Editor/ Panels/ AnimationGraphCanvasPanel.hpp	15
LicensesPanel	Panels	Source/Editor/ Panels/ LicensesPanel.hpp	11
LogBoyPanel	Panels	Source/Editor/ Panels/ LogBoyPanel.hpp	394
LogBoyRecord	Panels	Source/Editor/ Panels/ LogBoyPanel.hpp	105
LogBoyRing	Panels	Source/Editor/ Panels/ LogBoyPanel.hpp	121
LogBoySeverity	Panels	Source/Editor/ Panels/ LogBoyPanel.hpp	33
LogBoySystem	Panels	Source/Editor/ Panels/ LogBoyPanel.hpp	186
LogicGraphDocument	Panels	Source/Editor/ Panels/ VisualScriptPanel.hpp	23
MaterialGraphPanel	Panels	Source/Editor/ Panels/ MaterialGraphPanel.h	20

		pp	
Mode	Panels	Source/Editor/ Panels/ ProjectDashboard.hpp	84
NetSession	Panels	Source/Editor/ Panels/ NetworkDiagnosticsPa nel.hpp	6
NetworkDiagnostics Panel	Panels	Source/Editor/ Panels/ NetworkDiagnosticsPa nel.hpp	15
NodeCanvasPanel	Panels	Source/Editor/ Panels/ NodeCanvasPanel.hp p	54
PIEAnimFrame	Panels	Source/Editor/ Panels/ AnimationGraphCanv asPanel.hpp	101
PieClientViewport	Panels	Source/Editor/ Panels/ PieClientViewport.hpp	15
PIEFrameSlot	Panels	Source/Editor/ Panels/ AnimationGraphCanv asPanel.hpp	283
ProjectConfigStore	Panels	Source/Editor/ Panels/ SettingsPanels.hpp	29
ProjectConfigStore	Panels	Source/Editor/ Panels/ VoiceDevicePanel.hpp	17
ProjectDashboard	Panels	Source/Editor/ Panels/ ProjectDashboard.hpp	22
ProjectSettingsPanel	Panels	Source/Editor/ Panels/ SettingsPanels.hpp	31
QueryKind	Panels	Source/Editor/ Panels/ LogBoyPanel.hpp	430
SceneDiagnosticsPa nel	Panels	Source/Editor/ Panels/ SceneDiagnosticsPan	37

		el.hpp	
SceneDiagnosticsSample	Panels	Source/Editor/Panels/SceneDiagnosticsPanel.hpp	15
ScriptDebuggerPanel	Panels	Source/Editor/Panels/ScriptDebuggerPanel.hpp	28
ScriptRuntime	Panels	Source/Editor/Panels/AnimationTimelinePanel.hpp	16
SinkUser	Panels	Source/Editor/Panels/AnimationTimelinePanel.hpp	88
SkeletonInspectorState	Panels	Source/Editor/Panels/SkeletonAssetPanel.hpp	19
SpawnContext	Panels	Source/Editor/Panels/ContentBrowserPanel.hpp	68
SpriteAnimationInspectorState	Panels	Source/Editor/Panels/SpriteAnimationPanel.hpp	21
SystemContext	Panels	Source/Editor/Panels/SettingsPanels.hpp	22
SystemContext	Panels	Source/Editor/Panels/SkeletonAssetPanel.hpp	13
SystemContext	Panels	Source/Editor/Panels/SpriteAnimationPanel.hpp	13
SystemName	Panels	Source/Editor/Panels/LogBoyPanel.hpp	189
TextPreviewPanel	Panels	Source/Editor/Panels/TextPreviewPanel.hpp	24

TextureThumbnailCache	Panels	Source/Editor/Panels/SpriteAnimationPanel.hpp	19
TilemapEditorPanel	Panels	Source/Editor/Panels/TilemapEditorPanel.hpp	20
TimelinePanelContext	Panels	Source/Editor/Panels/AnimationTimelinePanel.hpp	22
VirtualFileSystem	Panels	Source/Editor/Panels/ContentBrowserPanel.hpp	18
VirtualFileSystem	Panels	Source/Editor/Panels/ProjectDashboard.hpp	13
VirtualFileSystem	Panels	Source/Editor/Panels/TextPreviewPanel.hpp	15
VirtualTexturePanel	Panels	Source/Editor/Panels/VirtualTexturePanel.hpp	18
VirtualTextureSystem	Panels	Source/Editor/Panels/VirtualTexturePanel.hpp	10
VisualScriptPanel	Panels	Source/Editor/Panels/ScriptDebuggerPanel.hpp	24
VisualScriptPanel	Panels	Source/Editor/Panels/VisualScriptPanel.hpp	32
VoiceDevicePanel	Panels	Source/Editor/Panels/VoiceDevicePanel.hpp	19
WidgetCanvas	Panels	Source/Editor/Panels/SettingsPanels.hpp	16
WindowManager	Panels	Source/Editor/Panels/ProjectDashboard.hpp	14

Engine/Assets (57)

Symbol	Kind	Header	Line
AeadAlgorithm	Assets	Source/Engine/Assets/FemCrypto.hpp	18
AeadKey	Assets	Source/Engine/Assets/FemCrypto.hpp	39
AeadNonce	Assets	Source/Engine/Assets/FemCrypto.hpp	46
AeadTag	Assets	Source/Engine/Assets/FemCrypto.hpp	54
ArchiveBuildOptions	Assets	Source/Engine/Assets/FemArchiver.hpp	99
ArchiveCipher	Assets	Source/Engine/Assets/FemArchiver.hpp	29
ArchiveCodec	Assets	Source/Engine/Assets/FemArchiver.hpp	22
ArchiveEntryView	Assets	Source/Engine/Assets/FemArchiver.hpp	115
ArchiveInput	Assets	Source/Engine/Assets/FemArchiver.hpp	72
ArchiveProgress	Assets	Source/Engine/Assets/FemArchiver.hpp	85
ArchiveProgressStage	Assets	Source/Engine/Assets/FemArchiver.hpp	78
ArchiveReader	Assets	Source/Engine/Assets/FemArchiver.hpp	126
ArchiveReader	Assets	Source/Engine/Assets/FemArchiver.hpp	145
ArchiveStatus	Assets	Source/Engine/Assets/	36

		FemArchiver.hpp	
AssetDatabase	Assets	Source/Engine/ Assets/ AssetDatabase.hpp	28
AssetDependency	Assets	Source/Engine/ Assets/AssetMeta.hpp	83
AssetHandle	Assets	Source/Engine/ Assets/AssetMeta.hpp	22
AssetImportSettings	Assets	Source/Engine/ Assets/AssetMeta.hpp	45
AssetMeta	Assets	Source/Engine/ Assets/AssetMeta.hpp	61
AssetRecord	Assets	Source/Engine/ Assets/ AssetDatabase.hpp	19
AstcBlock	Assets	Source/Engine/ Assets/ TextureCook.hpp	25
AtlasDesc	Assets	Source/Engine/ Assets/ TextureArray.hpp	28
AtlasLayer	Assets	Source/Engine/ Assets/ TextureArray.hpp	21
AudioClipHeader	Assets	Source/Engine/ Assets/ AudioClipFormat.hpp	87
BlockExtent	Assets	Source/Engine/ Assets/ TextureFormat.hpp	79
CleanRenderExtract	Assets	Source/Engine/ Assets/ ProjectScaffold.hpp	127
CookedAudioView	Assets	Source/Engine/ Assets/ AudioClipFormat.hpp	116
CookedTexture	Assets	Source/Engine/ Assets/ TextureCook.hpp	41
CookedTextureView	Assets	Source/Engine/ Assets/ TextureFormat.hpp	179
CookProgressFlags	Assets	Source/Engine/ Assets/	31

		CookProgress.hpp	
CookProgressRecord	Assets	Source/Engine/Assets/CookProgress.hpp	38
CookProgressStage	Assets	Source/Engine/Assets/CookProgress.hpp	18
CookTarget	Assets	Source/Engine/Assets/TextureCook.hpp	17
FanmHeader	Assets	Source/Engine/Assets/AnimationClipAsset.hpp	22
FemArchiver	Assets	Source/Engine/Assets/FemArchiver.hpp	128
FempakInput	Assets	Source/Engine/Assets/FempakWriter.hpp	15
FempakWriter	Assets	Source/Engine/Assets/FempakWriter.hpp	22
LocaleTable	Assets	Source/Engine/Assets/Localization.hpp	14
LocalizationGender	Assets	Source/Engine/Assets/Localization.hpp	21
LocalizationPack	Assets	Source/Engine/Assets/Localization.hpp	13
LocalizationTable	Assets	Source/Engine/Assets/Localization.hpp	28
MappedFile	Assets	Source/Engine/Assets/VFS.hpp	19
MountRecord	Assets	Source/Engine/Assets/VFS.hpp	128
PakEntry	Assets	Source/Engine/Assets/PakFormat.hpp	26
PakHeader	Assets	Source/Engine/Assets/	16

		PakFormat.hpp	
PcmFormat	Assets	Source/Engine/ Assets/ AudioClipFormat.hpp	21
PinnedFile	Assets	Source/Engine/ Assets/VFS.hpp	63
ReloadNotice	Assets	Source/Engine/ Assets/ AssetDatabase.hpp	86
StagedRow	Assets	Source/Engine/ Assets/ AssetDatabase.hpp	108
StringEntry	Assets	Source/Engine/ Assets/ Localization.hpp	15
TextureArrayAsset	Assets	Source/Engine/ Assets/ TextureArray.hpp	46
TextureCookOptions	Assets	Source/Engine/ Assets/ TextureCook.hpp	31
TextureFormat	Assets	Source/Engine/ Assets/ TextureFormat.hpp	22
TextureHeader	Assets	Source/Engine/ Assets/ TextureFormat.hpp	160
VfsEntry	Assets	Source/Engine/ Assets/VFS.hpp	55
VirtualFileSystem	Assets	Source/Engine/ Assets/ TextureArray.hpp	16
VirtualFileSystem	Assets	Source/Engine/ Assets/VFS.hpp	69

Engine/Audio (78)

Symbol	Kind	Header	Line
AudioBus	Audio	Source/Engine/ Audio/Audio.hpp	26
AudioBusComponent	Audio	Source/Engine/ Audio/Audio.hpp	40
AudioCaptureMonito	Audio	Source/Engine/ Audio/	144

r		AudioDevices.hpp	
AudioClip	Audio	Source/Engine/ Audio/Audio.hpp	69
AudioCommand	Audio	Source/Engine/ Audio/DspGraph.hpp	138
AudioCommandType	Audio	Source/Engine/ Audio/DspGraph.hpp	121
AudioContextComponent	Audio	Source/Engine/ Audio/ AudioZones.hpp	67
AudioDeviceEnumerator	Audio	Source/Engine/ Audio/ AudioDevices.hpp	93
AudioDeviceInfo	Audio	Source/Engine/ Audio/ AudioDevices.hpp	54
AudioDeviceKind	Audio	Source/Engine/ Audio/ AudioDevices.hpp	15
AudioDeviceList	Audio	Source/Engine/ Audio/ AudioDevices.hpp	66
AudioDevicePreferences	Audio	Source/Engine/ Audio/ AudioDevices.hpp	73
AudioDeviceStatus	Audio	Source/Engine/ Audio/ AudioDevices.hpp	21
AudioLimiter	Audio	Source/Engine/ Audio/Limiter.hpp	23
AudioListener	Audio	Source/Engine/ Audio/Audio.hpp	94
AudioSource	Audio	Source/Engine/ Audio/Audio.hpp	79
AudioSourceComponent	Audio	Source/Engine/ Audio/Audio.hpp	53
AudioSystem	Audio	Source/Engine/ Audio/Audio.hpp	100
AudioZoneComponent	Audio	Source/Engine/ Audio/ AudioZones.hpp	32
AudioZoneKind	Audio	Source/Engine/ Audio/	14

		AudioZones.hpp	
AudioZoneSystem	Audio	Source/Engine/ Audio/ AudioZones.hpp	91
BusAtomics	Audio	Source/Engine/ Audio/DspGraph.hpp	370
BusFilterParams	Audio	Source/Engine/ Audio/DspGraph.hpp	108
BusRuntime	Audio	Source/Engine/ Audio/Audio.hpp	212
BusShadow	Audio	Source/Engine/ Audio/DspGraph.hpp	291
BusState	Audio	Source/Engine/ Audio/DspGraph.hpp	338
CommandQueue	Audio	Source/Engine/ Audio/DspGraph.hpp	170
CookedClipRecord	Audio	Source/Engine/ Audio/Audio.hpp	187
DspClipView	Audio	Source/Engine/ Audio/DspGraph.hpp	205
DspGraph	Audio	Source/Engine/ Audio/DspGraph.hpp	216
DspGraph	Audio	Source/Engine/ Audio/ VoiceChannel.hpp	16
DspResult	Audio	Source/Engine/ Audio/DspGraph.hpp	34
DspResult	Audio	Source/Engine/ Audio/ VoiceChannel.hpp	19
DuckingParams	Audio	Source/Engine/ Audio/DspGraph.hpp	80
DuckingState	Audio	Source/Engine/ Audio/DspGraph.hpp	298
FilterState	Audio	Source/Engine/ Audio/DspGraph.hpp	327
FloatRing	Audio	Source/Engine/ Audio/Audio.hpp	24
FloatRing	Audio	Source/Engine/ Audio/ AudioStream.hpp	16
FloatRing	Audio	Source/Engine/	32

		Audio/DspGraph.hpp	
JitterSlot	Audio	Source/Engine/ Audio/ VoiceChannel.hpp	110
LimiterDesc	Audio	Source/Engine/ Audio/Limiter.hpp	11
OggOpusStream	Audio	Source/Engine/ Audio/OggOpus.hpp	131
OggPacket	Audio	Source/Engine/ Audio/OggOpus.hpp	76
OggPacketReader	Audio	Source/Engine/ Audio/OggOpus.hpp	86
OggPage	Audio	Source/Engine/ Audio/OggOpus.hpp	36
OggPageReader	Audio	Source/Engine/ Audio/OggOpus.hpp	57
OpusHeadInfo	Audio	Source/Engine/ Audio/OggOpus.hpp	106
OwnedPcmRecord	Audio	Source/Engine/ Audio/Audio.hpp	194
Peer	Audio	Source/Engine/ Audio/ VoiceChannel.hpp	120
RateBucket	Audio	Source/Engine/ Audio/ VoiceNetworkBridge.hpp	62
ResolvedZone	Audio	Source/Engine/ Audio/ AudioZones.hpp	113
ReverbParams	Audio	Source/Engine/ Audio/DspGraph.hpp	96
ReverbState	Audio	Source/Engine/ Audio/DspGraph.hpp	311
SourceRuntime	Audio	Source/Engine/ Audio/Audio.hpp	204
State	Audio	Source/Engine/ Audio/ AudioDevices.hpp	164
Stats	Audio	Source/Engine/ Audio/ AudioZones.hpp	94
VoiceAec	Audio	Source/Engine/	44

		Audio/VoiceAec.hpp	
VoiceAecSettings	Audio	Source/Engine/ Audio/VoiceAec.hpp	22
VoiceCapture	Audio	Source/Engine/ Audio/ AudioStream.hpp	64
VoiceChannel	Audio	Source/Engine/ Audio/ VoiceChannel.hpp	65
VoiceChannelConfig	Audio	Source/Engine/ Audio/ VoiceChannel.hpp	36
VoiceChannelStats	Audio	Source/Engine/ Audio/ VoiceChannel.hpp	46
VoiceCodecConfig	Audio	Source/Engine/ Audio/ VoiceCodec.hpp	15
VoiceDecoder	Audio	Source/Engine/ Audio/ VoiceCodec.hpp	51
VoiceDenoiseTier	Audio	Source/Engine/ Audio/VoiceDsp.hpp	10
VoiceDspChain	Audio	Source/Engine/ Audio/VoiceDsp.hpp	99
VoiceDspSettings	Audio	Source/Engine/ Audio/VoiceDsp.hpp	43
VoiceEncoder	Audio	Source/Engine/ Audio/ VoiceCodec.hpp	27
VoiceNetworkBridge	Audio	Source/Engine/ Audio/ VoiceNetworkBridge.hpp	18
VoiceParticipant	Audio	Source/Engine/ Audio/ VoiceSession.hpp	19
VoicePresetKind	Audio	Source/Engine/ Audio/VoiceDsp.hpp	68
VoiceRouting	Audio	Source/Engine/ Audio/ VoiceChannel.hpp	30
VoiceSession	Audio	Source/Engine/ Audio/	15

		VoiceChannel.hpp	
VoiceSession	Audio	Source/Engine/ Audio/ VoiceSession.hpp	31
VoiceState	Audio	Source/Engine/ Audio/DspGraph.hpp	350
VoiceSystem	Audio	Source/Engine/ Audio/ VoiceSystem.hpp	34
VoiceSystemConfig	Audio	Source/Engine/ Audio/ VoiceSystem.hpp	20
VorbisIdentInfo	Audio	Source/Engine/ Audio/OggOpus.hpp	121

Engine/Core (87)

Symbol	Kind	Header	Line
AssetId	Core	Source/Engine/Core/ AssetId.hpp	43
AssetKind	Core	Source/Engine/Core/ AssetId.hpp	15
BigIntT	Core	Source/Engine/Core/ BigInt.hpp	179
BigNumberStore	Core	Source/Engine/Core/ BigNumberStore.hpp	41
BinaryKind	Core	Source/Engine/Core/ BigNumberStore.hpp	259
BinaryKind	Core	Source/Engine/Core/ DecimalStore.hpp	206
BufferTag	Core	Source/Engine/Core/ Types.hpp	301
BuildInfo	Core	Source/Engine/Core/ BuildInfo.hpp	12
BuildProfile	Core	Source/Engine/Core/ BuildProfile.hpp	135
CameraView	Core	Source/Engine/Core/ Grid.hpp	337
Cell	Core	Source/Engine/Core/ Grid.hpp	74
CivilDate	Core	Source/Engine/Core/ Calendar.hpp	21

Clock	Core	Source/Engine/Core/Time.hpp	28
ClockTime	Core	Source/Engine/Core/Calendar.hpp	32
Color32	Core	Source/Engine/Core/Types.hpp	203
ComponentDesc	Core	Source/Engine/Core/Mirror.hpp	48
ComponentRegistry	Core	Source/Engine/Core/PluginHost.hpp	27
ComponentType	Core	Source/Engine/Core/PluginHost.hpp	18
DateTime	Core	Source/Engine/Core/Calendar.hpp	44
DecimalStore	Core	Source/Engine/Core/DecimalStore.hpp	17
DecimalT	Core	Source/Engine/Core/Decimal.hpp	34
DurationStyle	Core	Source/Engine/Core/Time.hpp	571
EddsaKeyPair	Core	Source/Engine/Core/CryptoAsymmetric.hpp	31
FieldDesc	Core	Source/Engine/Core/Mirror.hpp	37
FieldKind	Core	Source/Engine/Core/Mirror.hpp	17
FileFilter	Core	Source/Engine/Core/WindowManager.hpp	154
FormatSpec	Core	Source/Engine/Core/BigInt.hpp	36
FrameArena	Core	Source/Engine/Core/FrameArena.hpp	12
FrameTimer	Core	Source/Engine/Core/Time.hpp	501
GridDesc	Core	Source/Engine/Core/Grid.hpp	91
Handle	Core	Source/Engine/Core/BigNumberStore.hpp	20
Handle	Core	Source/Engine/Core/Types.hpp	275

Host	Core	Source/Engine/Core/Plugin.hpp	20
InputState	Core	Source/Engine/Core/WindowManager.hpp	98
JobSystem	Core	Source/Engine/Core/JobSystem.hpp	25
Key	Core	Source/Engine/Core/WindowManager.hpp	30
KeyLess	Core	Source/Engine/Core/FlatMap.hpp	198
LoadedPlugin	Core	Source/Engine/Core/PluginHost.hpp	64
LogLevel	Core	Source/Engine/Core/Log.hpp	12
Mat4	Core	Source/Engine/Core/Mat4.hpp	14
NoiseGenerator	Core	Source/Engine/Core/NoiseGenerator.hpp	10
Notation	Core	Source/Engine/Core/BigInt.hpp	30
NoticeEntry	Core	Source/Engine/Core/ThirdPartyNotices.hpp	12
OptimizationAdvice	Core	Source/Engine/Core/BuildProfile.hpp	154
OptimizationLevel	Core	Source/Engine/Core/BuildProfile.hpp	10
Pcg32	Core	Source/Engine/Core/Pcg32.hpp	20
PipelineTag	Core	Source/Engine/Core/Types.hpp	303
PluginHost	Core	Source/Engine/Core/PluginHost.hpp	76
PluginSystem	Core	Source/Engine/Core/PluginHost.hpp	54
ProjectionMode	Core	Source/Engine/Core/Grid.hpp	17
Rect	Core	Source/Engine/Core/Types.hpp	173
Reflection	Core	Source/Engine/Core/Mirror.hpp	66

Reflection	Core	Source/Engine/Core/Mirror.hpp	289
Reflection	Core	Source/Engine/Core/Mirror.hpp	301
RngState	Core	Source/Engine/Core/Pcg32.hpp	11
RoundingMode	Core	Source/Engine/Core/Decimal.hpp	22
SamplerTag	Core	Source/Engine/Core/Types.hpp	304
SecureRng	Core	Source/Engine/Core/SecureRng.hpp	11
Sha256Context	Core	Source/Engine/Core/Sha256.hpp	17
Slot	Core	Source/Engine/Core/BigNumberStore.hpp	252
Slot	Core	Source/Engine/Core/DecimalStore.hpp	199
Status	Core	Source/Engine/Core/BigInt.hpp	18
Stopwatch	Core	Source/Engine/Core/Time.hpp	209
Task	Core	Source/Engine/Core/JobSystem.hpp	96
TextureTag	Core	Source/Engine/Core/Types.hpp	302
TextWriter	Core	Source/Engine/Core/Calendar.hpp	189
ThreadCounter	Core	Source/Engine/Core/JobSystem.hpp	109
ThreadLoad	Core	Source/Engine/Core/JobSystem.hpp	31
TimerFired	Core	Source/Engine/Core/Time.hpp	255
TimerHandle	Core	Source/Engine/Core/Time.hpp	240
TimerPool	Core	Source/Engine/Core/Time.hpp	266
TypeNameOverride	Core	Source/Engine/Core/Definitions.hpp	40
TypeNameOverride	Core	Source/Engine/Core/	283

		Definitions.hpp	
TypeObject	Core	Source/Engine/Core/ Definitions.hpp	185
TypeRecord	Core	Source/Engine/Core/ Definitions.hpp	85
TypeTable	Core	Source/Engine/Core/ Definitions.hpp	93
TypeTraits	Core	Source/Engine/Core/ Definitions.hpp	170
TypeTraits	Core	Source/Engine/Core/ Definitions.hpp	267
Unregistered	Core	Source/Engine/ Core/Mirror.hpp	59
Vector2D	Core	Source/Engine/Core/ Types.hpp	13
Vector3D	Core	Source/Engine/Core/ Types.hpp	90
Vector4D	Core	Source/Engine/Core/ Types.hpp	139
WindowDesc	Core	Source/Engine/ Core/ WindowManager.hpp	175
WindowExtent	Core	Source/Engine/ Core/ WindowManager.hpp	21
WindowManager	Core	Source/Engine/ Core/ WindowManager.hpp	186
WindowMode	Core	Source/Engine/ Core/ WindowManager.hpp	166
X25519KeyPair	Core	Source/Engine/Core/ CryptoAsymmetric.hp p	20

Engine/ECS (185)

Symbol	Kind	Header	Line
AccessibilityKernel	ECS	Source/Engine/ECS/ Schedule.hpp	21
AICommandCompon ent	ECS	Source/Engine/ECS/ StateTree.hpp	150

AICommandKind	ECS	Source/Engine/ECS/StateTree.hpp	141
AnimationTimeline	ECS	Source/Engine/ECS/AnimationTimeline.hpp	90
AnimBoneKey	ECS	Source/Engine/ECS/AnimGraph.hpp	70
AnimClip	ECS	Source/Engine/ECS/AnimGraph.hpp	82
AnimConditionFlags	ECS	Source/Engine/ECS/AnimGraph.hpp	27
AnimGraphSystem	ECS	Source/Engine/ECS/AnimGraph.hpp	174
AnimRestBone	ECS	Source/Engine/ECS/AnimGraph.hpp	100
AnimStateComponent	ECS	Source/Engine/ECS/AnimGraph.hpp	110
AnimTransition	ECS	Source/Engine/ECS/AnimGraph.hpp	47
AssetDatabase	ECS	Source/Engine/ECS/Scene.hpp	21
AssetDatabase	ECS	Source/Engine/ECS/Schedule.hpp	29
AudioSystem	ECS	Source/Engine/ECS/Schedule.hpp	16
BigIntComponent	ECS	Source/Engine/ECS/Components.hpp	194
CachedTarget	ECS	Source/Engine/ECS/LightSource.hpp	225
Camera2D	ECS	Source/Engine/ECS/Components.hpp	80
CameraStackSystem	ECS	Source/Engine/ECS/Schedule.hpp	26
ChainIdTag	ECS	Source/Engine/ECS/StableIds.hpp	12
ChainSaveV3	ECS	Source/Engine/ECS/SnapshotWorld.hpp	28
ChangeMaskBit	ECS	Source/Engine/ECS/ChangeMask.hpp	19
CharacterDesc	ECS	Source/Engine/ECS/Descriptors.hpp	51
ClothIdTag	ECS	Source/Engine/ECS/	13

		StableIds.hpp	
ClothSaveV3	ECS	Source/Engine/ECS/ SnapshotWorld.hpp	59
ClothSystem	ECS	Source/Engine/ECS/ Schedule.hpp	23
ColliderSnapshot	ECS	Source/Engine/ECS/ Snapshot.hpp	181
CollisionMode	ECS	Source/Engine/ECS/ Presentation.hpp	21
CollisionPolicyComponent	ECS	Source/Engine/ECS/ Presentation.hpp	101
Command	ECS	Source/Engine/ECS/ CommandBuffer.hpp	29
CommandBuffer	ECS	Source/Engine/ECS/ CommandBuffer.hpp	37
CommandBuffer	ECS	Source/Engine/ECS/ Schedule.hpp	50
CommandKind	ECS	Source/Engine/ECS/ CommandBuffer.hpp	19
CommandLog	ECS	Source/Engine/ECS/ CommandLog.hpp	73
CommandLogHeader	ECS	Source/Engine/ECS/ CommandLog.hpp	47
CommandRecord	ECS	Source/Engine/ECS/ CommandLog.hpp	59
CommandType	ECS	Source/Engine/ECS/ CommandLog.hpp	22
CommandWorldCallbacks	ECS	Source/Engine/ECS/ CommandLog.hpp	37
ComponentChangeMask	ECS	Source/Engine/ECS/ ChangeMask.hpp	52
ComponentOps	ECS	Source/Engine/ECS/ ComponentRegistry.hpp	23
ComponentOpsFor	ECS	Source/Engine/ECS/ ComponentRegistry.hpp	46
ComponentRegistry	ECS	Source/Engine/ECS/ ComponentRegistry.hpp	107
CompressedClip	ECS	Source/Engine/ECS/ AnimationCompressio	47

		n.hpp	
CompressedKey	ECS	Source/Engine/ECS/AnimationCompression.hpp	19
CompressedTrack	ECS	Source/Engine/ECS/AnimationCompression.hpp	31
CompressionReport	ECS	Source/Engine/ECS/AnimationCompression.hpp	63
Cursor	ECS	Source/Engine/ECS/CommandLog.hpp	146
DecimalComponent	ECS	Source/Engine/ECS/Components.hpp	205
DrawOrder	ECS	Source/Engine/ECS/Components.hpp	69
EditorSelected	ECS	Source/Engine/ECS/Components.hpp	94
EmitterIdTag	ECS	Source/Engine/ECS/StableIds.hpp	14
EmitterSaveV3	ECS	Source/Engine/ECS/SnapshotWorld.hpp	79
EntityRemap	ECS	Source/Engine/ECS/Scene.hpp	59
Entry	ECS	Source/Engine/ECS/Schedule.hpp	120
FeskIHeader	ECS	Source/Engine/ECS/SkeletonAsset.hpp	37
FeskINameRef	ECS	Source/Engine/ECS/SkeletonAsset.hpp	87
FeskISprite	ECS	Source/Engine/ECS/SkeletonAsset.hpp	97
FluidSaveV3	ECS	Source/Engine/ECS/SnapshotWorld.hpp	71
FluidWaveSystem	ECS	Source/Engine/ECS/Schedule.hpp	24
FontAtlasSet	ECS	Source/Engine/ECS/Schedule.hpp	37
FrameArena	ECS	Source/Engine/ECS/Schedule.hpp	17
GameplayState	ECS	Source/Engine/ECS/Components.hpp	132

GameplayStateFlag	ECS	Source/Engine/ECS/Components.hpp	106
GraphSystem	ECS	Source/Engine/ECS/SystemGraph.hpp	17
Hidden	ECS	Source/Engine/ECS/Components.hpp	99
IkChain	ECS	Source/Engine/ECS/SkeletonIk.hpp	43
IkComponent	ECS	Source/Engine/ECS/SkeletonIk.hpp	64
IkGoalComponent	ECS	Source/Engine/ECS/SkeletonIk.hpp	77
IkSolver	ECS	Source/Engine/ECS/SkeletonIk.hpp	19
Interpolation	ECS	Source/Engine/ECS/AnimationTimeline.hpp	29
ITextShaper	ECS	Source/Engine/ECS/Schedule.hpp	43
Lifetime	ECS	Source/Engine/ECS/Components.hpp	88
Light2DComponent	ECS	Source/Engine/ECS/Lighting.hpp	33
Light2DOccluderComponent	ECS	Source/Engine/ECS/Lighting.hpp	102
Light2DResultComponent	ECS	Source/Engine/ECS/Lighting.hpp	65
Light2DTargetComponent	ECS	Source/Engine/ECS/Lighting.hpp	49
LightingSettings	ECS	Source/Engine/ECS/Lighting.hpp	113
LightingSystem	ECS	Source/Engine/ECS/Lighting.hpp	138
LightSourceComponent	ECS	Source/Engine/ECS/LightSource.hpp	48
LightSourceFlags	ECS	Source/Engine/ECS/LightSource.hpp	32
LightSourceSettings	ECS	Source/Engine/ECS/LightSource.hpp	149
LightSourceSystem	ECS	Source/Engine/ECS/LightSource.hpp	168
LightSourceType	ECS	Source/Engine/ECS/	22

		LightSource.hpp	
LocalizationTable	ECS	Source/Engine/ECS/Schedule.hpp	22
LocalizedTextComponent	ECS	Source/Engine/ECS/Components.hpp	252
LocalTransform2D	ECS	Source/Engine/ECS/Components.hpp	34
Mat3	ECS	Source/Engine/ECS/Skeleton.hpp	31
NameComponent	ECS	Source/Engine/ECS/Components.hpp	173
NarrativeLabelComponent	ECS	Source/Engine/ECS/Components.hpp	183
NativeCleanupHook	ECS	Source/Engine/ECS/RegistryOps.hpp	12
Parent2D	ECS	Source/Engine/ECS/Components.hpp	42
Phase	ECS	Source/Engine/ECS/Schedule.hpp	55
PhysicsBodyDesc	ECS	Source/Engine/ECS/Descriptors.hpp	22
PhysicsJointSystem	ECS	Source/Engine/ECS/Schedule.hpp	25
PhysicsShapeKind	ECS	Source/Engine/ECS/Descriptors.hpp	15
PhysicsSystem	ECS	Source/Engine/ECS/Schedule.hpp	18
PrefabAsset	ECS	Source/Engine/ECS/Descriptors.hpp	88
PrefabEntityDesc	ECS	Source/Engine/ECS/Descriptors.hpp	76
PrefabEntityFlags	ECS	Source/Engine/ECS/Descriptors.hpp	98
PrefabEntityOverride	ECS	Source/Engine/ECS/PrefabSystem.hpp	18
PrefabOverrides	ECS	Source/Engine/ECS/PrefabSystem.hpp	31
PrefabSystem	ECS	Source/Engine/ECS/PrefabSystem.hpp	36
QuantizedPose	ECS	Source/Engine/ECS/SkeletonIk.hpp	165

RenderMaskComponent	ECS	Source/Engine/ECS/Presentation.hpp	139
RenderPassMask	ECS	Source/Engine/ECS/Presentation.hpp	126
RetargetMap	ECS	Source/Engine/ECS/SkeletonIk.hpp	131
RetargetPair	ECS	Source/Engine/ECS/SkeletonIk.hpp	118
RHI	ECS	Source/Engine/ECS/Schedule.hpp	33
RigidBodySnapshot	ECS	Source/Engine/ECS/Snapshot.hpp	167
SceneComponent	ECS	Source/Engine/ECS/Scene.hpp	38
SceneData	ECS	Source/Engine/ECS/Scene.hpp	52
SceneEntity	ECS	Source/Engine/ECS/Scene.hpp	45
SceneLoadOptions	ECS	Source/Engine/ECS/Scene.hpp	76
SceneMeta	ECS	Source/Engine/ECS/Scene.hpp	31
Schedule	ECS	Source/Engine/ECS/Schedule.hpp	115
SdfLightingFrame	ECS	Source/Engine/ECS/Lighting.hpp	18
SdfSunComponent	ECS	Source/Engine/ECS/Lighting.hpp	77
SkeletonAssetRef	ECS	Source/Engine/ECS/SkeletonAssetRef.hpp	20
SkeletonAssetSource	ECS	Source/Engine/ECS/SkeletonAsset.hpp	189
SkeletonAssetView	ECS	Source/Engine/ECS/SkeletonAsset.hpp	121
SkeletonBoneName	ECS	Source/Engine/ECS/SkeletonAsset.hpp	168
SkeletonComponent	ECS	Source/Engine/ECS/Skeleton.hpp	121
SkeletonNameTable	ECS	Source/Engine/ECS/SkeletonAsset.hpp	175
SkeletonSystem	ECS	Source/Engine/ECS/	198

		Skeleton.hpp	
SkinComponent	ECS	Source/Engine/ECS/ Skeleton.hpp	136
SkinnedSprite	ECS	Source/Engine/ECS/ Skeleton.hpp	147
SleepingStaticComponent	ECS	Source/Engine/ECS/ Presentation.hpp	202
SnapshotBaseline	ECS	Source/Engine/ECS/ Snapshot.hpp	314
SnapshotComponent	ECS	Source/Engine/ECS/ Snapshot.hpp	76
SnapshotComponentOp	ECS	Source/Engine/ECS/ Snapshot.hpp	41
SnapshotExtras	ECS	Source/Engine/ECS/ Snapshot.hpp	265
SnapshotFlags	ECS	Source/Engine/ECS/ Snapshot.hpp	29
SnapshotHeader	ECS	Source/Engine/ECS/ Snapshot.hpp	47
SnapshotRecordFlags	ECS	Source/Engine/ECS/ Snapshot.hpp	35
SnapshotRestoreExtras	ECS	Source/Engine/ECS/ Snapshot.hpp	270
SnapshotSchema	ECS	Source/Engine/ECS/ Snapshot.hpp	91
SnapshotSchema	ECS	Source/Engine/ECS/ Snapshot.hpp	206
SnapshotView	ECS	Source/Engine/ECS/ Snapshot.hpp	255
Sprite2D	ECS	Source/Engine/ECS/ Components.hpp	54
SpriteAnimationComponent	ECS	Source/Engine/ECS/ SpriteAnimation.hpp	21
SpriteAnimationEventQueue	ECS	Source/Engine/ECS/ SpriteAnimation.hpp	74
SpriteAnimationSystem	ECS	Source/Engine/ECS/ SpriteAnimation.hpp	118
SpriteFrameEvent	ECS	Source/Engine/ECS/ SpriteAnimation.hpp	66
SpriteFrameRect	ECS	Source/Engine/ECS/ SpriteAnimation.hpp	55

SpriteSheetComponent	ECS	Source/Engine/ECS/SpriteAnimation.hpp	40
StableIdAllocator	ECS	Source/Engine/ECS/StableIds.hpp	25
StateTreeComponent	ECS	Source/Engine/ECS/StateTree.hpp	167
StateTreeDefinition	ECS	Source/Engine/ECS/StateTree.hpp	130
StateTreeEvaluator	ECS	Source/Engine/ECS/StateTree.hpp	48
StateTreeEvaluatorKind	ECS	Source/Engine/ECS/StateTree.hpp	26
StateTreeNode	ECS	Source/Engine/ECS/StateTree.hpp	94
StateTreeNodeKind	ECS	Source/Engine/ECS/StateTree.hpp	84
StateTreeSystem	ECS	Source/Engine/ECS/StateTree.hpp	212
StateTreeTask	ECS	Source/Engine/ECS/StateTree.hpp	71
StateTreeTaskKind	ECS	Source/Engine/ECS/StateTree.hpp	61
StateTreeTransition	ECS	Source/Engine/ECS/StateTree.hpp	113
SystemContext	ECS	Source/Engine/ECS/Schedule.hpp	83
SystemGraph	ECS	Source/Engine/ECS/Schedule.hpp	51
SystemGraph	ECS	Source/Engine/ECS/SystemGraph.hpp	29
TextComponent	ECS	Source/Engine/ECS/Components.hpp	224
TextConditionComponent	ECS	Source/Engine/ECS/Components.hpp	308
TextEffectComponent	ECS	Source/Engine/ECS/Components.hpp	278
TextFormatComponent	ECS	Source/Engine/ECS/Components.hpp	290
TextService	ECS	Source/Engine/ECS/Schedule.hpp	41
TextShapeCache	ECS	Source/Engine/ECS/	42

		Schedule.hpp	
TextShapeComponent	ECS	Source/Engine/ECS/Components.hpp	267
TilemapAssetRefComponent	ECS	Source/Engine/ECS/Tilemap.hpp	130
TilemapComponent	ECS	Source/Engine/ECS/Tilemap.hpp	63
TilemapRenderSystem	ECS	Source/Engine/ECS/Tilemap.hpp	150
TileQuad	ECS	Source/Engine/ECS/Tilemap.hpp	142
TimelineEvent	ECS	Source/Engine/ECS/AnimationTimeline.hpp	67
TimelineEventKind	ECS	Source/Engine/ECS/AnimationTimeline.hpp	36
TimelineKey	ECS	Source/Engine/ECS/AnimationTimeline.hpp	43
TimelineTrack	ECS	Source/Engine/ECS/AnimationTimeline.hpp	56
TimelineTriggerSink	ECS	Source/Engine/ECS/AnimationTimeline.hpp	83
TimeScaleComponent	ECS	Source/Engine/ECS/Presentation.hpp	168
TrackKind	ECS	Source/Engine/ECS/AnimationTimeline.hpp	20
Transform2D	ECS	Source/Engine/ECS/Components.hpp	23
Velocity	ECS	Source/Engine/ECS/Components.hpp	48
View	ECS	Source/Engine/ECS/CommandLog.hpp	156
VirtualFileSystem	ECS	Source/Engine/ECS/Scene.hpp	18
VirtualFileSystem	ECS	Source/Engine/ECS/Schedule.hpp	20
VirtualFileSystem	ECS	Source/Engine/ECS/SkeletonAsset.hpp	18

VirtualFileSystem	ECS	Source/Engine/ECS/ Tilemap.hpp	20
WindowManager	ECS	Source/Engine/ECS/ Schedule.hpp	19
WorldSnapshotV3	ECS	Source/Engine/ECS/ SnapshotWorld.hpp	86

Engine/Navigation (7)

Symbol	Kind	Header	Line
NavAgentComponent	Navigation	Source/Engine/ Navigation/ Navigation.hpp	161
NavAgentSettings	Navigation	Source/Engine/ Navigation/ Navigation.hpp	139
NavigationSystem	Navigation	Source/Engine/ Navigation/ Navigation.hpp	184
NavMap	Navigation	Source/Engine/ Navigation/ Navigation.hpp	25
NavPath	Navigation	Source/Engine/ Navigation/ Navigation.hpp	67
PathFinder	Navigation	Source/Engine/ Navigation/ Navigation.hpp	91
PathStats	Navigation	Source/Engine/ Navigation/ Navigation.hpp	80

Engine/Network (89)

Symbol	Kind	Header	Line
CellCoordinate	Network	Source/Engine/ Network/ RelevanceGrid.hpp	168
ChecksumComparison	Network	Source/Engine/ Network/ NetRollback.hpp	457
Config	Network	Source/Engine/ Network/ NetRollback.hpp	316

DatagramSocket	Network	Source/Engine/ Network/ Transport.hpp	32
Endpoint	Network	Source/Engine/ Network/NetLink.hpp	92
Frame	Network	Source/Engine/ Network/NetLink.hpp	84
FrameInfo	Network	Source/Engine/ Network/ NetRollback.hpp	221
FrameMeta	Network	Source/Engine/ Network/ NetRollback.hpp	473
InputChannel	Network	Source/Engine/ Network/ NetProtocol.hpp	56
InputChannelStats	Network	Source/Engine/ Network/ NetProtocol.hpp	44
InputPacket	Network	Source/Engine/ Network/ NetProtocol.hpp	24
InputRingBuffer	Network	Source/Engine/ Network/ NetRollback.hpp	123
LobbyError	Network	Source/Engine/ Network/Lobby.hpp	37
LobbyMessageType	Network	Source/Engine/ Network/Lobby.hpp	25
LobbyRoomInfo	Network	Source/Engine/ Network/Lobby.hpp	73
LobbyServer	Network	Source/Engine/ Network/Lobby.hpp	93
LobbyServerConfig	Network	Source/Engine/ Network/Lobby.hpp	82
LocalEntry	Network	Source/Engine/ Network/ NetRollback.hpp	205
Member	Network	Source/Engine/ Network/Lobby.hpp	126
MemoryEndpoint	Network	Source/Engine/ Network/NetLink.hpp	46
MemoryEndpoint	Network	Source/Engine/	113

		Network/NetLink.hpp	
MemoryNetwork	Network	Source/Engine/ Network/NetLink.hpp	53
NetAddress	Network	Source/Engine/ Network/ Transport.hpp	19
NetDiagnosticsSnapshot	Network	Source/Engine/ Network/ Diagnostics.hpp	12
NetDisconnectReason	Network	Source/Engine/ Network/Session.hpp	65
NetEntityId	Network	Source/Engine/ Network/ NetRollback.hpp	39
NetEntityMap	Network	Source/Engine/ Network/ NetRollback.hpp	167
NetEntry	Network	Source/Engine/ Network/ NetRollback.hpp	200
NetIdentity	Network	Source/Engine/ Network/Identity.hpp	32
NetInputFrame	Network	Source/Engine/ Network/ NetRollback.hpp	70
NetInputStatus	Network	Source/Engine/ Network/ NetRollback.hpp	99
NetLink	Network	Source/Engine/ Network/NetLink.hpp	19
NetLobbyClient	Network	Source/Engine/ Network/Lobby.hpp	160
NetLobbyClientConfig	Network	Source/Engine/ Network/Lobby.hpp	151
NetPlayerId	Network	Source/Engine/ Network/ NetRollback.hpp	59
NetPlayerSlot	Network	Source/Engine/ Network/Session.hpp	134
NetRole	Network	Source/Engine/ Network/Session.hpp	32
NetRollbackSession	Network	Source/Engine/ Network/	240

		NetRollback.hpp	
NetSession	Network	Source/Engine/ Network/Session.hpp	189
NetSessionCallbacks	Network	Source/Engine/ Network/Session.hpp	178
NetSessionConfig	Network	Source/Engine/ Network/Lobby.hpp	15
NetSessionConfig	Network	Source/Engine/ Network/Session.hpp	149
NetSessionMode	Network	Source/Engine/ Network/Session.hpp	22
NetSessionStatus	Network	Source/Engine/ Network/Session.hpp	46
NetSlotState	Network	Source/Engine/ Network/Session.hpp	58
NetTicket	Network	Source/Engine/ Network/Ticket.hpp	35
NetTickInputs	Network	Source/Engine/ Network/ NetRollback.hpp	110
NetTickMode	Network	Source/Engine/ Network/Session.hpp	40
NetTransport	Network	Source/Engine/ Network/ Transport.hpp	127
Outstanding	Network	Source/Engine/ Network/ Transport.hpp	100
Peer	Network	Source/Engine/ Network/ SecureSessionTransport.hpp	87
PendingEvent	Network	Source/Engine/ Network/Session.hpp	279
PendingState	Network	Source/Engine/ Network/ Replication.hpp	129
Phase	Network	Source/Engine/ Network/ SecureChannel.hpp	256
PhysicsSystem	Network	Source/Engine/ Network/ NetRollback.hpp	20

RateEntry	Network	Source/Engine/ Network/ SecureSessionTransport.hpp	120
Reassembly	Network	Source/Engine/ Network/ Transport.hpp	112
RelevanceBand	Network	Source/Engine/ Network/ RelevanceGrid.hpp	32
RelevanceCell	Network	Source/Engine/ Network/ RelevanceGrid.hpp	71
RelevanceConfig	Network	Source/Engine/ Network/ NetRollback.hpp	34
RelevanceConfig	Network	Source/Engine/ Network/ RelevanceGrid.hpp	56
RelevanceEntity	Network	Source/Engine/ Network/ RelevanceGrid.hpp	81
RelevanceGrid	Network	Source/Engine/ Network/ NetRollback.hpp	36
RelevanceGrid	Network	Source/Engine/ Network/ RelevanceGrid.hpp	131
RelevanceGridStats	Network	Source/Engine/ Network/ RelevanceGrid.hpp	121
RelevancePlan	Network	Source/Engine/ Network/ NetRollback.hpp	35
RelevancePlan	Network	Source/Engine/ Network/ RelevanceGrid.hpp	93
ReplicationStatus	Network	Source/Engine/ Network/ Replication.hpp	28
Result	Network	Source/Engine/ Network/ NetRollback.hpp	243
RollbackReport	Network	Source/Engine/ Network/	229

		NetRollback.hpp	
Room	Network	Source/Engine/ Network/Lobby.hpp	132
SecureChannel	Network	Source/Engine/ Network/ SecureChannel.hpp	198
SecureChannelConfig	Network	Source/Engine/ Network/ SecureChannel.hpp	151
SecureCookieIssuer	Network	Source/Engine/ Network/ SecureChannel.hpp	131
SecureEndpoint	Network	Source/Engine/ Network/ SecureEndpoint.hpp	16
SecureSessionAuthConfig	Network	Source/Engine/ Network/ SecureSessionTransport.hpp	24
SecureSessionTransport	Network	Source/Engine/ Network/ SecureSessionTransport.hpp	38
SecureStatus	Network	Source/Engine/ Network/ SecureChannel.hpp	65
Slot	Network	Source/Engine/ Network/ NetRollback.hpp	155
SnapshotReceiver	Network	Source/Engine/ Network/ Replication.hpp	177
SnapshotReceiverConfig	Network	Source/Engine/ Network/ Replication.hpp	166
SnapshotSender	Network	Source/Engine/ Network/ Replication.hpp	94
SnapshotSenderConfig	Network	Source/Engine/ Network/ Replication.hpp	70
SnapshotSenderStats	Network	Source/Engine/ Network/ Replication.hpp	80
SortableEntity	Network	Source/Engine/	195

		Network/ RelevanceGrid.hpp	
TicketStatus	Network	Source/Engine/ Network/Ticket.hpp	54
TransportChannel	Network	Source/Engine/ Network/ Transport.hpp	63
TransportPeer	Network	Source/Engine/ Network/ Transport.hpp	85
TransportStats	Network	Source/Engine/ Network/ Transport.hpp	69

Engine/Physics (54)

Symbol	Kind	Header	Line
BodyRecord	Physics	Source/Engine/ Physics/Physics.hpp	203
BodyType	Physics	Source/Engine/ Physics/Physics.hpp	22
ChainError	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	64
ChainRuntime	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	247
ClothComponent	Physics	Source/Engine/ Physics/Cloth.hpp	31
ClothDesc	Physics	Source/Engine/ Physics/Cloth.hpp	63
ClothFlags	Physics	Source/Engine/ Physics/Cloth.hpp	20
ClothHandle	Physics	Source/Engine/ Physics/Cloth.hpp	46
ClothInfo	Physics	Source/Engine/ Physics/Cloth.hpp	99
ClothRenderSystem	Physics	Source/Engine/ Physics/ ClothRenderSystem.hpp	15
ClothSystem	Physics	Source/Engine/ Physics/Cloth.hpp	110

Collider2D	Physics	Source/Engine/ Physics/Physics.hpp	59
CollisionFilterMatrix	Physics	Source/Engine/ Physics/ CollisionFilter.hpp	48
CollisionLayer	Physics	Source/Engine/ Physics/ CollisionFilter.hpp	15
FluidDesc	Physics	Source/Engine/ Physics/Fluid.hpp	114
FluidInteractorComponent	Physics	Source/Engine/ Physics/Fluid.hpp	208
FluidWaveSystem	Physics	Source/Engine/ Physics/Fluid.hpp	219
GerstnerField	Physics	Source/Engine/ Physics/Fluid.hpp	49
GerstnerSample	Physics	Source/Engine/ Physics/Fluid.hpp	39
GerstnerWave	Physics	Source/Engine/ Physics/Fluid.hpp	25
JointChainComponent	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	158
JointChainDesc	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	98
JointChainFlags	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	45
JointChainHandle	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	139
JointChainInfo	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	190
JointChainKind	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	37
JointLinkComponent	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	17
ParticleEmitterComponent	Physics	Source/Engine/ Physics/Particles.hpp	26
ParticleEmitterFlags	Physics	Source/Engine/	14

		Physics/Particles.hpp	
ParticleEmitterGpu	Physics	Source/Engine/ Physics/Particles.hpp	61
ParticleSeedPush	Physics	Source/Engine/ Physics/Particles.hpp	84
ParticleStateGpu	Physics	Source/Engine/ Physics/Particles.hpp	92
PhysicsBodyRemap Entry	Physics	Source/Engine/ Physics/ SnapshotRemap.hpp	17
PhysicsBodyRemap Table	Physics	Source/Engine/ Physics/ SnapshotRemap.hpp	24
PhysicsContactEven t	Physics	Source/Engine/ Physics/Physics.hpp	87
PhysicsDesc	Physics	Source/Engine/ Physics/Physics.hpp	98
PhysicsEventType	Physics	Source/Engine/ Physics/Physics.hpp	80
PhysicsJointSystem	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	208
PhysicsSystem	Physics	Source/Engine/ Physics/Physics.hpp	107
RigidBody2D	Physics	Source/Engine/ Physics/Physics.hpp	42
RigidBodyFlags	Physics	Source/Engine/ Physics/Physics.hpp	29
Runtime	Physics	Source/Engine/ Physics/Cloth.hpp	156
SdfCollisionDesc	Physics	Source/Engine/ Physics/ SdfCollision.hpp	45
SdfCollisionSystem	Physics	Source/Engine/ Physics/ SdfCollision.hpp	71
SdfContact	Physics	Source/Engine/ Physics/ SdfCollision.hpp	59
SdfField	Physics	Source/Engine/ Physics/ SdfCollision.hpp	19
SdfPrimitiveInfo	Physics	Source/Engine/	137

		Physics/ SdfPrimitives.hpp	
SdfShape	Physics	Source/Engine/ Physics/ SdfPrimitives.hpp	82
SdfShapeComponent	Physics	Source/Engine/ Physics/ SdfPrimitives.hpp	127
SdfShapeType	Physics	Source/Engine/ Physics/ SdfPrimitives.hpp	16
ShallowWaterGrid	Physics	Source/Engine/ Physics/Fluid.hpp	134
Storage	Physics	Source/Engine/ Physics/Cloth.hpp	155
Tracked	Physics	Source/Engine/ Physics/Fluid.hpp	248
WaterView	Physics	Source/Engine/ Physics/Fluid.hpp	85

Engine/Platform (21)

Symbol	Kind	Header	Line
AchievementFlags	Platform	Source/Engine/ Platform/ PlatformSystems.hpp	95
AchievementRequest	Platform	Source/Engine/ Platform/ PlatformSystems.hpp	104
CloudSaveComponent	Platform	Source/Engine/ Platform/ PlatformSystems.hpp	124
CloudSaveFlags	Platform	Source/Engine/ Platform/ PlatformSystems.hpp	115
EgsStoreSystem	Platform	Source/Engine/ Platform/ PlatformSystems.hpp	178
GogAchievementSystem	Platform	Source/Engine/ Platform/ PlatformSystems.hpp	224
GogStorageSystem	Platform	Source/Engine/ Platform/ PlatformSystems.hpp	258

PlatformAccountComponent	Platform	Source/Engine/Platform/PlatformSystems.hpp	137
PlatformCommand	Platform	Source/Engine/Platform/PlatformService.hpp	35
PlatformCommandLog	Platform	Source/Engine/Platform/PlatformSystems.hpp	46
PlatformCommandType	Platform	Source/Engine/Platform/PlatformService.hpp	24
PlatformLogEvent	Platform	Source/Engine/Platform/PlatformSystems.hpp	35
PlatformLogType	Platform	Source/Engine/Platform/PlatformSystems.hpp	26
PlatformService	Platform	Source/Engine/Platform/PlatformService.hpp	76
PlatformServiceCallbacks	Platform	Source/Engine/Platform/PlatformService.hpp	51
PlatformServiceDesc	Platform	Source/Engine/Platform/PlatformService.hpp	71
PlatformServiceStats	Platform	Source/Engine/Platform/PlatformService.hpp	59
PlatformState	Platform	Source/Engine/Platform/PlatformService.hpp	132
StagedSave	Platform	Source/Engine/Platform/PlatformSystems.hpp	204
StagedSave	Platform	Source/Engine/Platform/PlatformSystems.hpp	291
SteamAchievementSystem	Platform	Source/Engine/Platform/PlatformSystems.hpp	148

Engine/RHI (112)

Symbol	Kind	Header	Line
Api	RHI	Source/Engine/RHI/Vulkan/VulkanLoader.hpp	113
Backend	RHI	Source/Engine/RHI/RHI.hpp	19
CameraData	RHI	Source/Engine/RHI/RenderItem.hpp	209
CameraStackDesc	RHI	Source/Engine/RHI/CameraStack.hpp	27
CameraStackFlags	RHI	Source/Engine/RHI/CameraStack.hpp	17
CameraStackSystem	RHI	Source/Engine/RHI/CameraStack.hpp	52
CameraStackSystem	RHI	Source/Engine/RHI/RenderGraph.hpp	16
CameraStackSystem	RHI	Source/Engine/RHI/RenderPasses.hpp	12
ChangeMaskRenderExtract	RHI	Source/Engine/RHI/ChangeMaskExtract.hpp	33
ColorGradingConstants	RHI	Source/Engine/RHI/RenderPasses.hpp	116
ColorGradingPass	RHI	Source/Engine/RHI/RenderPasses.hpp	124
ColorGradingSettings	RHI	Source/Engine/RHI/RenderPasses.hpp	104
CookedMaterialView	RHI	Source/Engine/RHI/MaterialAsset.hpp	152
DeviceDesc	RHI	Source/Engine/RHI/RHI.hpp	49
DeviceInfo	RHI	Source/Engine/RHI/RHI.hpp	74
DirtyRange	RHI	Source/Engine/RHI/RenderItem.hpp	190
DrawBatch	RHI	Source/Engine/RHI/RenderItem.hpp	170
Entry	RHI	Source/Engine/RHI/MaterialPipelineCache.hpp	74
ExportStatus	RHI	Source/Engine/RHI/	100

		FrameExporter.hpp	
Extent2D	RHI	Source/Engine/RHI/RHI.hpp	40
ExtractStats	RHI	Source/Engine/RHI/ChangeMaskExtract.hpp	21
FrameAccessibility	RHI	Source/Engine/RHI/RenderItem.hpp	231
FrameAccessibilityFlags	RHI	Source/Engine/RHI/RenderItem.hpp	218
FrameExportDesc	RHI	Source/Engine/RHI/FrameExporter.hpp	53
FrameExporter	RHI	Source/Engine/RHI/FrameExporter.hpp	112
FramePacket	RHI	Source/Engine/RHI/RenderItem.hpp	304
FrameSink	RHI	Source/Engine/RHI/FrameExporter.hpp	85
FrameStats	RHI	Source/Engine/RHI/RHI.hpp	182
HashEntry	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	197
KeySorter	RHI	Source/Engine/RHI/Sort.hpp	14
LookupEntry	RHI	Source/Engine/RHI/ChangeMaskExtract.hpp	255
MaterialAssetError	RHI	Source/Engine/RHI/MaterialAsset.hpp	42
MaterialCompileOptions	RHI	Source/Engine/RHI/MaterialGraphCompiler.hpp	27
MaterialContract	RHI	Source/Engine/RHI/MaterialAsset.hpp	83
MaterialDomain	RHI	Source/Engine/RHI/MaterialGraphCompiler.hpp	21
MaterialGraph	RHI	Source/Engine/RHI/MaterialGraph.hpp	555
MaterialGraphCompiler	RHI	Source/Engine/RHI/MaterialGraphCompiler.hpp	58

MaterialGraphRequirements	RHI	Source/Engine/RHI/MaterialGraph.hpp	542
MaterialNode	RHI	Source/Engine/RHI/MaterialGraph.hpp	117
MaterialNodeFlags	RHI	Source/Engine/RHI/MaterialGraph.hpp	111
MaterialNodeKind	RHI	Source/Engine/RHI/MaterialGraph.hpp	29
MaterialParam	RHI	Source/Engine/RHI/MaterialGraph.hpp	99
MaterialParamKind	RHI	Source/Engine/RHI/MaterialGraph.hpp	88
MaterialPin	RHI	Source/Engine/RHI/MaterialGraph.hpp	129
MaterialPipelineCache	RHI	Source/Engine/RHI/MaterialPipelineCache.hpp	38
MaterialPipelineCacheStats	RHI	Source/Engine/RHI/MaterialPipelineCache.hpp	18
MaterialShaderProgram	RHI	Source/Engine/RHI/MaterialGraphCompiler.hpp	39
MaterialValueType	RHI	Source/Engine/RHI/MaterialGraph.hpp	78
MeshDesc	RHI	Source/Engine/RHI/Mesh.hpp	102
MeshDrawData	RHI	Source/Engine/RHI/Mesh.hpp	70
MeshDrawFlags	RHI	Source/Engine/RHI/Mesh.hpp	54
MeshHandle	RHI	Source/Engine/RHI/Mesh.hpp	87
MeshItemFlags	RHI	Source/Engine/RHI/Mesh.hpp	62
MeshRenderItem	RHI	Source/Engine/RHI/Mesh.hpp	114
MeshSkinRange	RHI	Source/Engine/RHI/Mesh.hpp	46
MeshVertex	RHI	Source/Engine/RHI/Mesh.hpp	17
NativeTextureView	RHI	Source/Engine/RHI/	151

		RHI.hpp	
PageSlot	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	188
PostFxConstants	RHI	Source/Engine/RHI/RenderPasses.hpp	89
PostFxFlags	RHI	Source/Engine/RHI/PostProcess.hpp	10
PostFxPass	RHI	Source/Engine/RHI/RenderPasses.hpp	145
PostFxSettings	RHI	Source/Engine/RHI/PostProcess.hpp	27
RenderCommandRecord	RHI	Source/Engine/RHI/RenderGraph.hpp	52
RenderFlags	RHI	Source/Engine/RHI/RenderItem.hpp	17
RenderGraph	RHI	Source/Engine/RHI/RenderGraph.hpp	119
RenderGraphContext	RHI	Source/Engine/RHI/RenderGraph.hpp	69
RenderGraphPass	RHI	Source/Engine/RHI/RenderGraph.hpp	106
RenderItem	RHI	Source/Engine/RHI/RenderItem.hpp	59
RenderPassDesc	RHI	Source/Engine/RHI/RenderGraph.hpp	94
RenderPassKind	RHI	Source/Engine/RHI/RenderGraph.hpp	26
RenderResourceKind	RHI	Source/Engine/RHI/RenderGraph.hpp	33
RenderTargetDesc	RHI	Source/Engine/RHI/RHI.hpp	134
RenderTargetHandle	RHI	Source/Engine/RHI/RHI.hpp	140
RHI	RHI	Source/Engine/RHI/RHI.hpp	228
SdfLightingFrame	RHI	Source/Engine/RHI/RenderItem.hpp	279
SdfLightingOccluder	RHI	Source/Engine/RHI/RenderItem.hpp	271
SdfShadowConstant	RHI	Source/Engine/RHI/	37

s		RenderPasses.hpp	
SdfShadowDesc	RHI	Source/Engine/RHI/RenderPasses.hpp	18
SdfShadowPass	RHI	Source/Engine/RHI/RenderPasses.hpp	58
SequenceFooter	RHI	Source/Engine/RHI/FrameExporter.hpp	42
SequenceHeader	RHI	Source/Engine/RHI/FrameExporter.hpp	31
ShaderPipelineDesc	RHI	Source/Engine/RHI/RHI.hpp	111
ShaderPipelineDomain	RHI	Source/Engine/RHI/RHI.hpp	105
ShaderPipelineHandle	RHI	Source/Engine/RHI/RHI.hpp	122
SkinnedVertex	RHI	Source/Engine/RHI/Mesh.hpp	30
Slot	RHI	Source/Engine/RHI/CameraStack.hpp	96
TextureArrayDesc	RHI	Source/Engine/RHI/RHI.hpp	90
TextureArrayHandle	RHI	Source/Engine/RHI/RHI.hpp	204
TintEntry	RHI	Source/Engine/RHI/ViewportPresenter.hpp	81
TransientRing	RHI	Source/Engine/RHI/RingBuffer.hpp	15
UiBackendInfo	RHI	Source/Engine/RHI/RHI.hpp	159
ViewportMode	RHI	Source/Engine/RHI/ViewportModes.hpp	15
ViewportModeDescriptor	RHI	Source/Engine/RHI/ViewportModes.hpp	46
ViewportModeLegend	RHI	Source/Engine/RHI/ViewportModes.hpp	37
ViewportPresenter	RHI	Source/Engine/RHI/ViewportPresenter.hpp	29
ViewportPresenterStats	RHI	Source/Engine/RHI/ViewportPresenter.hpp	17

VirtualFileSystem	RHI	Source/Engine/RHI/FrameExporter.hpp	15
VirtualFileSystem	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	16
VirtualTextureAddress	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	68
VirtualTextureCamera	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	59
VirtualTextureDesc	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	33
VirtualTextureHandle	RHI	Source/Engine/RHI/RenderItem.hpp	248
VirtualTexturePageLocation	RHI	Source/Engine/RHI/VirtualTextureGpu.hpp	11
VirtualTextureResourceDesc	RHI	Source/Engine/RHI/RHI.hpp	216
VirtualTextureRhiTarget	RHI	Source/Engine/RHI/VirtualTextureGpu.hpp	118
VirtualTextureSample	RHI	Source/Engine/RHI/VirtualTextureGpu.hpp	58
VirtualTextureStats	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	45
VirtualTextureSystem	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	111
VirtualTextureTableUpdate	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	101
VirtualTextureTileId	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	92
VirtualTextureTileUpload	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	77
VisibleSprite	RHI	Source/Engine/RHI/ViewportPresenter.hpp	87

Engine/Script (62)

Symbol	Kind	Header	Line
AotBuilder	Script	Source/Engine/Script/AotBuilder.hpp	71
AotBuildOptions	Script	Source/Engine/Script/AotBuilder.hpp	43
AotBuildResult	Script	Source/Engine/Script/AotBuilder.hpp	60
AotBuildState	Script	Source/Engine/Script/AotBuilder.hpp	19
AotGraphTranspileOptions	Script	Source/Engine/Script/AotGraphCompiler.hpp	13
AotHost	Script	Source/Engine/Script/AotHost.hpp	32
AotHost	Script	Source/Engine/Script/ScriptSystem.hpp	21
AotModule	Script	Source/Engine/Script/AotModule.hpp	22
AotProgramRef	Script	Source/Engine/Script/AotHost.hpp	22
AotTranspileOptions	Script	Source/Engine/Script/AotTranspiler.hpp	12
BoundProgram	Script	Source/Engine/Script/EcsBridge.hpp	101
Code	Script	Source/Engine/Script/Vm.hpp	25
CompiledDebug	Script	Source/Engine/Script/Bytecode.hpp	452
CompiledProgram	Script	Source/Engine/Script/Bytecode.hpp	467
ComponentAccessor	Script	Source/Engine/Script/EcsBridge.hpp	17
ComponentBinding	Script	Source/Engine/Script/Bytecode.hpp	396
Context	Script	Source/Engine/Script/AotAbi.hpp	98
DebugField	Script	Source/Engine/Script/	123

		ScriptDebug.hpp	
DebugFrame	Script	Source/Engine/ Script/ ScriptDebug.hpp	103
DebugSectionHeader	Script	Source/Engine/ Script/Bytecode.hpp	145
DebugSpan	Script	Source/Engine/ Script/Bytecode.hpp	123
DebugSpanFlags	Script	Source/Engine/ Script/Bytecode.hpp	114
DebugVariable	Script	Source/Engine/ Script/Bytecode.hpp	136
EcsBridge	Script	Source/Engine/ Script/EcsBridge.hpp	26
FieldBinding	Script	Source/Engine/ Script/Bytecode.hpp	406
FunctionEntry	Script	Source/Engine/ Script/Bytecode.hpp	419
HostApi	Script	Source/Engine/ Script/AotAbi.hpp	54
Instruction	Script	Source/Engine/ Script/Bytecode.hpp	376
LogicCompareOp	Script	Source/Engine/ Script/ GraphCompiler.hpp	22
LogicCompileOptions	Script	Source/Engine/ Script/ GraphCompiler.hpp	669
LogicCompileResult	Script	Source/Engine/ Script/ GraphCompiler.hpp	676
LogicFieldBinding	Script	Source/Engine/ Script/ GraphCompiler.hpp	90
LogicGraph	Script	Source/Engine/ Script/ GraphCompiler.hpp	319
LogicLink	Script	Source/Engine/ Script/ GraphCompiler.hpp	308
LogicMathOp	Script	Source/Engine/ Script/ GraphCompiler.hpp	56

LogicNode	Script	Source/Engine/ Script/ GraphCompiler.hpp	290
LogicNodeKind	Script	Source/Engine/ Script/ GraphCompiler.hpp	111
Module	Script	Source/Engine/ Script/AotAbi.hpp	293
NativeId	Script	Source/Engine/ Script/Bytecode.hpp	18
OpCode	Script	Source/Engine/ Script/Bytecode.hpp	194
PendingField	Script	Source/Engine/ Script/Bytecode.hpp	533
PinType	Script	Source/Engine/ Script/ GraphCompiler.hpp	275
ProgramBuilder	Script	Source/Engine/ Script/Bytecode.hpp	498
ProgramHeader	Script	Source/Engine/ Script/Bytecode.hpp	429
ResolvedComponent	Script	Source/Engine/ Script/EcsBridge.hpp	93
ScriptComponent	Script	Source/Engine/ Script/ ScriptComponent.hpp	16
ScriptEvent	Script	Source/Engine/ Script/Bytecode.hpp	169
ScriptRuntime	Script	Source/Engine/ Script/ ScriptSystem.hpp	20
ScriptRuntime	Script	Source/Engine/ Script/ ScriptSystem.hpp	24
ScriptVm	Script	Source/Engine/ Script/Vm.hpp	101
ScriptVm	Script	Source/Engine/ Script/Vm.hpp	215
Slot	Script	Source/Engine/ Script/AotHost.hpp	94
Slot	Script	Source/Engine/ Script/ ScriptSystem.hpp	112

SourceLocation	Script	Source/Engine/ Script/ ScriptDebug.hpp	17
TextCompileOptions	Script	Source/Engine/ Script/TextScript.hpp	14
Vector2D	Script	Source/Engine/ Script/AotAbi.hpp	28
VmContext	Script	Source/Engine/ Script/Vm.hpp	87
VmDebugState	Script	Source/Engine/ Script/Vm.hpp	140
VmFault	Script	Source/Engine/ Script/Vm.hpp	23
VmHalt	Script	Source/Engine/ Script/Vm.hpp	113
VmPauseReason	Script	Source/Engine/ Script/Vm.hpp	104
VmStopInfo	Script	Source/Engine/ Script/Vm.hpp	120

Engine/Text (67)

Symbol	Kind	Header	Line
AssetSlot	Text	Source/Engine/Text/ TextStore.hpp	218
ChainedShaper	Text	Source/Engine/Text/ TextShaper.hpp	133
CodepointShaper	Text	Source/Engine/Text/ TextShaper.hpp	99
Entry	Text	Source/Engine/Text/ TextShaper.hpp	247
FontCandidate	Text	Source/Engine/Text/ TextFontStack.hpp	24
FontSlot	Text	Source/Engine/Text/ TextFontStack.hpp	59
FormatEntry	Text	Source/Engine/Text/ TextService.hpp	583
FormattedLayoutEntry	Text	Source/Engine/Text/ TextService.hpp	566
FormattedLayoutKey	Text	Source/Engine/Text/ TextService.hpp	556
HarfBuzzShaper	Text	Source/Engine/Text/	110

		TextShaper.hpp	
Impl	Text	Source/Engine/Text/ TextShaper.hpp	128
ITextShaper	Text	Source/Engine/Text/ TextShaper.hpp	89
LayoutEntry	Text	Source/Engine/Text/ TextService.hpp	548
LayoutKey	Text	Source/Engine/Text/ TextService.hpp	528
MappedTextAsset	Text	Source/Engine/Text/ TextAssetFile.hpp	17
MarkupStackEntry	Text	Source/Engine/Text/ TextMarkup.hpp	71
MarkupVariantEntry	Text	Source/Engine/Text/ TextService.hpp	574
ScriptBackend	Text	Source/Engine/Text/ TextShapeSystem.hp p	111
ScriptShaper	Text	Source/Engine/Text/ TextShaper.hpp	207
ShapedGlyph	Text	Source/Engine/Text/ TextShaper.hpp	56
ShapedGlyphHit	Text	Source/Engine/Text/ TextShapedLayout.hp p	153
ShapedLayoutResult	Text	Source/Engine/Text/ TextShapedLayout.hp p	26
ShapedLine	Text	Source/Engine/Text/ TextShapedLayout.hp p	18
StyledAsset	Text	Source/Engine/Text/ TextService.hpp	595
TextAction	Text	Source/Engine/Text/ TextStyle.hpp	61
TextActionKind	Text	Source/Engine/Text/ TextStyle.hpp	51
TextAnimationParam s	Text	Source/Engine/Text/ TextRender.hpp	29
TextAssetBlock	Text	Source/Engine/Text/ TextAssetFormat.hpp	34
TextAssetHeader	Text	Source/Engine/Text/	20

		TextAssetFormat.hpp	
TextAssetRef	Text	Source/Engine/Text/ TextAsset.hpp	14
TextAssetStatus	Text	Source/Engine/Text/ TextAssetFormat.hpp	63
TextAssetView	Text	Source/Engine/Text/ TextAssetFormat.hpp	42
TextBlock	Text	Source/Engine/Text/ TextAsset.hpp	37
TextDirection	Text	Source/Engine/Text/ TextShaper.hpp	21
TextFontStack	Text	Source/Engine/Text/ TextFontStack.hpp	54
TextFormatArg	Text	Source/Engine/Text/ TextFormat.hpp	45
TextFormatStatus	Text	Source/Engine/Text/ TextFormat.hpp	33
TextGradientParams	Text	Source/Engine/Text/ TextRender.hpp	22
TextInlineHit	Text	Source/Engine/Text/ TextLayout.hpp	203
TextInlineKind	Text	Source/Engine/Text/ TextStyle.hpp	73
TextInlineObject	Text	Source/Engine/Text/ TextStyle.hpp	83
TextInlineSprite	Text	Source/Engine/Text/ TextStyle.hpp	99
TextLayoutParams	Text	Source/Engine/Text/ TextLayout.hpp	53
TextLayoutResult	Text	Source/Engine/Text/ TextLayout.hpp	71
TextLine	Text	Source/Engine/Text/ TextLayout.hpp	21
TextMarkupStatus	Text	Source/Engine/Text/ TextMarkup.hpp	16
TextRenderEffects	Text	Source/Engine/Text/ TextRender.hpp	44
TextRenderParams	Text	Source/Engine/Text/ TextRender.hpp	57
TextRenderStats	Text	Source/Engine/Text/ TextRender.hpp	76

TextRenderSystemStats	Text	Source/Engine/Text/TextRenderSystem.hpp	19
TextRun	Text	Source/Engine/Text/TextShaper.hpp	47
TextRuntime	Text	Source/Engine/Text/TextRuntime.hpp	17
TextScript	Text	Source/Engine/Text/TextShaper.hpp	30
TextService	Text	Source/Engine/Text/TextService.hpp	18
TextShapeCache	Text	Source/Engine/Text/TextShapeCache.hpp	59
TextShapeHandle	Text	Source/Engine/Text/TextShapeCache.hpp	19
TextShapeKey	Text	Source/Engine/Text/TextShapeCache.hpp	32
TextShapeParams	Text	Source/Engine/Text/TextShaper.hpp	71
TextShapeResult	Text	Source/Engine/Text/TextShaper.hpp	82
TextShapeSystemStats	Text	Source/Engine/Text/TextShapeSystem.hpp	16
TextShapeView	Text	Source/Engine/Text/TextShapeCache.hpp	50
TextSpan	Text	Source/Engine/Text/TextStyle.hpp	40
TextStore	Text	Source/Engine/Text/TextStore.hpp	19
TextStyle	Text	Source/Engine/Text/TextStyle.hpp	27
TextStyledBlock	Text	Source/Engine/Text/TextMarkup.hpp	30
TextStyleFlags	Text	Source/Engine/Text/TextStyle.hpp	12
VerticalFormEntry	Text	Source/Engine/Text/TextVertical.hpp	16

Engine/UI (101)

Symbol	Kind	Header	Line

AccessibilityEditorSettings	UI	Source/Engine/UI/Accessibility.hpp	250
AccessibilityFlags	UI	Source/Engine/UI/Accessibility.hpp	208
AccessibilityKernel	UI	Source/Engine/UI/Accessibility.hpp	304
AccessibilityOverlay	UI	Source/Engine/UI/AccessibilityOverlay.hpp	30
AccessibilityPresentationSettings	UI	Source/Engine/UI/Accessibility.hpp	233
AccessibilityProfile	UI	Source/Engine/UI/Accessibility.hpp	272
AccessibilitySettings	UI	Source/Engine/UI/Accessibility.hpp	217
AudioSystem	UI	Source/Engine/UI/Accessibility.hpp	26
AvSpeechTts	UI	Source/Engine/UI/Tts.hpp	60
BuildStatus	UI	Source/Engine/UI/PinkUI.hpp	1440
CardBoardComponent	UI	Source/Engine/UI/CardBoard.hpp	49
CardBoardSystem	UI	Source/Engine/UI/CardBoard.hpp	74
ColorMatrix3	UI	Source/Engine/UI/Accessibility.hpp	42
ColorVisionDeficiency	UI	Source/Engine/UI/Accessibility.hpp	34
Dropdown	UI	Source/Engine/UI/PinkUI.hpp	534
FlashGuard	UI	Source/Engine/UI/Accessibility.hpp	59
FocusDirection	UI	Source/Engine/UI/PinkUI.hpp	211
FocusPolicy	UI	Source/Engine/UI/PinkUI.hpp	187
FontAtlas	UI	Source/Engine/UI/FontAtlas.hpp	101
FontAtlasHeader	UI	Source/Engine/UI/FontAtlas.hpp	45
FontAtlasSet	UI	Source/Engine/UI/	167

		FontAtlas.hpp	
FontBuildOptions	UI	Source/Engine/UI/ FontAtlas.hpp	25
FontBuildOptions	UI	Source/Engine/UI/ FontBuilder.hpp	15
FontCoverage	UI	Source/Engine/UI/ FontCoverage.hpp	21
FontFamilyFile	UI	Source/Engine/UI/ FontAtlas.hpp	74
FontFamilyManifest	UI	Source/Engine/UI/ FontFamilyAsset.hpp	17
FontPixelFormat	UI	Source/Engine/UI/ FontAtlas.hpp	37
FontStyle	UI	Source/Engine/UI/ FontAtlas.hpp	65
FrameOutput	UI	Source/Engine/UI/ PinkUI.hpp	1418
GlyphRecord	UI	Source/Engine/UI/ FontAtlas.hpp	80
GridView	UI	Source/Engine/UI/ PinkUI.hpp	424
HyperlinkRange	UI	Source/Engine/UI/ Hypertext.hpp	51
HypertextComponent	UI	Source/Engine/UI/ Hypertext.hpp	32
HypertextFlags	UI	Source/Engine/UI/ Hypertext.hpp	21
HypertextPool	UI	Source/Engine/UI/ Hypertext.hpp	74
HypertextView	UI	Source/Engine/UI/ Hypertext.hpp	67
ITextShaper	UI	Source/Engine/UI/ PinkUI.hpp	33
LayoutCache	UI	Source/Engine/UI/ PinkUI.hpp	736
LazyFontFamily	UI	Source/Engine/UI/ FontFamilyAsset.hpp	101
ListView	UI	Source/Engine/UI/ PinkUI.hpp	560
LocalizationTable	UI	Source/Engine/UI/ Accessibility.hpp	24

LocalizationTable	UI	Source/Engine/UI/ Hypertext.hpp	16
LocalizationTable	UI	Source/Engine/UI/ PinkUI.hpp	39
NarrativeArchetype	UI	Source/Engine/UI/ NarrativeTree.hpp	22
NarrativeEvent	UI	Source/Engine/UI/ Accessibility.hpp	83
NarrativeGroup	UI	Source/Engine/UI/ NarrativeTree.hpp	36
NarrativeLocale	UI	Source/Engine/UI/ NarrativeTree.hpp	16
NarrativeLog	UI	Source/Engine/UI/ Accessibility.hpp	99
NarrativeNode	UI	Source/Engine/UI/ NarrativeTree.hpp	46
NarrativeTree	UI	Source/Engine/UI/ NarrativeTree.hpp	54
OledShiftState	UI	Source/Engine/UI/ PinkUI.hpp	463
PinkSdfDraw	UI	Source/Engine/UI/ PinkSdf.hpp	74
PinkSdfFlags	UI	Source/Engine/UI/ PinkSdf.hpp	24
PinkSdfGpuPrimitive	UI	Source/Engine/UI/ PinkSdf.hpp	51
PinkSdfPrimitive	UI	Source/Engine/UI/ PinkSdf.hpp	33
PinkSdfShape	UI	Source/Engine/UI/ PinkSdf.hpp	15
RadialProgress	UI	Source/Engine/UI/ PinkUI.hpp	584
RetainedFont	UI	Source/Engine/UI/ FontAtlas.hpp	293
SapiTts	UI	Source/Engine/UI/ Tts.hpp	30
ScrollView	UI	Source/Engine/UI/ PinkUI.hpp	491
SlotComponent	UI	Source/Engine/UI/ CardBoard.hpp	27
SlotFlags	UI	Source/Engine/UI/	15

		CardBoard.hpp	
SlotLayout	UI	Source/Engine/UI/ CardBoard.hpp	65
SoundCueCategory	UI	Source/Engine/UI/ SoundCues.hpp	26
SoundCueDef	UI	Source/Engine/UI/ SoundCues.hpp	62
SoundCueRegistry	UI	Source/Engine/UI/ SoundCues.hpp	71
SoundTrigger	UI	Source/Engine/UI/ Accessibility.hpp	128
StateTrack	UI	Source/Engine/UI/ Accessibility.hpp	411
SubtitleLine	UI	Source/Engine/UI/ Accessibility.hpp	139
SubtitleQueue	UI	Source/Engine/UI/ Accessibility.hpp	163
SystemContext	UI	Source/Engine/UI/ PinkUI.hpp	28
TabBar	UI	Source/Engine/UI/ PinkUI.hpp	515
TextAlign	UI	Source/Engine/UI/ PinkUI.hpp	222
TextShapeParams	UI	Source/Engine/UI/ PinkUI.hpp	34
TextVerticalAlign	UI	Source/Engine/UI/ PinkUI.hpp	229
TimedSoundCue	UI	Source/Engine/UI/ Accessibility.hpp	152
TooltipPlacer	UI	Source/Engine/UI/ PinkUI.hpp	595
TtsBackend	UI	Source/Engine/UI/ Accessibility.hpp	27
TtsBackend	UI	Source/Engine/UI/ Tts.hpp	19
TtsPriority	UI	Source/Engine/UI/ Tts.hpp	11
UiPalette	UI	Source/Engine/UI/ UiTheme.hpp	41
UiPaletteSlot	UI	Source/Engine/UI/ UiTheme.hpp	18

UiThemeAsset	UI	Source/Engine/UI/UiTheme.hpp	46
VirtualFileSystem	UI	Source/Engine/UI/FontAtlas.hpp	19
VoiceCaptionEvent	UI	Source/Engine/UI/Accessibility.hpp	193
VoiceCaptionProvider	UI	Source/Engine/UI/Accessibility.hpp	201
WidgetCanvas	UI	Source/Engine/UI/Accessibility.hpp	30
WidgetCanvas	UI	Source/Engine/UI/PinkUI.hpp	290
WidgetCanvas	UI	Source/Engine/UI/PinkUI.hpp	483
WidgetCanvas	UI	Source/Engine/UI/PinkUI.hpp	607
WidgetCanvasSystem	UI	Source/Engine/UI/PinkUI.hpp	1453
WidgetComponent	UI	Source/Engine/UI/PinkUI.hpp	251
WidgetComponentFlags	UI	Source/Engine/UI/PinkUI.hpp	263
WidgetDesc	UI	Source/Engine/UI/PinkUI.hpp	309
WidgetFlags	UI	Source/Engine/UI/PinkUI.hpp	145
WidgetHandle	UI	Source/Engine/UI/PinkUI.hpp	80
WidgetHyperlink	UI	Source/Engine/UI/PinkUI.hpp	300
WidgetKind	UI	Source/Engine/UI/PinkUI.hpp	99
WidgetLayoutAsset	UI	Source/Engine/UI/WidgetLayoutAsset.hpp	15
WidgetPaintContext	UI	Source/Engine/UI/PinkUI.hpp	291
WidgetPaintContext	UI	Source/Engine/UI/PinkUI.hpp	391

Engine/Video (20)

Symbol	Kind	Header	Line
AssetDatabase	Video	Source/Engine/ Video/ VideoStream.hpp	23
ClusterScan	Video	Source/Engine/ Video/ WebmDemuxer.hpp	108
ContainerKind	Video	Source/Engine/ Video/ WebmDemuxer.hpp	150
DecodeStatus	Video	Source/Engine/ Video/VideoTypes.hpp	29
DemuxStatus	Video	Source/Engine/ Video/ WebmDemuxer.hpp	52
Element	Video	Source/Engine/ Video/ WebmDemuxer.hpp	85
FrameBuffer	Video	Source/Engine/ Video/ VideoPlayer.hpp	118
FrameSlot	Video	Source/Engine/ Video/ VideoPlayer.hpp	125
MappedFile	Video	Source/Engine/ Video/ VideoPlayer.hpp	22
RHI	Video	Source/Engine/ Video/ VideoPlayer.hpp	27
VideoPlayer	Video	Source/Engine/ Video/ VideoPlayer.hpp	33
VideoStreamCompo nent	Video	Source/Engine/ Video/ VideoStream.hpp	31
VideoStreamFlags	Video	Source/Engine/ Video/VideoTypes.hpp	20
VideoSystem	Video	Source/Engine/ Video/ VideoStream.hpp	49
VirtualFileSystem	Video	Source/Engine/ Video/	20

		VideoStream.hpp	
Vp9Backend	Video	Source/Engine/ Video/ Vp9Backend.hpp	30
WebmDemuxer	Video	Source/Engine/ Video/ WebmDemuxer.hpp	60
WebmFrame	Video	Source/Engine/ Video/ WebmDemuxer.hpp	38
WebmTrackInfo	Video	Source/Engine/ Video/ WebmDemuxer.hpp	14
YuvFrameView	Video	Source/Engine/ Video/ Vp9Backend.hpp	12

Engine/Weather (12)

Symbol	Kind	Header	Line
ClothSystem	Weather	Source/Engine/ Weather/Weather.hpp	13
WeatherAtmosphere Pass	Weather	Source/Engine/ Weather/ WeatherPreconciliated .hpp	128
WeatherContextCom ponent	Weather	Source/Engine/ Weather/Weather.hpp	32
WeatherDesc	Weather	Source/Engine/ Weather/Weather.hpp	91
WeatherFlags	Weather	Source/Engine/ Weather/Weather.hpp	19
WeatherGpuConstan ts	Weather	Source/Engine/ Weather/ WeatherPreconciliated .hpp	46
WeatherGpuResourc es	Weather	Source/Engine/ Weather/ WeatherPreconciliated .hpp	18
WeatherGpuStats	Weather	Source/Engine/ Weather/ WeatherPreconciliated .hpp	68

WeatherPreconciliatedDesc	Weather	Source/Engine/Weather/WeatherPreconciliated.hpp	32
WeatherPreconciliatedSystem	Weather	Source/Engine/Weather/WeatherPreconciliated.hpp	78
WeatherSystem	Weather	Source/Engine/Weather/Weather.hpp	99
WeatherWindField	Weather	Source/Engine/Weather/Weather.hpp	75

40. Appendix B - macros and platform defines

Macro	Header	Line
FBY_AOT_EXPORT	Source/Engine/Script/AotAbi.hpp	310
FBY_AOT_EXPORT	Source/Engine/Script/AotAbi.hpp	312
FBY_ASSERT	Source/Engine/Core/Assert.hpp	13
FBY_ASSERT	Source/Engine/Core/Assert.hpp	31
FBY_ASSERT_MSG	Source/Engine/Core/Assert.hpp	22
FBY_ASSERT_MSG	Source/Engine/Core/Assert.hpp	32
FBY_DEFINE_PLUGIN	Source/Engine/Core/Plugin.hpp	56
FBY_DETAIL_CONCAT	Source/Engine/Core/Definitions.hpp	261
FBY_DETAIL_CONCAT_INNER	Source/Engine/Core/Definitions.hpp	260
FBY_FIELD	Source/Engine/Core/Mirror.hpp	279
FBY_FLUID_HAS_MDSPAN	Source/Engine/Physics/Fluid.hpp	16
FBY_FLUID_HAS_MDSPAN	Source/Engine/Physics/Fluid.hpp	18
FBY_HAS_MDSPAN	Source/Engine/ECS/	17

	Skeleton.hpp	
FBY_HAS_MDSPAN	Source/Engine/ECS/ Skeleton.hpp	19
FBY_HAS_STATIC_REFLECTION	Source/Engine/Core/ Mirror.hpp	97
FBY_HAS_STATIC_REFLECTION	Source/Engine/Core/ Mirror.hpp	100
FBY_HAS_STATIC_REFLECTION	Source/Engine/Core/ Mirror.hpp	104
FBY_HAS_STD_FLAT_MAP	Source/Engine/Core/ FlatMap.hpp	17
FBY_HAS_STD_FLAT_MAP	Source/Engine/Core/ FlatMap.hpp	19
FBY_PLATFORM_ABI_VERSION	Source/Plugins/ PlatformService/PlatformAPI.h	11
FBY_PLATFORM_CALL	Source/Plugins/ PlatformService/PlatformAPI.h	15
FBY_PLATFORM_CALL	Source/Plugins/ PlatformService/PlatformAPI.h	18
FBY_PLATFORM_EXPORT	Source/Plugins/ PlatformService/PlatformAPI.h	14
FBY_PLATFORM_EXPORT	Source/Plugins/ PlatformService/PlatformAPI.h	17
FBY_PLUGIN_ABI_VERSION	Source/Engine/Core/ PluginAPI.h	28
FBY_PLUGIN_API_H	Source/Engine/Core/ PluginAPI.h	18
FBY_PLUGIN_COMPONENT	Source/Engine/Core/ Plugin.hpp	53
FBY_PLUGIN_ENTRY	Source/Engine/Core/ PluginAPI.h	115
FBY_PLUGIN_EXPORT	Source/Engine/Core/ PluginAPI.h	31
FBY_PLUGIN_EXPORT	Source/Engine/Core/ PluginAPI.h	33
FBY_PLUGIN_EXPORT	Source/Engine/Core/ PluginAPI.h	35
FBY_PLUGIN_INFO_SYMBOL	Source/Engine/Core/ PluginAPI.h	107
FBY_PLUGIN_LOAD_SYMBOL	Source/Engine/Core/ PluginAPI.h	108

FBY_PLUGIN_UNLOAD_SYMBOL	Source/Engine/Core/PluginAPI.h	109
FBY_PROJECT_ASSETS	Source/Engine/Assets/ProjectScaffold.hpp	123
FBY_REFLECT_COMPONENT	Source/Engine/Core/Mirror.hpp	286
FBY_REFLECT_COMPONENT_EMPTY	Source/Engine/Core/Mirror.hpp	298
FBY_REGISTER_TYPE	Source/Engine/Core/Definitions.hpp	264
FBY_RELEVANCE_HAS_MDSPAN	Source/Engine/Network/RelevanceGrid.hpp	19
FBY_RELEVANCE_HAS_MDSPAN	Source/Engine/Network/RelevanceGrid.hpp	21
FBY_RESTRICT	Source/Engine/Core/Definitions.hpp	26
FBY_RESTRICT	Source/Engine/Core/Definitions.hpp	28
FBY_SDF_SHAPE_FIELD	Source/Engine/Physics/SdfPrimitives.hpp	172
FBY_TYPE_NAME	Source/Engine/Core/Definitions.hpp	280
FBY_UI_HAS_MDSPAN	Source/Engine/UI/PinkUI.hpp	21
FBY_UI_HAS_MDSPAN	Source/Engine/UI/PinkUI.hpp	23
FBY_UNREACHABLE	Source/Engine/Core/Assert.hpp	45
FBY_VERIFY	Source/Engine/Core/Assert.hpp	36
FBY_VK_DECLARE	Source/Engine/RHI/Vulkan/VulkanLoader.hpp	117
FBY_VK_DEVICE_FUNCTIONS	Source/Engine/RHI/Vulkan/VulkanLoader.hpp	36
FBY_VK_GLOBAL_FUNCTIONS	Source/Engine/RHI/Vulkan/VulkanLoader.hpp	15
FBY_VK_INSTANCE_FUNCTIONS	Source/Engine/RHI/Vulkan/VulkanLoader.hpp	20
FORCEINLINE	Source/Engine/Core/Definitions.hpp	17
FORCEINLINE	Source/Engine/Core/Definitions.hpp	19
FORCEINLINE	Source/Engine/Core/	21

	Definitions.hpp	